movtobase

База знаний по системному программированию

C# Low Level + Zero Allocations

GC изнутри · Span и Memory · Unsafe и MemoryMarshal · Структура NativeAOT · TCP-сервер без аллокаций · System.IO.Pipelines

Zig 0.17.0-dev с нуля

8 уроков: hello world, переменные и типы, строки, ошибки, optionals, циклы, функции, структуры

C# Low Level + Zero Allocations

Шесть разборов о том, как .NET работает под капотом и как писать код, который не будит сборщик мусора в горячем пути.

Материалы читаются по порядку — каждый следующий опирается на предыдущие.

#	Материал	О чём
1	GC изнутри	Поколения, кучи, zeroing, write barrier, regions: почему аллокации дороги на самом деле
2	Span и Memory	Взгляд на память без копирования: стек, куча, `MemoryManager<T>`
3	Unsafe и MemoryMarshal	Адреса, `ref T`, интринсики и типобезопасная прослойка
4	Структура NativeAOT	Как C# превращается в один нативный бинарник: рантайм, объектная модель, метаданные
5	TCP-сервер без аллокаций	`SocketAsyncEventArgs`, IOCP и epoll, ноль байт из кучи в горячем пути
6	System.IO.Pipelines	Протокол поверх TCP без копирований: `Pipe`, `SequenceReader`, backpressure

Как читать

В каждой статье есть разделы «Ловушки», «Частые ошибки на примерах» и «Что в итоге» — не пропускай их, там самая мякоть.

GC изнутри: поколения, кучи и как не будить сборщик

Мы весь цикл говорили "не аллоцируй в горячем пути", и пора объяснить, почему. Аллокация в .NET сама по себе почти бесплатна: это несколько инструкций и сдвиг указателя. Дорого стоит не она, а момент, когда выделенный кусок кончился. Тогда поток останавливается и зовет сборщик. Понимание того, как устроен этот механизм, отличает код, который просто работает, от кода, который держит хвосты latency.

Разберем по исходникам `src/coreclr/gc` и `src/coreclr/System.Private.CoreLib/src/System/GC.CoreCLR.cs`.

На пальцах: что вообще делает сборщик

Прежде чем читать код, полезно понять задачу, которую решает GC.

В управляемых языках вы не освобождаете память руками. Вы создаете объекты, а когда они становятся не нужны, их должен кто-то убрать. Этим занимается **сборщик мусора** (garbage collector). Его работа состоит из трех шагов.

Определить, что живо. Объект считается живым, если до него можно добраться от **корней** (roots): локальных переменных на стеке, статических полей, регистров процессора. Все, до чего можно дотянуться по ссылкам от корней, называется **достижимым**. Остальное - мусор.

Освободить мусор. Память от недостижимых объектов возвращается в оборот.

Прибрать. Чтобы память не превратилась в швейцарский сыр из дырок, сборщик может **сжать** (compact) кучу: сдвинуть живые объекты вплотную друг к другу. Это называется компактировкой.

Проблема в том, что для первых двух шагов нужно точно знать, где лежат ссылки. Если сборщик ошибется и примет число за ссылку, он утащит мусорный указатель. Поэтому .NET использует **точное** отслеживание: JIT для каждого метода генерирует карту, где на каждом участке кода указано, какие слоты содержат ссылки.

И главное. Чтобы безопасно двигать объекты, сборщик должен на время **остановить все потоки** приложения. Эта остановка называется **паузой**. Вот почему мы весь цикл боремся с аллокациями: меньше мусора - реже сборки - короче суммарные паузы.

Отсюда и вся конструкция с поколениями. Идея в том, чтобы не проверять всю кучу каждый раз. Большинство объектов умирают молодыми, поэтому дешевле собирать только маленькую молодую область и не трогать старую. Так появляются gen0, gen1 и gen2.

Словарь терминов

Куча (heap) - область памяти, где живут объекты.

Аллокация - выделение нового объекта в куче.

GC (Garbage Collector) - подсистема, которая находит недостижимые объекты и освобождает их память.

Корень (root) - точка, от которой отслеживается достижимость: локальная переменная, статическое поле, регистр.

Достижимость - свойство объекта, до которого можно добраться от корня по цепочке ссылок.

Поколение (generation) - gen0, gen1 или gen2. Чем выше номер, тем дольше объект живет.

Эфемерный сегмент - область, где вместе живут gen0 и gen1.

SOH (Small Object Heap) - куча обычных, небольших объектов.

LOH (Large Object Heap) - куча объектов от 85000 байт. Не компактируется по умолчанию.

POH (Pinned Object Heap) - куча для закрепленных объектов, которые сборщик не двигает.

Пауза (pause) - остановка потоков приложения на время сборки.

Компактировка (compaction) - сдвиг живых объектов, чтобы убрать дырки в памяти.

Фрагментация - разбиение памяти на мелкие куски из-за дырок между объектами.

Write barrier - код, который выполняется при записи ссылки в объект и помечает card как грязную.

Card table - битовая карта, отмечающая участки кучи, в которые недавно писали ссылки.

Регион - кусок кучи фиксированного размера, который можно отдать любому поколению.

Server GC / Workstation GC - два режима сборщика: многопоточный для серверов и однопоточный для клиентских приложений.

Бюджет (budget) - сколько байтов можно выделить в поколении до запуска сборки.

Финализатор - метод, который вызывается перед освобождением объекта, обычно для нативных ресурсов.

Пиннинг - запрет сборщику перемещать объект.

Latency mode - режим сборщика, управляющий частотой и типом сборок.

Memory pressure - сигнал сборщику, что нативная память занята и стоит прибраться активнее.

Аллокация - это сдвиг указателя

У каждого потока есть свой контекст аллокации. Вот его структура (`gcinterface.h:201`):

```
struct gc_alloc_context
{
    uint8_t*       alloc_ptr;
    uint8_t*       alloc_limit;
```

```
    int64_t        alloc_bytes;     // сколько SOH выделил этот контекст
    int64_t        alloc_bytes_uoh; // сколько UOH выделил этот контекст
    void*          gc_reserved_1;
    void*          gc_reserved_2;
    int            alloc_count;
};
```

Комментарий прямо над ней объясняет смысл (`gcinterface.h:198`):

```
// The allocation context must be known to the VM for use in the allocation
// fast path and known to the GC for performing the allocation. Every Thread
// has its own allocation context that it hands to the GC when allocating.
```

`alloc_ptr` - указатель на следующий свободный байт, `alloc_limit` - где заканчивается текущий кусок. Быстрый путь аллокации объекта выглядит так:

1. загрузить `alloc_ptr`;
2. прибавить размер объекта с выравниванием;
3. сравнить с `alloc_limit`;
4. если влезло, записать новый `alloc_ptr` и указатель на method table;
5. вернуть объект.

Вот этот процесс на схеме. Пусть нужно выделить объект размером 24 байта:

```
до аллокации:
  alloc_ptr   -------> [ free ............................. ]
                        ^                                  ^
                    alloc_ptr                          alloc_limit

после аллокации:
  alloc_ptr   -------> [ obj ][ free ...................... ]
                        ^      ^                           ^
                    вернули  alloc_ptr                  alloc_limit
```

Объект размером 24 байта занял первые 24 байта свободного куска, `alloc_ptr` сдвинулся на 24 вперед. Все, что произошло, - это одна операция сложения и одна запись.

Ни мьютексов, ни поиска свободного блока. Именно это JIT встраивает прямо в машинный код. Когда места не хватило, поток уходит в медленный путь `Alloc` (`gchelpers.cpp:270`), который заканчивается вызовом GC (`gchelpers.cpp:327`):

```
retVal = GCHeapUtilities::GetGCHeap()->Alloc(pAllocContext, size, flags);
```

Комментарий рядом (`gchelpers.cpp:349`) описывает иерархию:

```
// There are only two ways to allocate an object.
//     * Call optimized helpers that were generated on the fly. This is how JIT compiled
code does most
//         allocations, however they fall back code:Alloc, when for all but the most common
code paths.
```

```
//     * Call code:Alloc - When the jit helpers fall back, or we do allocations within the
runtime code itself
```

Вот и весь секрет дешевизны аллокации. Пока `alloc_ptr < alloc_limit`, мы просто двигаем указатель. Проблема начинается, когда контекст исчерпан: потоку нужен новый кусок, а GC решает, дать ли его без сборки или сначала прибраться.

Поколения: гипотеза о молодых объектах

GC строится на простом наблюдении: большинство объектов умирают молодыми. Поэтому память делится на поколения. Gen0 и gen1 живут вместе в одном эфемерном сегменте, gen2 отдельно. Границы эфемерного сегмента хранятся в полях `ephemeral_low` и `ephemeral_high` (`gc.cpp:906`):

```
VOLATILE(uint8_t*)    gc_heap::ephemeral_low;
VOLATILE(uint8_t*)    gc_heap::ephemeral_high;
```

Зачем это разделение. Когда кончается бюджет gen0, GC собирает только gen0: обходит живые объекты в маленьком сегменте и переносит их в gen1. Это быстро, потому что gen0 мал. Объект, переживший сборку, получает шанс пожить в gen1. Переживший и gen1 уезжает в gen2, где его будут собирать уже в полную, дорогую сборку.

Вот как это выглядит в памяти. Gen0 и gen1 сидят в одном эфемерном сегменте, gen2 живет отдельно:

```
эфемерный сегмент
+----------------------+----------------------+
|        gen0          |        gen1          |
+----------------------+----------------------+
   новые объекты         пережили одну сборку
   собирается чаще всего  собирается при gen1 GC

gen2 (отдельные сегменты или регионы)
+-------------------------------------------------+
|  объекты, пережившие две сборки и больше        |
|  собирается только при полной сборке            |
+-------------------------------------------------+
```

Путь объекта при сборках:

```
new -> gen0 --(пережил gen0 GC)--> gen1 --(пережил gen1 GC)--> gen2
       |                              |                          |
     умер молодым              пожил дольше              живет долго
```

Бюджет - это сколько байтов можно выделить в поколении до запуска следующей сборки. Когда `alloc_ptr` в контексте потока доходит до предела, поток просит новый кусок, и GC смотрит: бюджет gen0 еще не исчерпан? Тогда просто дает память. Исчерпан? Тогда запускает сборку gen0. Именно поэтому аллокации "приближают" сборку: каждая из них уменьшает оставшийся бюджет.

Практический вывод: каждая лишняя аллокация в горячем пути не просто занимает память, она наполняет gen0 и приближает сборку. И если объект доживает до gen1 или gen2, стоимость его

существования растет. Поэтому пулы, переиспользование и `stackalloc` бьют сразу по двум целям: не мусорят в gen0 и не дают объектам выживать.

Три кучи: SOH, LOH, POH

В .NET не одна куча, а несколько, и они ведут себя по-разному.

SOH (Small Object Heap) - обычные объекты. Именно про него мы говорили выше: gen0, gen1, gen2, компактирующая сборка, объекты двигаются.

LOH (Large Object Heap) - объекты от 85000 байт. Вот эта константа (`gcinterface.h:246`):

```
#define LARGE_OBJECT_SIZE ((size_t)(85000))
```

LOH не компактируется по умолчанию, потому что двигать большие объекты дорого. Отсюда фрагментация: массив на 100 КБ освободился, на его месте дырка, а новый массив на 120 КБ может туда не влезть, и куча растет. Компактировать LOH можно вручную через `GCSettings.LargeObjectHeapCompactionMode`, но это дорого.

POH (Pinned Object Heap) - отдельная куча для объектов, которые нельзя двигать. Она появилась именно потому, что пинить обычные объекты больно: закрепленный объект в SOH мешает компактировке и фрагментирует кучу. В POH объекты не двигаются по определению, и сборщик знает об этом заранее. Как туда попасть:

```
byte[] buffer = GC.AllocateUninitializedArray<byte>(64 * 1024, pinned: true);
```

Флаг `GC_ALLOC_PINNED_OBJECT_HEAP` выставляется в `AllocateUninitializedArray` (`GC.CoreCLR.cs:860`):

```
GC_ALLOC_FLAGS flags = GC_ALLOC_FLAGS.GC_ALLOC_ZEROING_OPTIONAL;
if (pinned)
{
    flags |= GC_ALLOC_FLAGS.GC_ALLOC_PINNED_OBJECT_HEAP;
}
```

А обычный `GCHandle.Alloc(obj, GCHandleType.Pinned)` пинит объект в его же куче и добавляет работу GC. Для долгоживущих буферов под interop POH лучше.

Сводка по трем кучам:

Куча	Что там	Компактируется	Типичный размер объекта
SOH	обычные объекты	да	меньше 85000 байт
LOH	большие объекты	нет (по умолчанию)	85000 байт и больше
POH	закрепленные объекты	не двигаются вообще	любой

Практический пример, где это важно. Возьмем буфер для сетевого обмена:

```
// плохо: 100 КБ уедет в LOH, где фрагментация
byte[] big = new byte[100_000];

// лучше: разбить на два буфера под порогом
byte[] part1 = new byte[50_000];
byte[] part2 = new byte[50_000];

// или взять из пула, тогда массив переиспользуется
byte[] pooled = ArrayPool<byte>.Shared.Rent(100_000);

// для interop, где нужен стабильный адрес: сразу в POH
byte[] pinned = GC.AllocateUninitializedArray<byte>(64 * 1024, pinned: true);
```

`ArrayPool` здесь не отменяет LOH: если рент вернет массив больше 85000, он все равно уедет в LOH. Но пул переиспользует его между запросами, поэтому новая память не выделяется и фрагментация не растет так быстро.

Zeroing: скрытая стоимость new

`new byte[1_000_000]` не только выделяет память, но и зануляет ее. Для большого буфера это лишние мегабайты записи, которые вы все равно перезапишете данными. `GC.AllocateUninitializedArray` умеет это пропускать через флаг `GC_ALLOC_ZEROING_OPTIONAL` (`GC.CoreCLR.cs:857`).

Но там есть нюанс (`GC.CoreCLR.cs:832`):

```
if (!pinned)
{
    if (RuntimeHelpers.IsReferenceOrContainsReferences<T>())
    {
        return new T[length];
    }

#if !DEBUG
    // small arrays are allocated using `new[]` as that is generally faster.
    if (length < 2048 / sizeof(T))
    {
        return new T[length];
    }
#endif
}
```

Если тип содержит ссылки, зануление обязательно: иначе GC увидит мусорные ссылки и упадет. Если массив маленький (меньше 2048 байт), обычный `new T[]` оказывается быстрее, чем поход в GC за неинициализированной памятью. Так что `AllocateUninitializedArray` имеет смысл для крупных буферов из простых типов, где зануление реально заметно.

Write barrier и card table

Тут начинается самое интересное. Как GC узнает, какие старые объекты ссылаются на молодые, не сканируя всю gen2 на каждой сборке gen0? Через card table и write barrier.

Куча логически нарезана на небольшие участки, cards. Размер card выводится из размера страницы GC и ширины слова (`gcpriv.h:6778`). На каждую card есть бит в card table. Когда код записывает ссылку на объект, срабатывает write barrier: он помечает card, в которую попала запись, как грязную. В исходниках это видно по `card_bundle_set` и работе с `card_bundle_table` (`card_table.cpp:56`):

```
void gc_heap::card_bundle_set (size_t cardb)
{
    uint32_t bits = (1 << card_bundle_bit (cardb));
    set_bundle_bits (&card_bundle_table [card_bundle_word (cardb)], bits);
}
```

При сборке gen0 GC не сканирует всю gen2. Он смотрит только грязные cards и находит ссылки оттуда. `card_bundle` - это второй уровень битовой карты, который позволяет пропускать целые чистые диапазоны одним махом, не проверяя каждую card.

Посмотрим на примере, для чего это. Пусть есть старый объект в gen2, который ссылается на молодой:

```
class Node
{
    public Node? Next; // ссылочное поле
}

Node oldNode = GetLongLivedNode(); // живет в gen2
Node young = new Node();           // создан только что, в gen0

oldNode.Next = young; // вот здесь срабатывает write barrier
```

При сборке gen0 нужно понять, не ссылается ли кто-то из старой кучи на молодой объект. Без card table пришлось бы просканировать всю gen2. С card table достаточно посмотреть только грязные участки:

```
gen2 в памяти, разбита на cards:
+-------+-------+-------+-------+-------+
| card0 | card1 | card2 | card3 | card4 |
| clean | dirty | clean | clean | dirty |
+-------+-------+-------+-------+-------+
          ^                         ^
     сюда писали               сюда писали
   oldNode.Next = young
```

GC сканирует только `card1` и `card4`, находит ссылку на молодой объект в `card1` и учитывает ее. Остальная gen2 не трогается.

Как это выглядит на уровне кода, который генерирует JIT:

```
// что вы пишете
oldNode.Next = young;

// что примерно делает JIT (упрощенно)
oldNode.Next = young;
CardTable.MarkCard(ref oldNode.Next); // write barrier помечает card
```

Карта делится не побайтово, а по участкам: один бит на card. Поэтому пометить card дешево, а на сборке можно пропустить целые чистые диапазоны. `card_bundle` добавляет еще один уровень: один

бит на группу cards, чтобы быстро находить грязные области, не проверяя каждый бит.

Практический вывод: write barrier не бесплатен. Он срабатывает на каждую запись ссылки в объект. Чем меньше у вас ссылочных полей и чем реже вы их меняете, тем меньше работы для барьера. Структуры без ссылок (blittable) вообще не требуют барьера, и это еще одна причина любить их в горячем пути.

Regions: новое поколение кучи

Долгое время куча была набором сегментов, и поколения жили в фиксированных местах. В современных .NET на 64-битных платформах включены regions (`gcpriv.h:153`):

```
#define USE_REGIONS
```

Инварианты описаны прямо в комментарии (`gcpriv.h:137`):

```
// Regions invariants -
//
// + each generation consists of 1+ regions.
// + a region is in a contiguous address range; different regions could have
//   gaps inbetween.
// + a region cannot contain more than one generation.
//
// This means any empty regions can be freely used for any generation.
```

Смысл в том, что регион - это кусок фиксированного размера, который можно отдать любому поколению. Освободился регион в gen2 - его можно отдать под gen0. Это убирает старую проблему, когда эфемерный сегмент заполнен мусором и его приходится расширять, а gen2 сидит на своих сегментах. С регионами GC может перекраивать кучу гораздо гибче. Плюс только с регионами работает динамическое число куч в server GC (`gcpriv.h:159`).

Вот разница на схеме. Со старыми сегментами поколения были привязаны к своим областям, а регионы можно перетасовывать:

```
до: регионы распределены по поколениям
+--------+--------+--------+--------+--------+
| gen0   | gen0   | gen1   | gen2   | gen2   |
+--------+--------+--------+--------+--------+

после сборки: освободившийся регион gen2 отдан gen0
+--------+--------+--------+--------+--------+
| gen0   | gen0   | gen0   | gen1   | gen2   |
+--------+--------+--------+--------+--------+
                    ^
          был gen2, стал gen0
```

Правило из комментария "a region cannot contain more than one generation" означает, что внутри одного региона не может быть смеси поколений. Зато сам регион можно целиком перевести из одного поколения в другое. Это делает кучу гораздо более гибкой: GC не привязан к тому, что эфемерное поколение должно лежать в одном конкретном сегменте.

Server GC против workstation

Есть два режима. Workstation GC - одна куча, сборки на потоке, который вызвал аллокацию, плюс фоновые сборки. Подходит для клиентских приложений, где важна отзывчивость и память.

Server GC - по куче на каждый процессор, отдельные потоки сборки для каждой кучи, параллельный маркинг и компактировка. Для серверов, которые держат много объектов и много ядер, это дает гораздо большую пропускную способность. Включается так:

```
<ServerGarbageCollection>true</ServerGarbageCollection>
```

или переменной окружения `DOTNET_gcServer=1`. Но у server GC выше базовое потребление памяти: у каждой кучи свои структуры. На маленьком контейнере это может быть неочевидно дорого.

Сравним режимы:

Свойство	Workstation GC	Server GC
Куч	одна	по одной на процессор
Потоки сборки	поток, вызвавший аллокацию	отдельный поток на каждую кучу
Параллелизм	ограниченный	маркинг и компактировка параллельно
Память	меньше	больше, у каждой кучи свои структуры
Пропускная способность	ниже	выше на многопоточной нагрузке
Для чего	клиентские приложения, контейнеры с малым лимитом	серверы с большим числом ядер и объектов

Отдельно про фоновые сборки. В workstation GC gen2 собирается в фоне, насколько это возможно, чтобы не морозить приложение надолго. Server GC тоже поддерживает фоновые сборки, но с отдельными потоками на кучу.

Как выбирать. Если у вас сервис на 8+ ядрах, который держит много объектов, server GC почти всегда выигрывает по пропускной способности. Если это маленький контейнер с лимитом памяти 256 МБ, server GC может съесть заметную долю на свои структуры, и workstation окажется экономнее.

Как не будить сборщик

Соберем практические правила.

Не аллоцируйте в горячем пути. Пул буферов, переиспользование объектов, `stackalloc`, `Span<T>`. Это то, о чем мы говорили весь цикл.

Держитесь под 85000 байт. Массив на 90 КБ уедет в LOH, где нет компактировки и есть фрагментация. Иногда лучше разбить буфер на несколько меньших.

Пиньте через POH, а не через GCHandle. `GC.AllocateUninitializedArray<byte>(size, pinned: true)` не фрагментирует SOH и не мешает компактировке.

Не зануляйте то, что перезапишете. Для больших буферов из простых типов `GC.AllocateUninitializedArray` экономит реальную работу.

Избегайте финализаторов. Объект с финализатором переживает как минимум одну сборку, попадает в очередь финализатора и добавляет работу финализаторному потоку. Если финализатор не нужен, не заводите его. Если нужен, вызывайте `GC.SuppressFinalize` после детерминированной очистки.

Для критичного по latency окна есть `NoGCRegion`. Если у вас есть участок, где сборка недопустима, например обработка пакета в торговом ядре, можно попросить GC не собирать:

```
if (GC.TryStartNoGCRegion(16 * 1024 * 1024))
{
    try
    {
        // работа, в которой недопустима пауза
    }
    finally
    {
        GC.EndNoGCRegion();
    }
}
```

Это работает только если вы заранее зарезервировали достаточно памяти, иначе регион молча завершится и вы получите `InvalidOperationException` при `EndNoGCRegion` (`GC.CoreCLR.cs:596`). Внутри региона аллокации продолжаются, но GC не запускается, пока вы не превысите бюджет.

Для длительных пауз есть `GCSettings.LatencyMode`. `SustainedLowLatency` заставляет GC откладывать полные блокирующие сборки (`GCSettings.cs:32`). Плата - потенциально большая куча и более резкая сборка, когда она все же случится.

Как смотреть, что делает GC

Не угадывайте, измеряйте.

```
GCMemoryInfo info = GC.GetGCMemoryInfo();
Console.WriteLine($"heap: {info.HeapSizeBytes}, committed: {info.TotalCommittedBytes}");
Console.WriteLine($"gen0: {GC.CollectionCount(0)}, gen1: {GC.CollectionCount(1)}, gen2:
{GC.CollectionCount(2)}");
Console.WriteLine($"allocated total: {GC.GetTotalAllocatedBytes()}");
```

Живые метрики:

```
dotnet-counters monitor --counters System.Runtime --process-id <PID>
```

Ключевые счетчики: `alloc-rate`, `gen-0-gc-count`, `gen-1-gc-count`, `gen-2-gc-count`, `time-in-gc`, `% time in GC`, `gen-0-size`. Если `alloc-rate` не падает после прогрева, где-то есть постоянный источник мусора.

Полезные переменные окружения:

- `DOTNET_gcServer=1` - server GC.
- `DOTNET_GCHeapHardLimit=0x40000000` - ограничить кучу (актуально в контейнерах).
- `DOTNET_GCConserveMemory=5` - агрессивнее отдавать память ОС, ценой CPU.
- `DOTNET_GCLOHThreshold=...` - изменить порог LOH (осторожно, меняет поведение GC).

Ловушки

`GC.Collect` в продакшене почти всегда ошибка. Вы не знаете лучше сборщика, когда собирать. Единственное разумное применение - диагностика или явная точка в тестах. Ручная сборка полной кучи в горячем пути превращает плавную работу в пилу из пауз.

`GC.KeepAlive` не запускает сборку. Это барьер для JIT: он говорит "считай объект живым до этой точки". Нужен для interop, где нативный код держит указатель на управляемый объект, а JIT не видит причины держать его живым. К управлению GC это отношения не имеет.

Финализатор не освобождает память. Он выполняется на отдельном потоке через неопределенное время после того, как объект стал недостижим. Управляемая память освобождается только сборкой. Финализатор нужен для нативных ресурсов, и то обычно `SafeHandle` справляется лучше.

`TryStartNoGCRegion` требует места. Если свободной памяти меньше, чем вы просите, метод вернет `false`. Если во время региона память кончится, регион завершится, и `EndNoGCRegion` бросит исключение. Не используйте это как серебряную пулю.

POH не бесконечен. Долгоживущие запиненные буферы в POH остаются там, пока на них есть ссылки. Если вы запинили мегабайт и забыли отпустить, GC его не соберет и не сдвинет. Пул буферов в POH надо чистить.

Частые ошибки на примерах

Соберу правила из практики в виде "было / стало".

Аллокация в горячем цикле.

```
// было: массив на каждую итерацию
for (int i = 0; i < requests.Count; i++)
{
    byte[] temp = new byte[4096];
    Process(requests[i], temp);
}
```

```
// стало: буфер берется один раз из пула
byte[] temp = ArrayPool<byte>.Shared.Rent(4096);
try
{
    for (int i = 0; i < requests.Count; i++)
        Process(requests[i], temp);
}
finally
{
    ArrayPool<byte>.Shared.Return(temp);
}
```

Крупный буфер уезжает в LOH.

```
// было: 100 КБ в LOH, фрагментация
byte[] big = new byte[100_000];

// стало: либо под порогом, либо из пула
byte[] pooled = ArrayPool<byte>.Shared.Rent(100_000);
```

Пиннинг обычного массива.

```
// было: GCHandle на каждый interop-вызов
var handle = GCHandle.Alloc(buffer, GCHandleType.Pinned);
try
{
    NativeCall(handle.AddrOfPinnedObject());
}
finally
{
    handle.Free();
}

// стало: буфер в POH, пин почти бесплатен
byte[] pinned = GC.AllocateUninitializedArray<byte>(4096, pinned: true);
// дальше используем pinned без GCHandle
```

Финализатор на каждый объект.

```
// было: объект с финализатором переживает сборку
class Wrapper
{
    ~Wrapper() => ReleaseNative();
}

// стало: SafeHandle, который освобождает детерминированно
class Wrapper : SafeHandleZeroOrMinusOneIsInvalid
{
    protected override bool ReleaseHandle() => ReleaseNative();
}
```

`GC.Collect` в горячем пути.

```
// было: пила из пауз
foreach (var item in items)
{
    Process(item);
```

```
    GC.Collect(); // никогда так не делайте
}

// стало: доверьтесь сборщику, а давление сообщайте явно
foreach (var item in items)
    Process(item);
```

Если у вас большая нативная память, которую GC не видит, сообщите ему через `GC.AddMemoryPressure` и `GC.RemoveMemoryPressure`, а не через ручную сборку.

`NoGCRegion` без запаса.

```
// было: если памяти не хватит, EndNoGCRegion кинет исключение
GC.TryStartNoGCRegion(1024);
// ...
GC.EndNoGCRegion();

// стало: проверяем, что регион реально начался
if (GC.TryStartNoGCRegion(16 * 1024 * 1024))
{
    try { /* работа без пауз */ }
    finally { GC.EndNoGCRegion(); }
}
else
{
    // запасной путь без гарантий
}
```

Что в итоге

Аллокация - это сдвиг указателя в per-thread контексте. Дешево ровно до момента, когда `alloc_ptr` догоняет `alloc_limit`, и поток зовет GC. Дальше вступают поколения: gen0 собирается часто и дешево, gen2 редко и дорого. LOH хранит крупные объекты без компактировки, POH хранит запиненные без фрагментации SOH. Write barrier помечает cards, чтобы GC не сканировал всю старую кучу.

Отсюда и все наши правила: переиспользуй, держись под порогом LOH, пинь через POH, не зануляй зря, не заводи финализаторы. Каждое из них убирает работу, которую иначе сделал бы сборщик. А если совсем прижало по latency, есть `NoGCRegion` и `LatencyMode`, но это уже точечные инструменты под конкретную задачу.

Span и Memory: смотрим на память, не копируя ее

Представьте, что вы пишете парсер CSV. Строка на входе - это `string`, 2 килобайта. Вы делите ее по запятым, каждое поле - это `Substring`. Пятьдесят полей - пятьдесят новых строк в куче. Потом вы эти поля парсите в числа. И вот уже горячий цикл обработки одной строки сгенерировал сотню объектов, которые умрут через микроволновку. GC недоволен, вы недоволен, p99 latency растет.

А ведь по-хорошему нам не нужны новые строки. Нам нужны окна в уже существующую память: "вот здесь, с 17-го по 24-й байт, лежит число". `Span<T>` и `Memory<T>` - это ровно такие окна. Указатель плюс длина. Никакого копирования.

Дальше я буду разбирать их по исходникам `System.Private.CoreLib` из `D:\runtime`, потому что вся суть этих типов видна именно в полях и в том, как рантайм обходится с ними.

На пальцах: стек, куча и цена аллокации

Прежде чем нырять в `ref T _reference`, договоримся о базовых вещах. Если вы это знаете, пролистайте до словаря.

В .NET память живет в двух основных местах. **Стек** - это небольшая область на каждый поток, где лежат локальные переменные и адреса возврата. Выделение на стеке бесплатно: компилятор просто резервирует место, сдвигая указатель стека. **Куча** - это большая общая область, где живут объекты. Выделение в куче тоже быстро (сдвиг указателя), но у него есть цена: объект нужно потом собрать, а сборка мусора останавливает потоки и стоит времени.

Что значит "аллокация". Это выделение нового объекта в куче. Каждый `new`, каждый `Substring`, каждая конкатенация строк создают объект. Объект занимает память и требует работы GC, когда перестает быть нужен.

Вернемся к примеру с CSV. Строка `"42,hello,3.14"` уже лежит в куче, это один объект. Когда вы делаете `Substring`, рантайм создает **новую** строку и копирует в нее символы. Для 50 полей это 50 новых объектов. Каждый из них нужно выделить, а потом собрать. Вот откуда сотня объектов на одну строку входных данных.

А теперь идея `Span`. Вместо копирования символов мы храним два числа: "где начало" и "сколько символов". Указатель на символ внутри исходной строки плюс длина. Это и есть "окно". Никакого нового объекта, никакого копирования, никакой работы для GC.

```
строка в куче:  [ 4 2 , h e l l o , 3 . 1 4 ]
                  ^         ^
                  |         |
   Span поле 1 ---+         |
   Span поле 2 -------------+
```

Оба span смотрят в ту же самую память. Просто с разных позиций и с разной длиной.

Вот та же операция в двух вариантах, чтобы разница была осязаемой:

```
string line = "42,hello,3.14";

// вариант 1: две аллокации
string a = line.Substring(0, 2);    // новый объект string
string b = line.Substring(3, 5);    // еще один объект string

// вариант 2: ноль аллокаций
ReadOnlySpan<char> s = line.AsSpan();
ReadOnlySpan<char> a2 = s.Slice(0, 2);  // просто ссылка + длина
ReadOnlySpan<char> b2 = s.Slice(3, 5);  // просто ссылка + длина
```

Вариант 2 не создает ни одного объекта. `a2` и `b2` - это 16 байт на стеке, которые указывают внутрь `line`.

Словарь терминов

Аллокация - выделение памяти под новый объект в куче.

Стек - область памяти потока для локальных переменных. Освобождается автоматически при выходе из метода.

Куча - область, где живут объекты. За ней следит сборщик мусора.

GC (Garbage Collector, сборщик мусора) - подсистема, которая находит объекты, ставшие ненужными, и освобождает их память.

Managed reference (управляемая ссылка) - ссылка на объект или его часть, которую видит и отслеживает GC. Если такая ссылка куда-то указывает, GC не тронет эту память.

Interior pointer (внутренний указатель) - ссылка не на начало объекта, а внутрь него. `Span<T>` держит именно такой указатель на элемент массива.

ref struct - структура, которая обязана жить на стеке. Компилятор запрещает класть ее в кучу.

ref return / ref local - возврат или хранение ссылки на переменную, а не ее копии. `ref T _reference` в `Span` - это ref-поле.

Bounds check (проверка границ) - проверка, что индекс не выходит за пределы массива или span. Каждое обращение `array[i]` содержит такую проверку.

Intrinsic (интринсик) - метод, который JIT не вызывает как обычный, а заменяет на специальный машинный код или на прямое обращение к данным.

JIT (Just-In-Time) - компилятор, который превращает IL в машинный код во время выполнения.

Inlining (встраивание) - замена вызова метода его телом прямо в месте вызова. Убирает накладные расходы на вызов.

Backing object (несущий объект) - объект, который реально владеет памятью. Для `Memory<T>` это массив, строка или `MemoryManager<T>`.

Pinning (пиннинг) - запрет сборщику перемещать объект. Нужен, когда нативный код держит указатель на управляемую память.

GCHandle - структура-дескриптор, через которую можно запинить объект и получить его адрес.

Boxing (упаковка) - превращение значимого типа в объект в куче. Требует аллокации.

State machine (машина состояний) - объект, в который компилятор превращает метод с `await`, чтобы приостановить и возобновить его.

Lifetime (время жизни) - как долго живет значение или ссылка. Главное правило span: он не живет дольше буфера, из которого сделан.

stackalloc - выделение буфера прямо на стеке. Не требует GC.

Две структуры, одна идея

`Span<T>` и `Memory<T>` описывают одно и то же: непрерывный кусок памяти с началом и длиной. Разница в одном слове - `ref`.

```
// Span.cs:28
public readonly ref struct Span<T>
{
    // Span.cs:31
    internal readonly ref T _reference;
    // Span.cs:33
    private readonly int _length;
}
```

`ref struct` - это структура, которая обязана жить на стеке. Компилятор и рантайм следят за тем, чтобы она никогда не оказалась в куче: ее нельзя упаковать в `object`, нельзя положить в поле класса, нельзя использовать в `async`-методе или в итераторе, потому что те превращаются в state machine на куче. Зато `ref T _reference` - это настоящая управляемая ссылка на элемент, которую видит GC. Она может указывать на массив, на стек (`stackalloc`), на нативную память. Рантайм знает, где эта ссылка, и не даст сборщику утащить массив, пока `Span` жив.

Вот сводка, что с `ref struct` можно, а что нельзя. Это стоит запомнить, потому что половина ошибок компиляции со span - именно отсюда.

Действие	Можно?	Почему
Локальная переменная в методе	да	живет на стеке
Параметр метода	да	передается на стеке
Возврат из метода	да	если не ссылается на локальный буфер
Поле класса или структуры	нет	структура может уехать в кучу

Действие	Можно?	Почему
Упаковка в `object`	нет	boxing создает объект в куче
Использование в `async` / `yield`	нет	метод превращается в state machine на куче
Захват в лямбду	нет	замыкание живет в куче
Элемент массива	нет	массив в куче
Поле `ref struct` внутри другого `ref struct`	да	обе на стеке

Ключевое отличие `ref T` от обычного указателя `T*`. Указатель - это просто число, GC про него ничего не знает. Ссылка `ref T` - это тоже адрес, но рантайм умеет ее распознавать: при сборке мусора он понимает, что по этому адресу лежит живая ссылка, и корректирует ее, если объект переехал. Поэтому `Span` может спокойно указывать на массив, и GC не утащит этот массив, пока span жив, и не оставит висячий указатель после компактировки.

```
// Memory.cs:21-28
public readonly struct Memory<T> : IEquatable<Memory<T>>
{
    private readonly object? _object;
    private readonly int _index;
    private readonly int _length;
}
```

`Memory<T>` - обычная структура. Ее можно положить в поле класса, передать через `await`, сохранить в список. Внутри лежит `object`, который держит backing-память живой. Обратите внимание: `Memory<T>` не хранит `ref`, он хранит объект плюс индекс. Именно поэтому он переживает перемещение GC и асинхронные границы.

Простая аналогия: `Span<T>` - это окно, которое живет ровно столько, сколько идет вычисление на текущем кадре стека. `Memory<T>` - это квитанция на багаж, которую можно положить в карман и предъявить позже.

Как Span превращается в адрес

Ключевой вопрос: откуда берется `ref T _reference`. Ответ в конструкторе от массива:

```
// Span.cs:52
_reference = ref MemoryMarshal.GetArrayDataReference(array);
_length = array.Length;
```

`GetArrayDataReference` - интринсик. Смотрите `MemoryMarshal.cs:100`:

```
[Intrinsic]
[NonVersionable]
public static ref T GetArrayDataReference<T>(T[] array) =>
    ref GetArrayDataReference(array);
```

А реализация для `Array` (`MemoryMarshal.cs:118`) делает так:

```
// MemoryMarshal.cs:124
return ref Unsafe.AddByteOffset(ref Unsafe.As<RawData>(array).Data,
    (nuint)RuntimeHelpers.GetMethodTable(array)->BaseSize - (nuint)(2 * sizeof(IntPtr)));
```

Это ручной сдвиг к первому элементу массива: берем адрес заголовка объекта, прибавляем `BaseSize` минус два указателя (указатель на method table и на длину массива). Получаем адрес данных без проверки границ, которую сделал бы `array[0]`. Именно поэтому `Span<T>` умеет смотреть в массив, не дергая никаких проверок на входе.

Разберем эти термины, потому что они встречаются везде. **Method table** - это структура, описывающая тип объекта: его размер, флаги, методы. В начале каждого объекта в куче лежит указатель на нее, так рантайм понимает, что это за тип. **BaseSize** - размер объекта в байтах, включая заголовок. У массива `BaseSize` включает заголовок, указатель на method table и поле длины.

Вот как выглядит массив `int[3]` в памяти:

```
адрес+0:  [заголовок]      8 байт (служебные данные GC)
адрес+8:  [MethodTable*]   8 байт указателя на тип int[]
адрес+16: [length]         4 байта (плюс 4 выравнивания)
адрес+24: [0]              4 байта, первый элемент
адрес+28: [1]              4 байта
адрес+32: [2]              4 байта
```

`GetArrayDataReference` берет адрес начала объекта, прибавляет `BaseSize` (для `int[3]` это 24) и получает адрес первого элемента. `array[0]` сделал бы то же самое, но дополнительно проверил бы, что индекс 0 меньше длины. Здесь проверка не нужна: мы точно знаем, что смотрим на элемент 0 существующего массива.

Почему это важно для производительности. Каждое обращение к массиву по индексу в .NET содержит проверку границ. В плотном цикле это лишние сравнения и ветвления. `GetArrayDataReference` дает адрес начала один раз, без проверки, и дальше можно идти по памяти. JIT часто сам убирает проверки в цикле, но не всегда. Здесь же компилятор получает гарантию от рантайма.

Срез тоже ничего не копирует, он просто двигает ссылку:

```
// Span.cs:396-401
public Span<T> Slice(int start)
{
    if ((uint)start > (uint)_length)
        ThrowHelper.ThrowArgumentOutOfRangeException();

    return new Span<T>(ref Unsafe.Add(ref _reference, (nint)(uint)start), _length - start);
}
```

А вот в `Slice(int start, int length)` зарыт красивый трюк с переполнением на 64 битах (`Span.cs:415-427`):

```
#if TARGET_64BIT
    // Since start and length are both 32-bit, their sum can be computed across a 64-bit
domain
    // without loss of fidelity. The cast to uint before the cast to ulong ensures that the
    // extension from 32- to 64-bit is zero-extending rather than sign-extending.
    if ((ulong)(uint)start + (ulong)(uint)length > (ulong)(uint)_length)
        ThrowHelper.ThrowArgumentOutOfRangeException();
#else
    if ((uint)start > (uint)_length || (uint)length > (uint)(_length - start))
        ThrowHelper.ThrowArgumentOutOfRangeException();
#endif
```

На 64-битной платформе обе проверки - отрицательный `start`, отрицательный `length`, выход за границы - сворачиваются в одну. `uint` перед `ulong` гарантирует нулевое расширение, а не знаковое, поэтому отрицательное число превращается в огромное положительное и сразу проваливает сравнение. Один `cmp`, одна ветка вместо двух. Рантайм экономит такт на каждом срезе, потому что срезы случаются в каждом парсере.

Теперь про размер. Два поля: ссылка (8 байт на x64) и длина (4 байта), плюс выравнивание. То есть `Span<T>` - это примерно 16 байт, которые целиком помещаются в пару регистров и обычно вообще не попадают в память, если JIT развернул инлайнинг. `Span<T>` помечен `[Intrinsic]` (`Span.cs:27`), и `Length` тоже (`Span.cs:165`), так что JIT умеет обращаться с ним как с первоклассной структурой, а не как с обычным типом.

Почему Span нельзя протащить в heap

Вот код, который выглядит невинно и не компилируется:

```
class Parser
{
    private Span<char> _buffer; // ошибка компилятора: ref struct не может быть полем класса
}
```

И вот такой тоже:

```
async Task ReadAsync(Span<byte> buffer) // ошибка: ref struct нельзя в async-метод
{
    await socket.ReceiveAsync(buffer);
}
```

Компилятор режет это не из вредности. Если бы `Span<T>` лежал в куче, его `ref T _reference` мог бы указывать на кадр стека, которого уже нет. GC не смог бы отследить такую ссылку корректно, и мы бы получили висячий указатель в управляемой памяти. Язык просто запрещает ситуацию на этапе компиляции. Это и есть та самая "memory safety без unsafe".

Разберем, почему `async` особенно опасен. Когда метод содержит `await`, компилятор переписывает его в **state machine** - объект в куче, который хранит состояние метода между приостановками. Все

локальные переменные, переживающие `await`, становятся полями этого объекта. Если бы среди них был `Span`, он оказался бы в куче. А его `ref` мог бы указывать на буфер, который к моменту возобновления уже освобожден.

Вот что примерно происходит с таким методом:

```
async Task Example()
{
    Span<byte> buffer = stackalloc byte[256]; // буфер на стеке
    await SomethingAsync();                   // здесь кадр стека исчезает
    buffer[0] = 1;                            // буфер уже не существует
}
```

`await` освобождает текущий кадр стека и возвращает управление вызывающему. Буфер `buffer` жил в этом кадре. Когда метод возобновится, это будет уже другой кадр, и адрес из `ref` будет указывать в никуда. Компилятор запрещает такой код, потому что иначе мы получили бы порчу памяти, от которой GC не защитит.

Отсюда практическое правило: `Span<T>` - для синхронного, локального кода. Как только нужно сохранить окно в объекте или пережить `await`, берем `Memory<T>`.

Memory: квитанция вместо окна

`Memory<T>` устроен вокруг `_object`. Посмотрите на конструктор от массива (`Memory.cs:47-49`): `_object = array; _index = 0; _length = array.Length`. Backing-объект держится живым, потому что на него ссылается поле структуры.

Вот как выглядит `Memory<int>`, который смотрит на кусок массива:

```
int[] data = new int[100];
Memory<int> mem = data.AsMemory(10, 5); // элементы с 10 по 14
```

```
Memory<int> (16 байт на стеке)
  _object = -------> int[100] в куче (весь массив)
  _index  = 10
  _length = 5
```

Обратите внимание: `Memory<int>` хранит ссылку на **весь** массив, а не на его часть. Смещение и длину он держит у себя. Именно поэтому GC видит обычную ссылку на объект и корректно его сохраняет. А когда нужно получить `Span`, рантайм берет адрес массива, прибавляет `_index` и получает нужное окно.

Отсюда и главный минус, о котором мы поговорим в ловушках: `Memory` держит живым весь массив целиком. Если массив на 100 МБ, а `Memory` смотрит на 10 байт, все 100 МБ останутся в памяти, пока жива структура.

Чтобы получить `Span<T>` из `Memory<T>`, рантайм делает нетривиальную работу (`Memory.cs:263`). Внутри он разбирает, что за объект лежит в `_object`:

- `string`, если `T` это `char`. Тогда берется `GetRawStringData` строки.

- массив. Тогда `GetArrayDataReference`.
- `MemoryManager<T>`. Тогда `manager.GetSpan()`.

Вот фрагмент (`Memory.cs:283-319`):

```
if (typeof(T) == typeof(char) && tmpObject.GetType() == typeof(string))
{
    refToReturn = ref Unsafe.As<char, T>(ref ((string)tmpObject).GetRawStringData());
    lengthOfUnderlyingSpan = Unsafe.As<string>(tmpObject).Length;
}
else if (RuntimeHelpers.ObjectHasComponentSize(tmpObject))
{
    refToReturn = ref MemoryMarshal.GetArrayDataReference(Unsafe.As<T[]>(tmpObject));
    lengthOfUnderlyingSpan = Unsafe.As<T[]>(tmpObject).Length;
}
else
{
    Span<T> memoryManagerSpan = Unsafe.As<MemoryManager<T>>(tmpObject).GetSpan();
    refToReturn = ref MemoryMarshal.GetReference(memoryManagerSpan);
    lengthOfUnderlyingSpan = memoryManagerSpan.Length;
}
```

Тут же есть защита от "порванной" структуры: `desiredStartIndex = (uint)_index & (uint)RemoveFlagsBitMask` (`Memory.cs:328`). `RemoveFlagsBitMask` - это `0x7FFFFFFF` (`ReadOnlyMemory.cs:30`). Старший бит индекса используется как флаг: если он выставлен, значит память уже запинена и `Pin()` не будет выделять новый `GCHandle`. Об этом прямо написано в комментарии над полями (`Memory.cs:23-25`):

```
// The highest order bit of _index is used to discern whether _object is a pre-pinned array.
// (_index < 0) => _object is a pre-pinned array, so Pin() will not allocate a new GCHandle
//       (else) => Pin() needs to allocate a new GCHandle to pin the object.
```

Экономия одного бита индекса, чтобы не заводить отдельное поле-флаг. Классика низкоуровневого дизайна рантайма.

`Memory<T>` весит ровно 16 байт: `object` (8), `_index` (4), `_length` (4). Никакого boxing внутри, никаких скрытых аллокаций при копировании структуры.

Span или Memory: как выбрать

Правило простое, но его стоит закрепить таблицей.

Ситуация	Что брать	Почему
Локальная обработка внутри метода	`Span<T>`	быстрее, нет объекта, живет на стеке
Параметр синхронного метода	`ReadOnlySpan<T>`	нет аллокации, читаемость

Ситуация	Что брать	Почему
Поле класса	`Memory<T>` или `ReadOnlyMemory<T>`	span в поле нельзя
Данные переживают `await`	`Memory<T>`	span запрещен в async
Возврат из `async` метода	`Memory<T>`	span нельзя вернуть через await
Буфер под interop	`Memory<T>` + `Pin`	нужен `void*` и контроль времени жизни
Парсинг строки на месте	`ReadOnlySpan<char>`	ноль копирований
Долгий цикл по массиву	`Span<T>`	JIT убирает проверки границ

Типичный прием на границе: снаружи API принимает `Memory<T>`, а внутри метода сразу превращает его в `Span<T>` и дальше работает только со span. Так и аллокаций нет, и API удобный.

```
public void Process(ReadOnlyMemory<byte> data)
{
    ReadOnlySpan<byte> span = data.Span; // один раз
    // дальше везде span, без обращений к Memory
    Parse(span);
}
```

Важно: `data.Span` каждый раз делает работу по разбору `_object`. Если вызывать его в цикле, вы будете repeatedly платить за то, что можно сделать один раз. Поэтому вынесли в локальную переменную.

Строка как ReadOnlySpan

Самая частая задача - пройтись по строке без создания новых строк. У `string` есть внутренний метод:

```
// String.cs:531
internal ref char GetRawStringData() => ref _firstChar;
```

Это ссылка на первый символ. А расширение `AsSpan` (`MemoryExtensions.cs:97`) собирает из нее span:

```
[Intrinsic] // When input is a string literal
public static ReadOnlySpan<char> AsSpan(this string? text)
{
    if (text == null)
        return default;

    return new ReadOnlySpan<char>(ref text.GetRawStringData(), text.Length);
}
```

`[Intrinsic]` здесь не для красоты. Когда вы пишете `"GET / HTTP/1.1".AsSpan()`, JIT видит строковый литерал и может свернуть весь span в статическую ссылку на данные в куче без единой проверки.

Сравните:

```
string line = "42,hello,3.14";

// аллокация на каждое поле
string a = line.Substring(0, 2);   // "42"
string b = line.Substring(3, 5);   // "hello"

// ноль аллокаций
ReadOnlySpan<char> span = line.AsSpan();
ReadOnlySpan<char> a2 = span.Slice(0, 2);
ReadOnlySpan<char> b2 = span.Slice(3, 5);
```

`a2` и `b2` - это просто ссылка внутрь исходной строки плюс длина. Строка не копируется, `Substring` не вызывается, GC молчит. Парсеры и `int.Parse(ReadOnlySpan<char>)`, и `double.Parse(ReadOnlySpan<char>)`, и `Encoding.GetBytes(ReadOnlySpan<char>, Span<byte>)` работают именно так.

MemoryManager: память, которую придумали вы

Откуда `Memory<T>` может смотреть в нативную память или в пул? Через `MemoryManager<T>` (`Buffers/MemoryManager.cs:11`):

```
public abstract class MemoryManager<T> : IMemoryOwner<T>, IPinnable
{
    public virtual Memory<T> Memory => new Memory<T>(this, GetSpan().Length);

    public abstract Span<T> GetSpan();
    public abstract MemoryHandle Pin(int elementIndex = 0);
    public abstract void Unpin();

    protected abstract void Dispose(bool disposing);
}
```

Вы наследуетесь, реализуете `GetSpan`, `Pin`, `Unpin`, и `Memory<T>` начинает смотреть куда угодно. `Memory<T>.Span` в ветке `else` (`Memory.cs:315`) дергает `manager.GetSpan()`, а `Memory<T>.Pin` (`Memory.cs:428`) дергает `manager.Pin(_index)`.

Пин возвращает `MemoryHandle` (`Buffers/MemoryHandle.cs:11`):

```
public unsafe struct MemoryHandle : IDisposable
{
    private void* _pointer;
    private GCHandle _handle;
    private IPinnable? _pinnable;
    ...
    public void Dispose()
    {
        if (_handle.IsAllocated) _handle.Free();
        if (_pinnable != null) { _pinnable.Unpin(); _pinnable = null; }
        _pointer = null;
```

```
    }
}
```

Это то, что нужно для interop: получаете `void*`, отдаете его нативной функции, а `Dispose` снимает пин. `ArrayPool<T>` и `MemoryPool<T>` построены на этом же механизме.

Что при этом делает процессор

JIT знает про `Span<T>` не как про обычную структуру, а как про встроенный паттерн. Несколько практических следствий:

- Проверка границ в цикле по span часто устраняется. Если вы пишете `for (int i = 0; i < span.Length; i++)`, JIT видит, что индекс ограничен длиной, и убирает per-element bounds check. На массиве такое тоже возможно, но со span это надежнее.
- `GetArrayDataReference` экономит проверку на нулевом элементе, которую делает обычное индексирование массива.
- `CopyTo` дергает `Buffer.Memmove` напрямую (`Span.cs:334`), без промежуточного слоя и с корректной обработкой перекрытия областей.
- `stackalloc` теперь типизирован: `Span<byte> buf = stackalloc byte[256];` дает буфер на стеке с проверкой границ. Ноль аллокаций, ноль работы для GC.

Но есть и цена. `Span<T>` - это структура, которую передают по значению. Если JIT не заинлайнил метод, каждый вызов копирует 16 байт. На горячем пути это не проблема, потому что все инлайнится, но в рекурсивных или виртуальных сценариях может внезапно стать заметно.

Ловушки, на которые легко наступить

Span, возвращенный из метода, может указывать на мертвый стек. Вот это ловушка номер один:

```
Span<int> GetNumbers()
{
    Span<int> local = stackalloc int[4];
    return local; // компилятор не даст, если вернуть напрямую
}
```

Компилятор защищает от прямого возврата, но `ref struct` все равно легко протащить через `ref`-параметры или замыкания так, что lifetime станет неочевидным. Правило простое: span живет не дольше того буфера, из которого он сделан.

`Span<T>.Equals` всегда кидает исключение. Это не забывчивость авторов, а следствие того, что span нельзя упаковать (`Span.cs:192-206`):

```
[Obsolete("Equals() on Span will always throw an exception. Use the equality operator
instead.")]
public override bool Equals(object? obj) =>
    throw new NotSupportedException(SR.NotSupported_CannotCallEqualsOnSpan);
```

Потому что любой вызов `Equals(object)` требует боксинга, а `ref struct` боксить нельзя. Для сравнения есть `operator ==` (`Span.cs:365`), который сравнивает адрес и длину, а не содержимое.

`ToArray` аллоцирует. `Span.cs:437` честно об этом пишет в комментарии. Если вы вызываете `ToArray` в горячем пути, вы вернули все аллокации, от которых убегали. Мост к старым API на массивах неизбежен, но его стоит держать на границе, а не в цикле.

`Memory<T>` держит весь backing-объект. Если вы нарезали `Memory<byte>` из массива в 100 МБ и сохранили срез в 10 байт, массив все равно останется живым, пока живет `Memory<T>`. GC не умеет освобождать часть массива. Для долгоживущих мелких срезов это утечка по памяти.

`Memory<T>.Pin` на обычном массиве аллоцирует `GCHandle`. Смотрите `Memory.cs:420-422`: для не-пре-пиненного массива создается `GCHandle.Alloc(..., GCHandleType.Pinned)`. Если это происходит на каждую операцию, вы снова платите. Либо используйте память, помеченную как pre-pinned (тогда старший бит `_index` выставится и `Pin` обойдется без хэндла), либо `MemoryManager<T>`, который сам управляет пином.

Частые ошибки на примерах

Разберем типовые грабли в виде "было / стало", потому что на словах все понятно, а в коде ошибки повторяются.

Срез строки в цикле.

```
// было: аллокация на каждую итерацию
for (int i = 0; i < lines.Count; i++)
{
    string field = lines[i].Substring(0, 4);
    Handle(field);
}

// стало: ноль аллокаций
for (int i = 0; i < lines.Count; i++)
{
    ReadOnlySpan<char> field = lines[i].AsSpan(0, 4);
    Handle(field);
}
```

Span как поле класса.

```
// было: не компилируется
class Reader
{
    private Span<byte> _buffer;
}

// стало: Memory в поле, Span локально
class Reader
{
    private Memory<byte> _buffer;

    public void Read()
    {
```

```
        Span<byte> span = _buffer.Span;
        // работаем со span
    }
}
```

Возврат буфера со стека.

```
// было: указывает на мертвый стек
Span<int> Broken()
{
    Span<int> local = stackalloc int[4];
    return local;
}

// стало: либо копируем наружу
int[] CopyOut()
{
    Span<int> local = stackalloc int[4];
    return local.ToArray();
}

// либо просим буфер у вызывающего
void Fill(Span<int> destination)
{
    // пишем прямо в переданный буфер
}
```

Второй вариант лучше: вызывающий сам решает, откуда взять буфер, и аллокаций по-прежнему ноль, если он передал `stackalloc` или кусок пула.

Memory держит большой массив.

```
// было: весь массив на 100 МБ остается живым
byte[] big = new byte[100 * 1024 * 1024];
ReadOnlyMemory<byte> slice = big.AsMemory(0, 10);
Store(slice); // теперь big нельзя собрать

// стало: если нужен маленький долгоживущий кусок, скопируйте его
ReadOnlyMemory<byte> slice2 = big.AsMemory(0, 10).ToArray();
Store(slice2); // держит только 10 байт
```

Пиннинг на каждом вызове.

```
// было: GCHandle создается на каждую операцию
using MemoryHandle h = memory.Pin();
NativeCall(h.Pointer);

// стало: либо заранее запиненная память (POH)
byte[] pinned = GC.AllocateUninitializedArray<byte>(4096, pinned: true);
using MemoryHandle h2 = pinned.AsMemory().Pin(); // без нового GCHandle

// либо один долгоживущий хэндл на весь жизненный цикл
```

ToArray в горячем пути.

```
// было: аллокация ради передачи в старый API
ProcessData(span.ToArray());

// стало: используйте перегрузку на span, если она есть
ProcessData(span);

// а если нет, держите ToArray на границе, а не в цикле
```

Сравнение span через Equals.

```
// было: исключение
bool eq = span1.Equals(span2); // NotSupportedException

// стало: сравнение содержимого
bool eq2 = span1.SequenceEqual(span2);

// или сравнение адреса и длины
bool same = span1 == span2;
```

Практика: парсим без аллокаций

Соберем кусок, который раньше создавал гору строк:

```
using System;
using System.Buffers.Text;

static bool TryParsePoint(ReadOnlySpan<char> line, out int x, out int y)
{
    x = 0;
    y = 0;

    int comma = line.IndexOf(',');
    if (comma < 0) return false;

    ReadOnlySpan<char> left = line.Slice(0, comma);
    ReadOnlySpan<char> right = line.Slice(comma + 1);

    return int.TryParse(left, out x) && int.TryParse(right, out y);
}
```

Ни одного `Substring`, ни одной промежуточной строки. `int.TryParse(ReadOnlySpan<char>)` читает символы прямо из исходной строки.

А вот запись без аллокаций через `Span<byte>` и UTF-8:

```
static int WritePoint(Span<byte> destination, int x, int y)
{
    Utf8Formatter.TryFormat(x, destination, out int written);
    destination[written++] = (byte)':';
    Utf8Formatter.TryFormat(y, destination.Slice(written), out int writtenY);
    return written + writtenY;
}
```

Если буфер выделен из `ArrayPool<byte>`, вся операция проходит без единого объекта в куче.

Соберем полный пример: разбор CSV-строки на числа без единой аллокации.

```
static bool TryParseCsv(ReadOnlySpan<char> line, Span<int> results, out int count)
{
    count = 0;
    int start = 0;

    for (int i = 0; i <= line.Length; i++)
    {
        // разделитель - запятая или конец строки
        if (i == line.Length || line[i] == ',')
        {
            ReadOnlySpan<char> field = line.Slice(start, i - start);
            if (!int.TryParse(field, out int value))
                return false;

            results[count++] = value;
            start = i + 1;
        }
    }
    return true;
}
```

Вызываем так:

```
Span<int> numbers = stackalloc int[16];
if (TryParseCsv("42,7,13,99".AsSpan(), numbers, out int n))
{
    for (int i = 0; i < n; i++)
        Console.WriteLine(numbers[i]);
}
```

Здесь нет ни одной строки, ни одного массива в куче. `line` - окно в существующую строку, `numbers` - буфер на стеке. Весь разбор это хождение по памяти плюс `int.TryParse`, который читает символы прямо из span.

Сравните с наивным вариантом, который делает то же самое:

```
static List<int> ParseCsvNaive(string line)
{
    var result = new List<int>();
    foreach (string field in line.Split(','))   // массив строк + строки
        result.Add(int.Parse(field));           // int.Parse(string)
    return result;                              // список в куче
}
```

На строке из 16 чисел наивный вариант создает массив из 16 строк, 16 объектов строк, список и его внутренний массив. Это десятки объектов и десятки килобайт мусора. Вариант со span - ноль. Разница в аллокациях измеряется не процентами, а порядками.

Как убедиться, что вы действительно не аллоцируете. В тесте:

```
long before = GC.GetAllocatedBytesForCurrentThread();
TryParseCsv(line, numbers, out _);
```

```
long after = GC.GetAllocatedBytesForCurrentThread();
Console.WriteLine($"allocated: {after - before} bytes");
```

На корректно написанном span-коде разница будет нулевой. Если нет, значит где-то спрятался `ToString`, `ToArray` или бокс.

Что в итоге

`Span<T>` и `Memory<T>` - это не просто удобные обертки. Это способ сказать рантайму: "вот область памяти, работай с ней напрямую, не копируй". `Span<T>` дает максимальную скорость и живет на стеке, `Memory<T>` дает возможность сохранить окно и пережить `await` ценой одного поля-объекта. Вместе они убирают самый частый источник мусора в .NET - промежуточные массивы и строки.

Когда в следующий раз увидите `Substring` в цикле или `new byte[]` под каждую операцию, вспомните: скорее всего, там достаточно пары `Slice` над `ReadOnlySpan`. И тогда GC скажет вам спасибо.

Unsafe и MemoryMarshal: работаем с адресами, не выключая GC

Обычный C# прячет от вас адреса. Вы не знаете, где лежит объект, GC может его подвинуть, а разыменовать `ref` за пределами времени жизни объекта компилятор не даст. Это и есть memory safety. Но когда вы пишете высокопроизводительный код, эта забота иногда мешает: хочется прочитать структуру прямо из байтов, переинтерпретировать массив, посчитать адрес поля, скопировать блок памяти. Для этого в .NET есть два инструмента: `System.Runtime.CompilerServices.Unsafe` и `System.Runtime.InteropServices.MemoryMarshal`.

Это не `unsafe` из C++ и не выключение GC. Это способ сказать рантайму: "я знаю, что делаю, дай мне управляемую ссылку в обход обычных проверок". GC по-прежнему отслеживает `ref T`, просто проверки компилятора и часть рантайм-гарантий вы берете на себя.

На пальцах: что вообще такое адрес и ссылка

Чтобы дальше было понятно, разберемся с тремя способами сослаться на данные в .NET.

Обычная ссылка на объект (`object o`, `string s`). Это адрес объекта в куче. GC знает про эту ссылку, отслеживает ее и, если объект переехал при компактировке, обновляет значение. Для программиста это прозрачно.

ref T (managed byref, управляемая ссылка). Это адрес конкретного значения: элемента массива, поля структуры, локальной переменной. GC тоже про нее знает и корректирует, если объект, внутри которого она лежит, сдвинулся. Именно так работают `ref`-параметры и `Span<T>`.

void* (неуправляемый указатель). Это просто число-адрес. GC про него не знает ничего. Если объект сдвинется, указатель останется смотреть на старое место, и вы получите мусор.

Вся тема этой главы - это мосты между тремя мирами. `Unsafe.AsPointer` превращает `ref T` в `void*`. `Unsafe.AsRef` делает обратно. `MemoryMarshal` строит поверх этого удобные и относительно безопасные операции.

Где это применяется. Вот реальный пример: у вас есть буфер байтов, пришедший из сети, и в нем лежит структура заголовка. Наивный способ прочитать ее - скопировать байты в структуру через `BitConverter` для каждого поля. Это медленно и громоздко:

```
// медленно и шумно
uint magic = BitConverter.ToUInt32(buffer, 0);
ushort version = BitConverter.ToUInt16(buffer, 4);
ushort flags = BitConverter.ToUInt16(buffer, 6);
int length = BitConverter.ToInt32(buffer, 8);
```

С `MemoryMarshal` это одна операция:

```
// быстро и коротко
PacketHeader header = MemoryMarshal.Read<PacketHeader>(buffer);
```

`MemoryMarshal.Read` берет адрес буфера, переинтерпретирует его как `PacketHeader` и копирует структуру. Ни `BitConverter`, ни промежуточных объектов.

Словарь терминов

Managed byref (управляемая ссылка, `ref T`) - адрес значения, который отслеживает GC. Может указывать внутрь объекта или на стек.

void* - неуправляемый указатель. Число-адрес, невидимое для GC.

Intrinsic (интринсик) - метод, который JIT не вызывает, а заменяет на специальные инструкции. Тело метода в исходнике обычно `throw`.

Method table - структура, описывающая тип объекта. В начале каждого объекта лежит указатель на нее.

Object header (заголовок объекта) - служебные данные перед полями объекта: указатель на method table, длина массива, служебные биты GC.

BaseSize - полный размер объекта в байтах, включая заголовок.

Blittable - тип, у которого управляемое и нативное представление совпадают. Такой тип можно копировать в нативную память без конвертации.

Unmanaged type - тип, не содержащий управляемых ссылок: числа, `bool`, `char`, структуры из них, указатели.

Alignment (выравнивание) - требование, чтобы адрес был кратен размеру. Процессоры читают выровненные данные быстрее, а некоторые архитектуры (ARM) на невыровненных могут падать.

Endianness (порядок байтов) - порядок хранения байтов числа в памяти: little-endian (младший байт первый) или big-endian (старший первый). Сетевые протоколы обычно big-endian.

Pinning (пиннинг) - запрет GC перемещать объект. Нужен, когда вы отдаете адрес управляемой памяти в нативный код.

fixed - конструкция C#, которая пинит объект на время блока и дает на него указатель.

GCHandle - дескриптор для пиннинга и получения адреса объекта вручную.

memcpy / memset - классические C-функции копирования и заполнения памяти. В .NET им соответствуют `CopyBlock` и `InitBlock`.

cpblk / initblk - IL-инструкции копирования и заполнения блока памяти. JIT превращает их в машинный код, часто векторизованный.

Bounds check (проверка границ) - проверка, что индекс не выходит за пределы массива. Каждое `array[i]` ее содержит.

Reinterpret (переинтерпретация) - прочитать те же байты как другой тип. `Unsafe.As` меняет только тип ссылки, байты не трогает.

Covariance (ковариантность) - разрешение считать `Derived[]` совместимым с `Base[]`. Из-за нее `array[i]` может делать проверку типа.

Boxing (упаковка) - превращение значимого типа в объект в куче.

Lifetime (время жизни) - как долго ссылка остается валидной. Для `void*` и `AsRef` это целиком на вашей ответственности.

scoped - модификатор, который ограничивает время жизни ссылки текущим методом. Помогает компилятору проверять безопасность.

ref T - это не просто указатель

Сначала разберемся с главным. `ref T` в .NET - это managed byref, управляемая ссылка. GC про нее знает и корректирует, если объект подвинулся. `void*` - это неуправляемый указатель, GC про него не знает ничего.

`Unsafe.AsPointer` (`Unsafe.cs:31`) превращает одно в другое:

```
[Intrinsic]
public static void* AsPointer<T>(ref readonly T value)
{
    throw new PlatformNotSupportedException();
    // ldarg.0
    // conv.u
    // ret
}
```

Комментарий с IL здесь не декорация. Тело метода в исходнике - это `throw`, а настоящую реализацию подставляет JIT: он видит `[Intrinsic]` и заменяет вызов на пару инструкций. Так работает почти весь `Unsafe`. В шапке файла прямо написано (`Unsafe.cs:8`):

```
// The implementations of most the methods in this file are provided as intrinsics.
// In CoreCLR, the body of the functions are replaced by the EE with unsafe code.
```

Обратная операция, `Unsafe.AsRef<T>(void* source)` (`Unsafe.cs:720`), делает `ref T` из сырого указателя. Вот тут и начинается опасность: полученная ссылка уже не отслеживается GC, и если объект сдвинется, вы держите мусорный адрес. Поэтому для работы с `void*` объект надо пинить.

Как GC вообще понимает, где на стеке и в регистрах лежат `ref T`. Для каждого метода JIT генерирует **GC info**: карту, которая говорит, в какой точке кода какие слоты содержат управляемые ссылки. Когда GC запускается, он проходит по стеку, смотрит эту карту и корректирует каждую живую ссылку, если объект переехал. Это называется "tracked byref" (отслеживаемая ссылка).

Теперь становится ясно, почему `AsRef(void*)` опасен. Результат - это `ref T`, но он не зарегистрирован ни в одной GC-карте. GC про него не знает. Если между `AsRef` и использованием объекта произойдет

сборка с компактировкой, ссылка станет указывать на старое место, а объект уже будет в другом. Это классический dangling reference.

Поэтому правило простое: получили `void*` из управляемого объекта - держите объект запиненным, пока используете указатель. Пиннинг говорит GC "не трогай этот объект", и адрес перестает меняться.

```
byte[] buffer = new byte[1024];

// fixed пинит массив на время блока
unsafe
{
    fixed (byte* p = buffer)
    {
        // пока мы здесь, buffer не сдвинется, p валиден
        NativeCall(p, buffer.Length);
    }
}
// здесь пин снят, массив снова может двигаться
```

Unsafe: набор интринсиков, а не библиотека

Список того, что умеет `Unsafe`, стоит пройтись по нему как по арсеналу.

Переинтерпретация:

```
Unsafe.As<TFrom, TTo>(ref source)   // Unsafe.cs:87, тот же адрес, другой тип
Unsafe.As<T>(object o)              // Unsafe.cs:68, без проверки типа
Unsafe.BitCast<TFrom, TTo>(source)  // Unsafe.cs:267, с проверкой размера
```

`As<TFrom, TTo>` не проверяет ничего. Если размеры не совпадают, вы читаете за пределами значения. `BitCast` аккуратнее: он проверяет, что `sizeof(TFrom) == sizeof(TTo)` и что оба типа value type (`Unsafe.cs:271`).

Арифметика по ссылкам:

```
Unsafe.Add(ref source, elementOffset)          // Unsafe.cs:107
Unsafe.AddByteOffset(ref source, byteOffset)   // Unsafe.cs:675
Unsafe.ByteOffset(ref origin, ref target)      // Unsafe.cs:754
Unsafe.AreSame(ref left, ref right)            // Unsafe.cs:248
```

`Add` умножает смещение на `sizeof(T)`, `AddByteOffset` добавляет байты как есть. `AreSame` - это сравнение адресов, а не содержимого.

Сырая память:

```
Unsafe.CopyBlock(dest, src, byteCount)          // Unsafe.cs:345, как memcpy
Unsafe.CopyBlockUnaligned(dest, src, byteCount) // Unsafe.cs:384, без требования
выравнивания
Unsafe.InitBlock(start, value, byteCount)       // Unsafe.cs:503, как memset
Unsafe.ReadUnaligned<T>(ref source)             // Unsafe.cs:596
Unsafe.WriteUnaligned<T>(ref dest, value)       // Unsafe.cs:648
```

`CopyBlock` компилируется в `cpblk`, `InitBlock` в `initblk`, а `ReadUnaligned` / `WriteUnaligned` добавляют к обычной загрузке/записи префикс `unaligned. 0x1`. Это критично на ARM, где невыровненный доступ может стоить дорого или падать.

Наконец, экзотика:

```
Unsafe.SizeOf<T>()       // Unsafe.cs:51
Unsafe.NullRef<T>()      // Unsafe.cs:773, ссылка, указывающая в никуда
Unsafe.IsNullRef(ref x) // Unsafe.cs:794
Unsafe.Unbox<T>(object) // Unsafe.cs:983, mutable ref на бокс
Unsafe.SkipInit(out x)  // Unsafe.cs:815, обойти definite assignment
```

Покажу каждую категорию на маленьком примере, потому что по одним сигнатурам смысл не всегда очевиден.

Переинтерпретация. `Unsafe.As` не трогает байты, только меняет тип ссылки. Классический пример - посмотреть на биты `int` как на `float`:

```
int bits = 0x3F800000;
float f = Unsafe.As<int, float>(ref bits);
Console.WriteLine(f); // 1
```

Байты не изменились, мы просто сказали компилятору "считай эти четыре байта числом с плавающей точкой". `BitCast` делает то же самое, но проверяет, что размеры совпадают:

```
float f2 = Unsafe.BitCast<int, float>(0x3F800000); // ок
// long bad = Unsafe.BitCast<int, long>(0);        // ошибка компиляции: размеры разные
```

Арифметика по ссылкам. `Unsafe.Add` двигает ссылку на N элементов, сам умножая N на размер типа:

```
int[] data = { 10, 20, 30, 40 };
ref int first = ref MemoryMarshal.GetArrayDataReference(data);
ref int third = ref Unsafe.Add(ref first, 2);
Console.WriteLine(third); // 30
```

Здесь `Add(ref first, 2)` прибавляет `2 * sizeof(int)` байтов. `AddByteOffset` сделал бы то же, но с байтами напрямую.

Сырая память. `CopyBlock` работает как `memcpy`:

```
Span<byte> src = stackalloc byte[] { 1, 2, 3, 4 };
Span<byte> dst = stackalloc byte[4];
Unsafe.CopyBlock(ref dst[0], ref src[0], (uint)src.Length);
```

А `InitBlock` как `memset`:

```
Span<byte> buffer = stackalloc byte[16];
Unsafe.InitBlock(ref buffer[0], 0xAB, (uint)buffer.Length); // все байты 0xAB
```

Экзотика. `SkipInit` позволяет объявить переменную, которую вы гарантированно заполните сами, и не платить за зануление:

```
Unsafe.SkipInit(out int value);
value = 42; // если этого не сделать, будет мусор, а не 0
```

`Unbox` дает мутабельную ссылку на значение внутри бокса:

```
object boxed = 42;
ref int inside = ref Unsafe.Unbox<int>(boxed);
inside = 100; // меняет значение прямо в боксе
```

Обычный `unbox` дает копию, а `Unsafe.Unbox` - ссылку на сами данные в куче. Работает только для значимых типов и требует, чтобы бокс был именно этого типа.

SizeOf и почему не sizeof(T)

`sizeof(T)` в C# требует, чтобы `T` был unmanaged. Это неудобно: `MemoryMarshal.Cast` и `Span` должны уметь работать с любым `T`, включая структуры с ссылками. Для этого есть `Unsafe.SizeOf<T>()` (`Unsafe.cs:51`):

```
[Intrinsic]
public static int SizeOf<T>()
{
    return sizeof(T);
}
```

JIT подставляет реальный размер типа. Для ссылочных полей он считает их как размер указателя. И когда `T` известен на этапе компиляции, `SizeOf<T>()` сворачивается в константу, так что это не вызов, а число. Именно на этом строится вся арифметика `MemoryMarshal`.

GetArrayDataReference: заглядываем за заголовок массива

Как `Span<T>` получает указатель на данные массива, минуя проверку границ? Через `MemoryMarshal.GetArrayDataReference` (`MemoryMarshal.cs:118`):

```
[Intrinsic]
public static unsafe ref byte GetArrayDataReference(Array array)
{
    return ref Unsafe.AddByteOffset(ref Unsafe.As<RawData>(array).Data,
        (nuint)RuntimeHelpers.GetMethodTable(array)->BaseSize - (nuint)(2 *
sizeof(IntPtr)));
}
```

Разберем по частям. Управляемый массив в памяти - это заголовок объекта, в котором лежат указатель на method table и длина (для массивов это `2 * sizeof(IntPtr)`), а сразу за ним начинаются данные. `Unsafe.As<RawData>(array).Data` дает адрес первого поля данных, но так как `RawData` не знает реальный тип, нужно сдвинуться к началу данных. `GetMethodTable(array)->BaseSize` - это размер заголовка объекта для данного типа, а вычитание двух указателей убирает длину и указатель на method table. Результат - ровно адрес элемента с индексом 0.

Зачем так сложно, если есть `array[0]`? Потому что `array[0]` делает проверку индекса и возвращает ссылку с проверкой ковариантности. А здесь одна арифметика, и JIT может свернуть всю конструкцию в адрес без ветвлений. Этот трюк используется в конструкторах `Span<T>` (`Span.cs:52`) на каждом срезе массива.

Вот как выглядит массив `int[4]` в памяти, чтобы формула стала наглядной:

```
адрес+0:  [заголовок GC]   8 байт
адрес+8:  [MethodTable*]   8 байт  <- на int[]
адрес+16: [length = 4]     4 байта
адрес+20: [padding]        4 байта выравнивания
адрес+24: [0]              4 байта  <- сюда указывает GetArrayDataReference
адрес+28: [1]
адрес+32: [2]
адрес+36: [3]
```

`BaseSize` для `int[4]` равен 24: заголовок (8) плюс указатель на method table (8) плюс длина (4) плюс выравнивание (4). Формула вычитает два указателя из `BaseSize`, потому что `Unsafe.As<RawData>(array).Data` уже указывает на начало объекта, и нужно перепрыгнуть заголовок. В итоге получается ровно `адрес+24`.

Про ковариантность, раз уж она упомянута. В C# можно написать:

```
string[] strings = { "a", "b" };
object[] objects = strings; // ковариантность массивов
objects[0] = 42;            // InvalidCastException в рантайме
```

Компилятор разрешает присваивание, но при записи рантайм обязан проверить, что `42` совместимо с реальным типом массива `string[]`. Это и есть проверка ковариантности. `GetArrayDataReference` возвращает `ref byte` на начало данных, без понятия о типе элементов, поэтому такой проверки не делает. Быстрее, но и безопасность перекладывается на вас.

MemoryMarshal: типобезопасная прослойка

`MemoryMarshal` построен поверх `Unsafe` и добавляет ровно то, чего не хватает: проверки, которые не дает обойти обычный код, но которые можно свести к минимуму.

Получение ссылки:

```
MemoryMarshal.GetReference<T>(Span<T> span)          // MemoryMarshal.cs:82
MemoryMarshal.GetReference<T>(ReadOnlySpan<T> span)  // MemoryMarshal.cs:88
```

Это просто `ref span._reference`. Ноль работы.

Переинтерпретация с проверками - `Cast` (`MemoryMarshal.cs:155`):

```
public static unsafe Span<TTo> Cast<TFrom, TTo>(Span<TFrom> span)
{
    if (RuntimeHelpers.IsReferenceOrContainsReferences<TFrom>())
        ThrowHelper.ThrowArgument_TypeContainsReferences(typeof(TFrom));
    if (RuntimeHelpers.IsReferenceOrContainsReferences<TTo>())
```

```
            ThrowHelper.ThrowArgument_TypeContainsReferences(typeof(TTo));

        uint fromSize = (uint)sizeof(TFrom);
        uint toSize = (uint)sizeof(TTo);
        uint fromLength = (uint)span.Length;
        int toLength;
        if (fromSize == toSize)
        {
            toLength = (int)fromLength;
        }
        else if (fromSize == 1)
        {
            toLength = (int)(fromLength / toSize);
        }
        else
        {
            ulong toLengthUInt64 = (ulong)fromLength * (ulong)fromSize / (ulong)toSize;
            toLength = checked((int)toLengthUInt64);
        }

        return new Span<TTo>(ref Unsafe.As<TFrom, TTo>(ref span._reference), toLength);
    }
```

Тут три вещи, на которые стоит посмотреть. Первая: проверка `IsReferenceOrContainsReferences` защищает от того, чтобы переинтерпретировать массив с ссылками как массив байтов и испортить GC. Вторая: вычисление новой длины делается в `uint` / `ulong` и разбито на случаи, потому что JIT так лучше оптимизирует деление на константу и checked-приведение. Третья: сама переинтерпретация - это `Unsafe.As`, то есть ноль инструкций, просто новый тип для той же ссылки.

Создание span из произвольной ссылки - `CreateSpan` (`MemoryMarshal.cs:267`):

```
public static Span<T> CreateSpan<T>(scoped ref T reference, int length) =>
    new Span<T>(ref Unsafe.AsRef(in reference), length);
```

Комментарий честно предупреждает: длина не проверяется, и lifetime не валидируется. Это инструмент для тех, кто точно знает, что делает.

Извлечение backing-памяти:

```
MemoryMarshal.TryGetArray<T>(memory, out ArraySegment<T> segment)   // MemoryMarshal.cs:311
MemoryMarshal.TryGetMemoryManager<T, TManager>(memory, out mgr)     // MemoryMarshal.cs:373
MemoryMarshal.TryGetString(ReadOnlyMemory<char> memory, out string) // MemoryMarshal.cs:485
```

Это позволяет достать исходный массив или строку из `Memory<T>`, если он там есть. Например, для interop, которому нужен именно `T[]`.

И совсем отдельная история - `CreateFromPinnedArray` (`MemoryMarshal.cs:643`):

```
return new Memory<T>((object)array, start | (1 << 31), length);
```

Обратите внимание на `start | (1 << 31)`. Тот самый старший бит индекса, о котором мы говорили в уроке про `Span` / `Memory`. Здесь он и выставляется: метод сообщает `Memory<T>`, что массив уже запинен, и `Pin()` не будет создавать `GCHandle`. Естественно, если массив на самом деле не запинен, вы получите порчу памяти.

Что происходит с процессором

`Unsafe` и `MemoryMarshal` - это не библиотечные вызовы, а почти всегда ноль или одна инструкция.

- `Unsafe.As` и `Unsafe.AsRef` в JIT исчезают совсем: это переименование типа для одной и той же ссылки.
- `Unsafe.SizeOf<T>()` сворачивается в константу, когда `T` статически известен.
- `Unsafe.Add` превращается в `add` с уже домноженным смещением.
- `CopyBlock` / `InitBlock` ложатся в `cpblk` / `initblk`, которые на больших размерах разворачиваются в векторизованный цикл.
- `GetArrayDataReference` дает адрес без проверки границ, так что цикл по `Span` может обойтись вообще без bounds check.

Покажу на псевдоассемблере, что это значит. Вот как примерно выглядит обычное обращение к массиву с проверкой границ:

```
; array[i] = 5
cmp rcx, [rdx+16]      ; сравнить i с длиной массива
jae out_of_range       ; если i >= length, прыгнуть на исключение
mov dword [rdx+rcx*4+24], 5  ; записать по адресу данных
```

А вот что получается, если адрес данных получен через `GetArrayDataReference` и цикл ограничен длиной:

```
; для i в [0, length)
mov dword [rbase+rcx*4], 5   ; просто запись, ни cmp, ни jae
```

Проверка исчезла совсем. На массиве из миллиона элементов это экономит миллион сравнений и ветвлений, а ветвления на границах еще и мешают предсказателю переходов.

Еще пример, `Unsafe.As`:

```
long value = 0x1122334455667788;
double asDouble = Unsafe.As<long, double>(ref value);
```

После JIT от `Unsafe.As` не остается ничего, потому что это переименование типа для одной и той же ссылки. Регистр с `value` просто используется дальше как `double`.

Именно поэтому эти API появились: не чтобы писать опасный код ради опасности, а чтобы убрать из горячего пути проверки, которые JIT не может доказать самостоятельно.

Практика: читаем бинарный протокол

Допустим, у нас есть заголовок пакета, и мы хотим прочитать из него поля без копирования.

```
using System;
using System.Runtime.CompilerServices;
using System.Runtime.InteropServices;
```

```
[StructLayout(LayoutKind.Sequential, Pack = 1)]
struct PacketHeader
{
    public uint Magic;
    public ushort Version;
    public ushort Flags;
    public int PayloadLength;
}

static PacketHeader ReadHeader(ReadOnlySpan<byte> buffer)
{
    if (buffer.Length < Unsafe.SizeOf<PacketHeader>())
        throw new ArgumentException("buffer too small");

    // копирует байты в локальную структуру без выравнивания на источник
    return MemoryMarshal.Read<PacketHeader>(buffer);
}
```

`MemoryMarshal.Read<T>` (`MemoryMarshal.cs:509`) внутри проверяет, что `T` не содержит ссылок и что буфера хватает, а потом читает через `Unsafe.ReadUnaligned<T>` (`MemoryMarshal.cs:520`, сам интринсик в `Unsafe.cs:571`), то есть умеет работать с невыровненной памятью. Для чтения из сети, где данные не выровнены, это ровно то, что нужно.

А вот как разбить буфер байтов на массив `ushort` без копии:

```
ReadOnlySpan<byte> raw = stackalloc byte[8];
Span<ushort> words = MemoryMarshal.Cast<byte, ushort>(raw); // 4 элемента
```

Если нужно наоборот, сериализовать структуру в байты:

```
static void WriteHeader(Span<byte> destination, in PacketHeader header)
{
    MemoryMarshal.Write(destination, in header);
}
```

Ни одной аллокации, ни одного `BitConverter`, только арифметика и пара инструкций.

Соберем более полный пример: разбор пакета, где первые 4 байта - длина в big-endian, дальше идет полезная нагрузка.

```
using System;
using System.Buffers.Binary;
using System.Runtime.CompilerServices;
using System.Runtime.InteropServices;

static bool TryReadPacket(ReadOnlySpan<byte> buffer, out ReadOnlySpan<byte> payload)
{
    payload = default;
    if (buffer.Length < 4)
        return false;

    // сетевой порядок байтов: big-endian
    uint length = BinaryPrimitives.ReadUInt32BigEndian(buffer);

    // проверить, что длина не превышает буфер и не переполняет int
```

```
    if (length > (uint)(buffer.Length - 4))
        return false;

    payload = buffer.Slice(4, (int)length);
    return true;
}
```

Здесь `BinaryPrimitives.ReadUInt32BigEndian` решает проблему порядка байтов, а `MemoryMarshal` для этого не нужен, потому что мы не переинтерпретируем структуру, а читаем число явно. Если же у вас структура в native-порядке байтов (например, из общей памяти с нативным кодом), используйте `MemoryMarshal.Read`:

```
// структура лежит в native-порядке байтов
PacketHeader header = MemoryMarshal.Read<PacketHeader>(buffer);
ReadOnlySpan<byte> body = buffer.Slice(Unsafe.SizeOf<PacketHeader>());
```

А если хочется разом разобрать массив чисел из байтов без копирования:

```
ReadOnlySpan<byte> raw = new byte[] { 1, 0, 2, 0, 3, 0, 4, 0 };
ReadOnlySpan<ushort> numbers = MemoryMarshal.Cast<byte, ushort>(raw);
// numbers.Length == 4, значения 1, 2, 3, 4 (на little-endian)
```

Обратите внимание: `Cast` не копирует. `numbers` смотрит в тот же самый массив `raw`. Если изменить `raw`, изменится и `numbers`, и наоборот (для мутабельного span).

И напоследок, чтение структуры с диска через `unsafe`, чтобы стало понятно, чем это отличается от `MemoryMarshal`:

```
static unsafe PacketHeader ReadHeaderUnsafe(byte* p)
{
    // разыменование сырого указателя, без проверок
    return *(PacketHeader*)p;
}
```

Это работает, но: нет проверки длины буфера, нет проверки выравнивания, нет проверки на ссылки внутри `PacketHeader`. `MemoryMarshal.Read` делает часть этих проверок за вас и не требует `unsafe`-блока. Поэтому в обычном коде предпочитайте его, а `unsafe` оставляйте для мест, где накладные расходы реально мешают.

Ловушки

`Unsafe.As<TFrom, TTo>` не проверяет размеры. Если `sizeof(TFrom) != sizeof(TTo)`, вы получите мусор или выйдете за границы. Для безопасной версии есть `BitCast`, который падает при несовпадении.

Ссылка из `Unsafe.AsRef(void*)` не отслеживается GC. Объект может сдвинуться, и ссылка станет висячей. Для `void*` объект нужно пинить через `fixed` или `GCHandle`.

`CreateSpan` не проверяет длину. Вы можете создать span длиннее реальной памяти. Рантайм не спасет, это ваша ответственность.

`Cast` **требует blittable-типов и уважения к выравниванию.** На x86 невыровненный доступ прощается, на ARM может стоить дорого или падать. Если источник не выровнен, читайте через `ReadUnaligned`, а не через прямую переинтерпретацию.

Endianness остается на вас. `MemoryMarshal.Read<PacketHeader>` читает в порядке байтов платформы. Для сетевых протоколов, где все big-endian, нужен `BinaryPrimitives.ReadUInt32BigEndian` или ручное переставление. Рантайм не будет переставлять байты за вас.

`Unsafe.As<T>(object)` **не проверяет тип.** Классический способ получить объект, у которого method table от одного типа, а данные от другого. Диспетчеризация пойдет не туда, и падение будет далеко от места ошибки.

`CreateFromPinnedArray` **требует реального пина.** Если массив не запинен, а вы пометили его как запиненный, GC его сдвинет, и `Memory<T>.Pin` вернет неверный указатель.

`MemoryMarshal.AsMemory` **на** `ReadOnlyMemory<char>` **поверх строки дает мутабельный доступ к неизменяемой строке.** Это технически дыра в типах, но она существует, чтобы один и тот же буфер мог быть то `ReadOnlyMemory<char>`, то `Memory<char>`. Не пишите в него, если не хотите странных багов.

Частые ошибки на примерах

Соберу главные грабли в виде "было / стало".

Переинтерпретация несовпадающих размеров.

```
// было: читаем мусор за пределами int
long value = 0;
int x = Unsafe.As<long, int>(ref value); // нет проверки размера

// стало: проверяемый вариант
// int x = Unsafe.BitCast<long, int>(value); // ошибка компиляции, размеры разные

// если действительно нужно взять младшие 4 байта, делайте это явно
int low = (int)value;
```

Ссылка на незапиненный объект.

```
// было: указатель может стать висячим
byte[] buffer = new byte[1024];
unsafe
{
    byte* p = (byte*)Unsafe.AsPointer(ref buffer[0]);
    // GC может подвинуть buffer прямо здесь
    NativeCall(p);
}

// стало: пиним на время использования
unsafe
{
    fixed (byte* p = buffer)
    {
        NativeCall(p);
```

```
    }
}
```

Создание span длиннее памяти.

```
// было: span длиной 1000 поверх 10 байт
Span<byte> small = stackalloc byte[10];
Span<byte> bad = MemoryMarshal.CreateSpan(ref small[0], 1000); // порча памяти

// стало: длина из реального буфера
Span<byte> good = MemoryMarshal.CreateSpan(ref small[0], small.Length);
```

Невыровненный Cast.

```
// было: на ARM может упасть или работать медленно
ReadOnlySpan<byte> raw = buffer.Slice(1); // сдвиг на 1 байт ломает выравнивание
ReadOnlySpan<int> ints = MemoryMarshal.Cast<byte, int>(raw);

// стало: читаем по одному через ReadUnaligned
for (int i = 0; i + 4 <= raw.Length; i += 4)
{
    int value = Unsafe.ReadUnaligned<int>(ref Unsafe.AsRef(in raw[i]));
    // ...
}
```

Забытый порядок байтов.

```
// было: на little-endian машине прочитает наоборот
uint value = MemoryMarshal.Read<uint>(networkBuffer);

// стало: явно указываем сетевой порядок
uint value2 = BinaryPrimitives.ReadUInt32BigEndian(networkBuffer);
```

Помеченный как запиненный, но не запиненный массив.

```
// было: Memory думает, что массив запинен, а GC двигает его
byte[] notPinned = new byte[1024];
Memory<byte> bad = MemoryMarshal.CreateFromPinnedArray(notPinned, 0, notPinned.Length);

// стало: либо реально запинить через POH
byte[] pinned = GC.AllocateUninitializedArray<byte>(1024, pinned: true);
Memory<byte> good = MemoryMarshal.CreateFromPinnedArray(pinned, 0, pinned.Length);

// либо не использовать этот метод и брать обычный AsMemory
Memory<byte> safe = notPinned.AsMemory();
```

Что в итоге

`Unsafe` - это набор интринсиков, которые JIT превращает в одну инструкцию или в ноль. Он дает переинтерпретацию, арифметику по ссылкам, сырые копирования и чтения без выравнивания. `MemoryMarshal` - это слой поверх него, который добавляет проверки на ссылки внутри типов, вычисляет новую длину при переинтерпретации и умеет доставать backing-память из `Span` / `Memory`.

Вместе они дают то, чего не хватает обычному C#: работу с памятью как с байтами, но с сохранением managed-ссылок, которые GC продолжает видеть. Пользуйтесь ими там, где профилировщик показал реальную стоимость проверок или копирований, и держите их подальше от бизнес-логики. Это инструмент для узкого горячего пути, а не стиль жизни.

Структура NativeAOT: как C# превращается в один нативный бинарник

Обычный .NET тащит с собой JIT: IL лежит в сборках, а машинный код рождается на ходу, при первом вызове метода. NativeAOT выворачивает это наизнанку. Компиляция целиком происходит на этапе сборки, IL больше не нужен, JIT в рантайме нет вообще. На выходе один нативный исполняемый файл, который стартует как C++ программа и не требует установленного .NET.

Разберем, из чего эта штука собрана. В репозитории есть две большие половины: компилятор в `src/coreclr/tools/aot` и рантайм с урезанной библиотекой в `src/coreclr/nativeaot`.

На пальцах: что вообще происходит

Представьте две столовые. Первая держит повара, который готовит каждое блюдо только когда его заказали. Плюс: меню бесконечное, повар подстраивается под гостя. Минус: каждый заказ ждать, а повар занимает место на кухне. Это JIT.

Вторая столовые привозит еду уже готовой в контейнерах. Плюс: подал на стол мгновенно, повар не нужен. Минус: меню зафиксировано заранее, и все, что не положили в контейнер, гость не получит. Это NativeAOT.

Разница в том, **когда** происходит превращение C# в машинный код. Обычный .NET компилирует C# в промежуточный байт-код под названием IL. IL - это не машинный код, а что-то вроде универсальной инструкции для виртуальной машины: "положи на стек", "сложи", "вызови метод". При запуске JIT читает IL метода и на ходу переводит его в инструкции конкретного процессора.

NativeAOT переводит весь IL в машинный код заранее, при сборке. В готовом бинарнике нет ни IL, ни JIT. Есть обычный машинный код, как будто программу написали на C++.

Вот как выглядит один и тот же код в обоих мирах. Возьмем такой C#:

```
static int Add(int a, int b) => a + b;

static void Main()
{
    int sum = Add(2, 3);
    Console.WriteLine(sum);
}
```

При JIT происходит так. Компилятор C# кладет в сборку IL метода `Add`:

```
.method static int32 Add(int32 a, int32 b)
{
    ldarg.0        // положить a на стек
    ldarg.1        // положить b на стек
    add            // сложить два верхних значения
```

```
    ret             // вернуть результат
}
```

При первом вызове `Add` рантайм замечает, что у метода еще нет машинного кода, зовет JIT, тот выдает что-то вроде:

```
lea eax, [rcx + rdx]   ; eax = a + b
ret
```

и только потом вызов выполняется. При NativeAOT этот `lea/ret` уже лежит в бинарнике с самого начала, а IL из сборок можно выбросить. JIT в рантайме просто нет.

Теперь важный момент, из которого растет все остальное. Раз JIT нет, код нельзя дооптимизировать в рантайме. Зато компилятор в момент сборки видит **всю программу целиком** и может позволить себе то, что JIT не может: выбросить неиспользуемое, заменить виртуальные вызовы прямыми, посчитать статику заранее. Этим и объясняется большая часть архитектуры, которую мы разбираем.

Словарь терминов

Дальше по тексту будут встречаться слова, которые стоит расшифровать сразу. Сюда можно возвращаться.

AOT (Ahead-Of-Time) - компиляция заранее, до запуска программы. Противоположность JIT.

JIT (Just-In-Time) - компиляция в момент выполнения, при первом вызове метода.

IL (Intermediate Language) - промежуточный байт-код, в который компилятор C# превращает ваш код. Не машинный код, а описание операций для виртуальной машины.

ILC (IL Compiler) - компилятор NativeAOT. Читает IL и выдает машинный код и объектные файлы.

RyuJIT - бэкенд кодогенерации, который превращает IL в машинные инструкции. Его использует и CoreCLR (в рантайме), и ILC (на этапе сборки). Один и тот же движок, разное время вызова.

Root (корень компиляции) - точка, с которой начинается обход программы. Все, что достижимо от корней, попадает в бинарник.

Dependency graph (граф зависимостей) - модель "кто кого использует". Узлы - типы и методы, ребра - ссылки между ними. Компилятор обходит его от корней.

Trimming (тримминг) - удаление всего, что не достижимо от корней.

Scanner (сканер) - предварительный проход по IL до настоящей кодогенерации. Собирает факты: что создается, что вызывается, что можно оптимизировать.

Devirtualization (девиртуализация) - замена виртуального вызова (через таблицу) на прямой вызов конкретного метода, когда компилятор доказал, что других вариантов нет.

Method table (MethodTable) - структура, описывающая тип: размер объекта, флаги, базовый тип, список интерфейсов, vtable. В CoreCLR ее называют MethodTable, в старых текстах про .NET Native - EEType. Это

одно и то же.

Vtable (таблица виртуальных методов) - массив указателей на методы, который лежит сразу за заголовком MethodTable. По индексу в этом массиве работает виртуальный вызов.

TypeManager - объект рантайма, соответствующий одному скомпилированному модулю. Знает про method table'ы, generic-словари и статику своего модуля. Аналог модуля в CoreCLR.

Bookend - пара символов-маркеров ("закладка") начала и конца секции. Линкер склеивает все куски одной секции между закладками, и рантайм получает непрерывный диапазон.

Classlib - базовая библиотека классов: `Object`, `String`, `Array` и так далее. У NativeAOT есть своя минимальная classlib, `Runtime.Base`.

PAL (Platform Abstraction Layer) - слой, который прячет различия между Windows и Unix под общим API. Чтобы код рантайма не был усыпан `#ifdef`.

EH (Exception Handling) - обработка исключений. В NativeAOT устроена иначе, чем в CoreCLR, и завязана на диапазоны управляемого кода.

Stack walking - обход стека вызовов: от текущего кадра к вызывающему и так далее. Нужен для stack trace, исключений и сборки мусора.

Bump pointer - аллокация сдвигом указателя: "взять текущую позицию, сдвинуть на размер, вернуть старое значение". Самый быстрый способ выделить память.

Tiered compilation - многоуровневая JIT-компиляция: сначала быстрый неоптимизированный код, потом, для горячих методов, оптимизированный. У NativeAOT этого нет.

PGO (Profile-Guided Optimization) - оптимизация на основе профиля реального выполнения. JIT может собрать профиль на ходу, ILC - только заранее, из внешних данных.

rd.xml - XML-файл, в котором перечисляют типы, нужные рефлексии, чтобы тримминг их не выбросил.

Vector - аппаратно-ускоренный векторный тип. JIT подбирает его размер под процессор в рантайме, ILC обязан решить это на этапе сборки.

Sync block / thin lock - служебные данные в заголовке объекта: хэш-код, тонкая блокировка, индекс записи в таблице синхронизации.

Blittable - тип, у которого управляемое и нативное представление в памяти совпадают, поэтому его можно передавать в нативный код без конвертации.

FCIMPL / HCIMPL - макросы, которыми в рантайме объявляют нативные функции, вызываемые из управляемого кода. `FCIMPL1` означает "один аргумент".

HANDLE - дескриптор, непрозрачный числовой идентификатор ресурса ОС (модуля, потока и т.д.).

Две половины

Компилятор (ILC) живет в `src/coreclr/tools/aot`. Ключевые проекты:

- `ILCompiler` - точка входа, разбор командной строки, оркестрация.
- `ILCompiler.Compiler` - сам компилятор: dependency analysis, code generation.
- `ILCompiler.TypeSystem` - собственная модель типов, не зависящая от рантайма.
- `ILCompiler.RyuJit` - бэкенд кодогенерации, тот же RyuJIT, что и в CoreCLR.
- `ILCompiler.DependencyAnalysisFramework` - движок графа зависимостей.

Рантайм живет в `src/coreclr/nativeaot`:

- `Bootstrap` - нативный C++ `main`, который запускает все.
- `Runtime` - нативный рантайм на C++: GC-интеграция, потоки, EH, stack walk, type manager.
- `Runtime.Base` - минимальная управляемая библиотека (Object, String, Array, Exception), которая компилируется прямо в рантайм.
- `System.Private.CoreLib` - настоящая CoreLib для NativeAOT.
- `Common` - общий код между компилятором и рантаймом: `MethodTable`, `StartupCodeHelpers`.
- `System.Private.TypeLoader`, `System.Private.Reflection.Execution` - загрузка типов и рефлексия.

Важно понять разделение труда. Компилятор - это обычная .NET-программа, которая запускается на вашей машине при `dotnet publish`. Она читает сборки и генерирует объектные файлы. Рантайм - это C++ код, который компилируется нативным компилятором (MSVC, clang, gcc) и линкуется вместе с этими объектными файлами. Дальше эти две части встречаются только в готовом бинарнике.

Проект `Common` - это то, что нужно обеим половинам. Например, `MethodTable` должен быть описан одинаково и в компиляторе (чтобы генерировать правильные данные), и в рантайме (чтобы их читать). Поэтому общий код лежит в `Common/src/Internal/Runtime` и подключается к обоим.

Еще полезно держать в голове такую аналогию. Компилятор - это проектировщик, который составляет чертеж дома. Рантайм - это фундамент и коммуникации, которые уже стоят на участке. Линкер - это стройка, которая сводит чертеж и фундамент вместе.

Пайплайн компиляции

Точка входа компилятора - `Program.Run` (`ILCompiler/Program.cs:72`). Порядок примерно такой.

Сначала создается контекст системы типов (`Program.cs:126`):

```
CompilerTypeSystemContext typeSystemContext =
    new CompilerTypeSystemContext(targetDetails, genericsMode, supportsReflection ?
DelegateFeature.All : 0,
        genericCycleDepthCutoff: Get(_command.MaxGenericCycleDepth),
        genericCycleBreadthCutoff: Get(_command.MaxGenericCycleBreadth));
```

Это отдельный, самостоятельный type system. Он читает входные сборки и reference-сборки, но ничего не знает про CoreCLR. Компилятор должен уметь ответить на любой вопрос о типах сам, потому что в рантайме спросить будет некого.

Что такое "type system" в этом контексте. Это программная модель, которая отвечает на вопросы о типах: какой у типа базовый класс, какие поля, какой размер, какие методы, совместим ли один тип с другим. В CoreCLR такая модель встроена в рантайм, потому что типы могут подгружаться на ходу. У компилятора она своя, потому что все типы известны заранее и рантайм ему недоступен.

Что такое "reference-сборки". Это сборки-заглушки, в которых есть только сигнатуры, без реализации. Они нужны, чтобы компилятор знал, как выглядят типы из платформы, но не тащил в анализ чужой код. Скажем, `System.Runtime.dll` из reference pack говорит "есть тип `System.String` с такими-то методами", и этого достаточно для проверки типов.

Дальше собираются **корни компиляции** (`Program.cs:171`). Root - это то, с чего начинается обход графа: метод `Main` , инициализаторы библиотек, точки входа `UnmanagedCallersOnly` , типы из rd.xml. Все, что достижимо от корней, попадет в бинарник. Все, что нет, будет выброшено. Это и есть тримминг, только тотальный.

Вот как это выглядит на примере. Пусть программа такая:

```
class Program
{
    static void Main()
    {
        Console.WriteLine(Greeting());
    }

    static string Greeting() => "hello";

    static void NeverCalled() => Console.WriteLine("dead code");
}
```

Корень - `Main` . От него граф тянется к `Console.WriteLine` и `Greeting` . Метод `NeverCalled` ни от кого не достижим, значит его машинного кода в бинарнике не будет вообще. Если бы это была библиотека, корнями были бы еще и все публичные API, помеченные как экспортируемые.

Что такое "граф зависимостей" и зачем он нужен. Компилятор не может просто пройтись по всем методам подряд: тогда он сгенерирует код для того, что никогда не вызовется. Вместо этого он строит граф: "метод A вызывает метод B", "метод B создает тип C", "тип C содержит поле типа D". Обход от корней по этому графу дает точный список всего нужного. Это и есть тот самый whole-program view, которого нет у JIT.

Схематично:

```
Main
 ├── Console.WriteLine
 └── Greeting
       └── (возвращает string)

NeverCalled          <- ни одного входящего ребра, выброшен
```

Потом запускается **сканер** (`Program.cs:510`). Он проходит по всему графу в режиме интерпретации IL и собирает факты: какие типы реально создаются, какие методы реально вызываются, какие интерфейсные вызовы можно девиртуализировать, какие статические поля можно проинициализировать. Результаты сканера отдаются билдеру (`Program.cs:552`):

```
builder.UseVTableSliceProvider(scanResults.GetVTableLayoutInfo());
builder.UseGenericDictionaryLayoutProvider(scanResults.GetDictionaryLayoutInfo());
builder.UseDevirtualizationManager(devirtualizationManager);
builder.UseInliningPolicy(scanResults.GetInliningPolicy());
```

Вот где NativeAOT выигрывает. Компилятор видит всю программу целиком. Если тип ни разу не был создан, его method table можно не генерировать. Если метод никто не переопределяет, вызов становится прямым. Это то, чего JIT не может, потому что он компилирует методы по одному и не знает будущего.

Разберем девиртуализацию подробнее, потому что это самый наглядный выигрыш. Пусть есть интерфейс и одна реализация:

```
interface IShape { double Area(); }

class Circle : IShape
{
    public double Radius;
    public double Area() => Math.PI * Radius * Radius;
}

class Program
{
    static double Total(IShape s) => s.Area() * 2;

    static void Main()
    {
        var c = new Circle { Radius = 1.0 };
        Console.WriteLine(Total(c));
    }
}
```

JIT видит метод `Total(IShape s)` и не знает, что в него передадут. Значит, он генерирует виртуальный вызов: смотрит на реальный тип объекта в рантайме, идет в его vtable, берет указатель на `Area` и вызывает. Это лишняя загрузка и косвенный переход.

Компилятор NativeAOT проходит по графу и обнаруживает, что `Circle` - единственная реализация `IShape`, и объект создается только как `Circle`. Значит, вызов `s.Area()` можно заменить на прямой вызов `Circle.Area`:

```
// было
static double Total(IShape s) => s.Area() * 2;

// после девиртуализации компилятором (схематично)
static double Total(IShape s) => Circle.Area((Circle)s) * 2;
```

Выигрыш двойной: исчезает косвенный переход, и открывается дорога к инлайну. JIT тоже умеет девиртуализировать, но только когда видит профиль или локальный контекст, а не всю программу.

Еще один трюк - **преинициализация статики**. Если статическое поле можно вычислить на этапе сборки, компилятор посчитает его и положит в бинарник готовое значение, вместо того чтобы выполнять код при старте. Например:

```
static readonly int[] Primes = ComputePrimes();
```

Если `ComputePrimes` детерминирована и не зависит от внешних данных, ILC может выполнить ее во время сборки и записать результат прямо в данные. Рантайм тогда ничего не считает.

Сама кодогенерация идет через `RyuJitCompilationBuilder` (`Program.cs:366`), который внутри вызывает тот же RyuJIT, что и CoreCLR. Разница в том, что здесь он вызывается на этапе сборки, а не в рантайме. Финальный шаг:

```
ICompilation compilation = builder.ToCompilation();
CompilationResults compilationResults = compilation.Compile(tempOutputFilePath,
ObjectDumper.Compose(dumpers));
```

`Compile` (`Program.cs:654`) выдает набор объектных файлов, которые потом линкуются с рантаймом в один бинарник.

Точка входа: bootstrap

Нативный `main` лежит в `Bootstrap/main.cpp`. Именно он первым получает управление. Есть два режима: обычное приложение и нативная библиотека (DLL). Вот как выбирается точка входа (`main.cpp:179`):

```
#ifndef NATIVEAOT_DLL
#define NATIVEAOT_ENTRYPOINT __managed__Main
...
extern "C" int __managed__Main(int argc, wchar_t* argv[]);
#else
#define NATIVEAOT_ENTRYPOINT __managed__Startup
extern "C" void __managed__Startup();
#endif
```

`__managed__Main` - это ваш `Main`, скомпилированный в нативный код с C-соглашением о вызовах. Для рантайма это обычная extern-функция.

Разберемся с терминами в этом фрагменте. `extern "C"` в C++ отключает искажение имен (name mangling). Компилятор C++ обычно превращает имя `Foo(int)` в нечто вроде `_Z3Fooi`, чтобы поддерживать перегрузки. Для линковки с кодом, где таких правил нет, нужно `extern "C"`: тогда имя остается `Foo`. Именно поэтому управляемые точки входа получают предсказуемые имена, которые ищет линкер.

"Соглашение о вызовах" (calling convention) - это правила о том, кто кладет аргументы в регистры, кто чистит стек, кто восстанавливает регистры. У `__managed__Main` на Windows x64 это `__cdecl`, на других платформах обычное системное. Компилятор ILC генерирует функцию ровно по этим правилам, чтобы нативный `main` мог ее вызвать, ничего не зная про .NET.

Что происходит по шагам, когда вы запускаете бинарник:

```
ОС
 └── вызывает main() из Bootstrap/main.cpp
       └── InitializeRuntime()
```

```
            ├── RhInitialize()          // поднять PAL, рантайм, GC
            ├── RhRegisterOSModule()    // отдать диапазоны кода и таблицу classlib
            └── InitializeModules()     // создать TypeManager'ы, зарегистрировать модули
        └── вызывает __managed__Main()  // это ваш Main
            └── ... ваш код ...
```

Никакой магии: нативный `main` просто вызывает управляемую функцию как обычную C-функцию. Все, что делает NativeAOT, - заранее превращает `Main` в такую функцию.

Инициализация (`main.cpp:190`):

```
static int InitializeRuntime()
{
    if (!RhInitialize(/* isDll */ false))
        return -1;

    void * osModule = PalGetModuleHandleFromPointer((void*)&NATIVEAOT_ENTRYPOINT);

    if (!RhRegisterOSModule(
        osModule,
        (void*)&__managedcode_a, (uint32_t)((char *)&__managedcode_z -
(char*)&__managedcode_a),
        (void*)&__unbox_a, (uint32_t)((char *)&__unbox_z - (char*)&__unbox_a),
        (void **)&c_classlibFunctions, _countof(c_classlibFunctions)))
    ...
    InitializeModules(osModule, __modules_a, (int)((__modules_z - __modules_a)), ...);
    return 0;
}
```

Здесь интересна механика с bookends. ILC генерирует объектные файлы с секциями, а линкер склеивает их в непрерывные диапазоны. `__managedcode_a` и `__managedcode_z` - маркеры начала и конца всего управляемого кода. Тот же трюк с `.modules` для списка модулей и `.unbox` для unboxing-стабов (`main.cpp:25`, `main.cpp:54`, `main.cpp:65`).

Поясню, что такое секция и линкер, потому что это ключ ко всему механизму. Объектный файл - это результат компиляции одного файла кода. Внутри он разбит на **секции** (их еще называют сегментами): код в одной, данные в другой, константы в третьей. Линкер - это программа, которая собирает объектные файлы в один исполняемый и раскладывает их секции по памяти.

Фокус в том, что линкер склеивает одноименные секции. Если у нас есть объектные файлы `a.obj`, `b.obj`, `c.obj`, и в каждом есть секция `.managedcode`, то линкер положит их подряд в одну общую секцию. А если задать специальные имена вида `.managedcode$A` и `.managedcode$Z`, линкер положит их по алфавиту: `$A` окажется в начале, `$Z` в конце. Получаются те самые "закладки":

```
.managedcode$A   -> символ __managedcode_a
  a.obj: метод Main
  b.obj: метод Greeting
  c.obj: метод Console.WriteLine
.managedcode$Z   -> символ __managedcode_z
```

Теперь, зная адреса `__managedcode_a` и `__managedcode_z`, рантайм знает, где начинается и где заканчивается весь управляемый код, хотя каждый метод лежал в отдельном объектном файле.

Секция `.unbox` устроена так же, только для unboxing-стабов, а `.modules` - для указателей на данные модулей.

Этот трюк не уникален для .NET. Ровно так же C++ компиляторы собирают список глобальных конструкторов: каждый файл кладет свой указатель в секцию `.CRT$XCU`, а линкер склеивает их в один массив, который выполняется при старте.

Рантайму эти диапазоны нужны, чтобы понимать, где управляемый код. Вот как это используется (`Runtime/RuntimeInstance.cpp:95`):

```
bool RuntimeInstance::IsManaged(PTR_VOID pvAddress)
{
    return (dac_cast<TADDR>(pvAddress) - dac_cast<TADDR>(m_pvManagedCodeStartRange) <
m_cbManagedCodeRange);
}
```

Проверка адреса на попадание в управляемый код нужна для stack walking, разбора исключений и поиска method info по return address.

Еще один важный кусок - таблица функций classlib (`main.cpp:150`). Рантайм не знает, как создать исключение или разрешить интерфейсный вызов, потому что это управляемый код. Поэтому bootstrap передает ему указатели:

```
static const pfn c_classlibFunctions[] = {
    &MANAGED_RUNTIME_EXPORT_NAME(GetRuntimeException),
    &MANAGED_RUNTIME_EXPORT_NAME(RuntimeFailFast),
    (pfn)&ThreadEntryPoint,
    &MANAGED_RUNTIME_EXPORT_NAME(AppendExceptionStackFrame),
    &MANAGED_RUNTIME_EXPORT_NAME(ResolveDispatch),
    ...
};
```

Это разрыв циклической зависимости: нативный рантайм дергает управляемые функции через таблицу указателей, а не через линковку.

Почему здесь вообще цикл. Рантайму нужно уметь бросить исключение, а исключение - это управляемый объект, для которого нужна classlib. Но classlib сама написана на C# и зависит от рантайма. Получается замкнутый круг: чтобы создать исключение, нужен рантайм, а чтобы рантайм работал, нужна classlib.

Решение - инверсия зависимости. Рантайм ничего не знает о конкретных управляемых функциях. Он знает только, что по индексу `N` в переданной таблице лежит указатель на функцию создания исключения. Управляемая сторона заполняет таблицу своими методами и отдает ее рантайму при старте. Кто кого вызывает, решается в рантайме, а не на этапе линковки.

Вот что лежит в таблице и зачем:

- `GetRuntimeException` - создать объект исключения по идентификатору (например, `NullReferenceException`).
- `RuntimeFailFast` - аварийно завершить процесс, когда продолжать нельзя.

- `ThreadEntryPoint` - точка входа для нового потока, чтобы поток начал выполнять управляемый код.
- `AppendExceptionStackFrame` - добавить кадр в строящийся stack trace при разборе исключения.
- `ResolveDispatch` - разрешить интерфейсный вызов, когда компилятор не смог девиртуализировать его заранее.

Индекс в таблице и идентификатор совпадают с перечислением `ClassLibFunctionId` в управляемом коде (`InternalCalls.cs:20`):

```
internal enum ClassLibFunctionId
{
    GetRuntimeException = 0,
    FailFast = 1,
    ThreadEntryPoint = 2,
    AppendExceptionStackFrame = 3,
    ResolveDispatch = 4,
    GetSystemArrayEEType = 5,
    OnFirstChance = 6,
    OnUnhandledException = 7,
    ...
}
```

Комментарий рядом с этим перечислением предупреждает: если меняешь порядок, не забудь поменять определение в `ICodeManager.h` . Два мира связаны позицией в массиве, и компилятор за этим не следит. Классическое место для ошибок в нативном interop.

Рантайм

`RhInitialize` (`Runtime/startup.cpp:349`) инициализирует PAL, ставит обработчик выхода и зовет `InitDLL` (`startup.cpp:87`). Внутри:

```
static bool InitDLL(HANDLE hPalInstance)
{
    InitializeGCEventLock();
    ...
    if (!RestrictedCallouts::Initialize())
        return false;
    if (!RuntimeInstance::Initialize(hPalInstance))
        return false;
    ...
    if (!InitializeGC())
        return false;
    ...
    if (!DetectCPUFeatures())
        return false;
    ...
}
```

Разберем термины из этого кода. **PAL** (Platform Abstraction Layer) - это тонкий слой над системными API. Вместо того чтобы в каждом месте писать "на Windows вызови `VirtualAlloc` , а на Linux `mmap` ", рантайм вызывает `PalVirtualAlloc` , а PAL уже внутри разбирается, что делать. Благодаря этому код `RuntimeInstance` и `GCHelpers` выглядит одинаково на всех платформах.

FCIMPL4 в объявлении `RhpCreateTypeManager` - это макрос, который разворачивается в правильное объявление нативной функции, вызываемой из управляемого кода. Цифра - количество аргументов. Рантайм использует такие макросы, чтобы не повторять вручную соглашение о вызовах, переход в кооперативный режим GC и проверки на каждом вызове.

Порядок критичен: сначала базовое состояние рантайма, потом GC, потом проверка CPU. `DetectCPUFeatures` (`startup.cpp:166`) сверяет набор инструкций процессора с `g_requiredCpuFeatures`, который зашил компилятор. Если бинарник собран под AVX2, а процессор его не умеет, приложение упадет сразу с внятным сообщением, а не через полчаса на случайной инструкции. Это плата за статическую кодогенерацию.

Почему проверка CPU идет после GC, а не раньше. Потому что сообщение об ошибке нужно как-то напечатать, а для этого уже нужен работающий рантайм и память. Поэтому сначала поднимается минимум, и только потом проверяются инструкции.

`RuntimeInstance` (`RuntimeInstance.cpp:303`) - центральный объект рантайма. Он хранит список TypeManager'ов, диапазон управляемого кода, список unboxing-стабов. Через него регистрируются модули:

```
FCIMPL4(TypeManagerHandle, RhpCreateTypeManager, HANDLE osModule, void* pModuleHeader, ...)
{
    TypeManager * typeManager = TypeManager::Create(osModule, pModuleHeader,
pClasslibFunctions, nClasslibFunctions);
    GetRuntimeInstance()->RegisterTypeManager(typeManager);
    return TypeManagerHandle::Create(typeManager);
}
```

`TypeManager` - это аналог модуля в CoreCLR: он знает про свои method table'ы, generic-словари и статические данные.

Проведем аналогию. В CoreCLR модуль - это загруженная сборка. Рантайм может в любой момент попросить ее: "дай тип с таким именем". В NativeAOT сборок в привычном смысле нет, но есть скомпилированные модули, и у каждого - свой `TypeManager`, который знает все свои типы статически. Разница в том, что CoreCLR ищет тип динамически, а TypeManager читает уже готовые структуры, которые положил компилятор.

Еще одна важная задача рантайма - **stack walking**, то есть обход стека вызовов. Чтобы пройти от текущего метода к вызывающему, нужно понять, где в стеке лежит обратный адрес, какие регистры сохранены и какого размера кадр. Для этого рантайм использует **GC info** - метаданные, которые компилятор сгенерировал для каждого метода и которые говорят, где на каждом участке кода живут ссылки и как восстановить стек.

Это нужно для трех вещей: собрать stack trace при исключении, найти все живые ссылки при сборке мусора и корректно пройти по стеку при отладке. Именно поэтому рантайм хранит диапазон управляемого кода: получив произвольный адрес, он должен быстро понять, управляемый ли это код и где искать его метаданные.

Runtime.Base: минимальная CoreLib

Есть курица и яйцо: рантайму нужен `System.Object`, но полноценная CoreLib слишком сложна для старта. Поэтому существует `Runtime.Base` - крошечная управляемая библиотека, в которой есть только самое необходимое: `Object`, `String`, `Array`, `Exception`, `RuntimeHelpers`, `Unsafe`.

Она компилируется прямо в рантайм. Вот как это включается (`System.Private.CoreLib.csproj:488`):

```
<!-- For now, link Runtime.Base into System.Private.CoreLib until there is proper multifile
build -->
<PropertyGroup>
  <InPlaceRuntime>true</InPlaceRuntime>
</PropertyGroup>
<PropertyGroup Condition="'$(InPlaceRuntime)' == 'true'">
  <DefineConstants>INPLACE_RUNTIME;$(DefineConstants)</DefineConstants>
```

Дальше файлы из `Runtime.Base` просто линкуются в CoreLib как `<Compile Include="$(RuntimeBasePath)...">`. Поэтому в коде полно `#if INPLACE_RUNTIME` и `#if !INPLACE_RUNTIME`: одна и та же логика компилируется по-разному в зависимости от того, встроена она в рантайм или собрана отдельно.

Мост между управляемым и нативным здесь устроен через два атрибута. `RuntimeExport` экспортирует управляемый метод наружу под заданным именем (`RuntimeExportAttribute.cs:7`):

```
[AttributeUsage(AttributeTargets.Method, Inherited = false)]
internal sealed class RuntimeExportAttribute : Attribute
{
    public string EntryPoint;
    public RuntimeExportAttribute(string entry) { EntryPoint = entry; }
}
```

`RuntimeImport` делает обратное: объявляет нативную функцию, которую зовет управляемый код. Вместе они дают тот самый мост, по которому ходят аллокации, бокс и исключения.

Вот пример импорта в управляемом коде (`InternalCalls.cs:77`):

```
[RuntimeImport(RuntimeLibrary, "RhpNewFast")]
[MethodImpl(MethodImplOptions.InternalCall)]
internal static extern unsafe object RhpNewFast(MethodTable* pEEType);
```

Здесь `RuntimeLibrary` - это строка `"*"`, что означает "текущий бинарник". `RhpNewFast` - имя нативной функции, которую найдет линкер. `InternalCall` говорит компилятору, что тела у этого метода нет и что вызов надо превратить в нативный. Так управляемый код аллокации попадает в `portable.cpp`, который мы разберем ниже.

Отдельно про `UnmanagedCallersOnly`. Этот атрибут на управляемом методе делает его вызываемым из нативного кода напрямую, без переходного кадра. Синтаксис:

```
[UnmanagedCallersOnly(EntryPoint = "MyNativeEntry")]
static int Add(int a, int b) => a + b;
```

После компиляции из нативного кода можно объявить `extern "C" int MyNativeEntry(int, int)` и вызвать. Именно так `InitializeModules` из `StartupCodeHelpers` становится доступной нативному

`main`.

Смотрите, как выглядит бокс (`Runtime.Base/src/System/Runtime/RuntimeExports.cs:121`):

```
public static unsafe object RhBox(MethodTable* pEEType, ref byte data)
{
    ...
    result = InternalCalls.RhpNewFast(pEEType);
    if (pEEType->ContainsGCPointers)
        InternalCalls.RhBulkMoveWithWriteBarrier(ref result.GetRawData(), ref
dataAdjustedForNullable, pEEType->ValueTypeSize);
    else
        Unsafe.CopyBlock(ref result.GetRawData(), ref dataAdjustedForNullable, pEEType-
>ValueTypeSize);
    return result;
}
```

Это ровно тот же низкоуровневый код, о котором мы говорили в уроках про `Unsafe` и GC, только здесь он сам является рантаймом.

Объектная модель

В NativeAOT объект устроен максимально просто. Контракт прописан прямо в комментарии (`Runtime.Base/src/System/Object.cs:11`):

```
// CONTRACT with Runtime
// The Object type is one of the primitives understood by the compilers and runtime
// Data Contract: Single field of type MethodTable*
// VTable Contract: The first vtable slot should be the finalizer for object
```

И сам класс (`Object.cs:16`):

```
public unsafe class Object
{
    private MethodTable* m_pEEType;
}
```

Все. Указатель на method table и есть объект. Никаких sync block в объекте, как в старом .NET Framework.

Зачем вообще нужен указатель на method table в каждом объекте. Представьте, что у вас есть переменная типа `object`, а в ней лежит либо `string`, либо `int`, либо ваш класс. Как понять, что это на самом деле, и какие методы можно вызвать. Ответ: посмотреть на первый указатель в объекте. Он ведет на MethodTable, а та уже говорит, что это за тип. Отсюда и растет вся система типов в рантайме: тип объекта определяется его method table, а не тем, что написано в переменной.

Вот раскладка объекта в памяти. Пусть у нас есть такой класс:

```
class Point
{
    public int X;
```

```
    public int Y;
}
```

Тогда объект `Point` в куче выглядит так:

```
адрес+0:  [alignpad]       4 байта выравнивания
адрес+4:  [sync block]     4 байта заголовка (хэш, блокировка)
адрес+8:  [MethodTable*]   8 байт указателя на тип
адрес+16: [X]              4 байта
адрес+20: [Y]              4 байта
```

Почему так. Заголовок 32-битный и лежит вплотную перед указателем на MethodTable. На 64-битной системе перед ним добавляется 4 байта выравнивания, чтобы весь восьмибайтовый блок был выровнен. Вот нативное определение (`Runtime/ObjectLayout.h:15`):

```
class ObjHeader
{
private:
#if defined(HOST_64BIT)
    uint32_t   m_uAlignpad;
#endif // HOST_64BIT
    uint32_t   m_uSyncBlockValue;
    ...
};
```

Именно `m_uAlignpad` и есть те самые 4 байта пустоты. А `m_uSyncBlockValue` - это `sync block` в комментарии `GetHeaderPtr`, который читается как `(int*)ppMethodTable - 1`, то есть за 4 байта до указателя. Компилятор знает весь расклад и генерирует обращения к полям по смещениям: `X` - это `адрес+16`, `Y` - `адрес+20`.

Теперь про vtable. Это массив указателей на методы, который лежит сразу за заголовком MethodTable. Когда вы пишете виртуальный вызов:

```
Shape s = new Circle();
double a = s.Area();
```

Компилятор превращает это примерно в такой код на псевдо-C++:

```
// s - это указатель на объект
MethodTable* mt = *(MethodTable**)s;          // взять method table из объекта
void** vtable = mt->GetVtable();              // vtable лежит сразу за заголовком
double (*area)(void*) = vtable[AreaSlot];     // взять метод по слоту
double a = area(s);                           // вызвать
```

Индекс слота `AreaSlot` компилятор знает заранее: он одинаков для всех типов, реализующих `Area`. Именно поэтому один и тот же вызов работает для любого наследника. А девиртуализация, о которой мы говорили выше, заменяет этот танец на прямой `Circle.Area(s)`, когда компилятор доказал, что других вариантов нет.

Перед этим указателем, в памяти, лежит заголовок. Он описан так (`Common/src/Internal/Runtime/MethodTable.cs:15`):

```
[StructLayout(LayoutKind.Sequential)]
internal struct ObjHeader
{
    // Contents of the object header
    private IntPtr _objHeaderContents;
}
```

Заголовок расположен ровно за 4 байта до указателя на method table (`ObjectHeader.NativeAot.cs:49`):

```
private static unsafe int* GetHeaderPtr(MethodTable** ppMethodTable)
{
    // The header is 4 bytes before m_pEEType field on all architectures
    return (int*)ppMethodTable - 1;
}
```

В этих 4 байтах живут хэш-код объекта, тонкая блокировка (thin lock) и индекс записи в sync-таблице (`ObjectHeader.NativeAot.cs:21`):

```
//   BIT_SBLK_UNUSED        = 0x80000000
//   BIT_SBLK_FINALIZER_RUN = 0x40000000
//   BIT_SBLK_GC_RESERVE    = 0x20000000
```

Разберем, что это значит. **Хэш-код** - значение, которое возвращает `GetHashCode()` . Его невыгодно хранить в каждом объекте, потому что большинство объектов в хэш-таблицы не попадают. Поэтому он вычисляется лениво и кладется в заголовок только при первом обращении.

Thin lock (тонкая блокировка) - оптимизация для `lock` . Когда поток входит в `lock(obj)` , вместо создания полноценного объекта-мьютекса его идентификатор записывается прямо в заголовок объекта. Если объект никто не конкурирует, это одна атомарная операция вместо системного вызова. Только при реальной конкуренции создается "толстая" блокировка в **sync-таблице** - отдельном массиве, где на каждый спорный объект есть запись.

Комментарий в исходнике объясняет порог (`ObjectHeader.NativeAot.cs:254`):

```
// If we see a thin lock held by other thread for longer than ~5 microseconds, we will
"inflate" the lock
// and let the adaptive spinning in the fat Lock sort it out whether we have a contentious
lock or a long-held lock.
```

Идея в том, что тонкая блокировка хороша для коротких некотенциозных критических секций. Если ее держат долго, выгоднее "раздуть" ее в полноценный объект и отдать решение операционной системе.

Итого раскладка объекта: `[ObjHeader 4 байта][padding 4 байта][MethodTable* 8 байт][поля...]` . Именно поэтому `GetRawDataSize` вычитает оба заголовка (`Object.cs:67`):

```
internal uint GetRawDataSize()
{
    return m_pEEType->BaseSize - (uint)sizeof(ObjHeader) - (uint)sizeof(MethodTable*);
}
```

Сам `MethodTable` - это структура с фиксированным заголовком и vtable'ом следом (`MethodTable.cs:169`):

```
private uint _uFlags;
private uint _uBaseSize;
private RelatedTypeUnion _relatedType;
private ushort _usNumVtableSlots;
private ushort _usNumInterfaces;
private uint _uHashCode;

// vtable follows
```

`_uFlags` хранит флаги типа, а младшие биты - размер элемента для массивов и строк либо количество generic-параметров. `_relatedType` - это union: для обычного типа там базовый тип, для массива - тип элемента (`MethodTable.cs:137`). `_uBaseSize` - размер объекта, который так важен для аллокации.

Проясним каждый элемент, потому что здесь легко запутаться.

`_uFlags` - это одно 32-битное слово, в котором упаковано много всего. Старшие 16 бит - флаги: является ли тип интерфейсом, содержит ли ссылки, финализируемый ли он, и так далее. Младшие 16 бит переиспользуются под разные нужды в зависимости от типа: для массива и строки это размер одного элемента (чтобы посчитать размер массива из длины), для generic-типа - число параметров. Это экономия памяти: не заводить отдельные поля, если их можно совместить.

`_uBaseSize` - базовый размер объекта в байтах. Именно его читает `RhpNewFast` , чтобы сдвинуть указатель аллокации. Для `Point` из примера выше это 24 байта: 8 на заголовок с указателем и 8 на два `int` с выравниванием.

`_relatedType` - это `union` , то есть объединение, где все поля лежат по одному адресу и занимают одну память. Вот его определение (`MethodTable.cs:137`):

```
[StructLayout(LayoutKind.Explicit)]
private unsafe struct RelatedTypeUnion
{
    // Kinds.CanonicalEEType
    [FieldOffset(0)]
    public MethodTable* _pBaseType;

    // Kinds.ArrayEEType
    [FieldOffset(0)]
    public MethodTable* _pRelatedParameterType;
}
```

`[FieldOffset(0)]` у обоих полей означает "положи по смещению 0". Значит, это одна и та же память, просто два имени. Для обычного класса здесь базовый тип: `Circle` ссылается на `Shape` . Для массива здесь тип элемента: `int[]` ссылается на `int` . Читать надо правильное поле в зависимости от того, массив это или нет. Это классический прием из мира C, где память экономят ценой внимательности программиста.

`_usNumVtableSlots` и `_usNumInterfaces` - сколько слотов в vtable и сколько интерфейсов реализует тип. По ним рантайм знает размеры массивов, которые идут следом за заголовком.

Аллокация

Быстрый путь аллокации - это тот самый bump pointer. Вот он в `Runtime/portable.cpp:57`:

```
FCIMPL1(Object *, RhpNewFast, MethodTable* pEEType)
{
    ASSERT(!pEEType->HasFinalizer());

    Thread * pCurThread = ThreadStore::GetCurrentThread();
    gc_alloc_context * acontext = pCurThread->GetAllocContext();
    size_t size = pEEType->GetBaseSize();

    uint8_t* alloc_ptr = acontext->alloc_ptr;
    uint8_t* combined_limit = pCurThread->GetEEAllocContext()->GetCombinedLimit();
    if ((size_t)(combined_limit - alloc_ptr) >= size)
    {
        acontext->alloc_ptr = alloc_ptr + size;
        Object* pObject = (Object *)alloc_ptr;
        pObject->SetMethodTable(pEEType);
        return pObject;
    }

    return AllocateObject(pEEType, 0, 0);
}
```

Ничего нового после урока про GC: сдвинули указатель, записали method table, вернули объект. Отличие в `combined_limit`. Это предел, до которого можно аллоцировать, не заботясь о сэмплировании аллокаций для профилировщика. Если включено сэмплирование, `combined_limit` может быть меньше реального `alloc_limit`, и пересечение сэмплированной точки уводит на медленный путь.

Разберем по шагам, что делает этот код, на примере выделения `Point` размером 24 байта.

Пусть у потока сейчас такое состояние аллокационного контекста:

```
alloc_ptr   = 0x1000   // следующий свободный байт
alloc_limit = 0x2000   // конец текущего куска
```

Шаги:

1. `size = pEEType->GetBaseSize()` дает 24.
2. `alloc_ptr = 0x1000`, `combined_limit = 0x2000`.
3. Проверяем `0x2000 - 0x1000 = 0x1000 >= 24` - да, места хватает.
4. `acontext->alloc_ptr = 0x1000 + 24 = 0x1018` - сдвинули указатель.
5. `pObject = 0x1000`, пишем туда указатель на MethodTable.
6. Возвращаем `0x1000`.

Все. Никаких поисков свободного блока, никакой блокировки: каждый поток работает со своим контекстом. Отсюда и скорость.

Схематично:

```
до:    [занято........][ free ......................]
         ^              ^                          ^
```

```
       начало         alloc_ptr                  alloc_limit

после: [занято........][Point][ free ..............]
                        ^       ^                   ^
                      вернули  alloc_ptr          alloc_limit
```

Что такое **сэмплирование аллокаций**. Профилировщику не нужно видеть каждую аллокацию: он выбирает случайные точки в потоке аллокаций и записывает, что там было выделено. Это дешевле, чем логировать все подряд. Точка, которую надо засэмплировать, может попасть внутрь текущего куска. Тогда `combined_limit` ставится раньше `alloc_limit`, чтобы объект, накрывающий эту точку, ушел на медленный путь и был залогирован. Если сэмплирование выключено, `combined_limit` всегда равен `alloc_limit`, и проверка становится тривиальной.

Медленный путь `AllocateObject` заканчивается вызовом GC (`Runtime/GCHelpers.cpp:611`):

```
Object* pObject = GCHeapUtilities::GetGCHeap()->Alloc(pAllocContext, cbSize, uFlags);
```

То есть NativeAOT использует тот же самый сборщик из `src/coreclr/gc`, что и CoreCLR. Никакого отдельного GC тут нет. В `GCHelpers.cpp` есть даже проверка, что аллокационный квант не больше порога больших объектов (`GCHelpers.cpp:641`):

```
ASSERT(acontext->alloc_limit - acontext->alloc_ptr <= RH_LARGE_OBJECT_SIZE);
```

Запуск управляемого кода

После того как нативный `main` вызвал `RhInitialize` и `InitializeModules`, управление уходит в `__managed__Main`. Но перед `Main` нужно подготовить модули. Это делает `StartupCodeHelpers.InitializeModules` (`Common/.../StartupCodeHelpers.cs:30`):

```
[UnmanagedCallersOnly(EntryPoint = "InitializeModules")]
internal static unsafe void InitializeModules(IntPtr osModule, IntPtr* pModuleHeaders, int
count, IntPtr* pClasslibFunctions, int nClasslibFunctions)
{
    RuntimeImports.RhpRegisterOsModule(osModule);
    TypeManagerHandle[] modules = CreateTypeManagers(osModule, pModuleHeaders, count,
pClasslibFunctions, nClasslibFunctions);
    object[] gcStaticBaseSpines = new object[count];
    ...
    for (int i = 0; i < modules.Length; i++)
        RunInitializers(modules[i], ReadyToRunSectionType.EagerCctor);
}
```

Обратите внимание на комментарий (`StartupCodeHelpers.cs:49`):

```
// These two loops look funny but it's important to initialize the global tables before
running
// the first class constructor to prevent them calling into another uninitialized module
```

Сначала все модули регистрируются и получают таблицы статики, и только потом запускаются eager-конструкторы. Иначе первый же конструктор мог бы обратиться к еще не созданным данным

соседнего модуля.

Ленивые статические конструкторы запускаются позже, при первом обращении к типу, через `ClassConstructorRunner`.

Разберем, что такое статический конструктор и почему с ним столько возни. Статический конструктор - это блок, который выполняется один раз, перед первым использованием типа:

```
class Config
{
    public static readonly string Path;

    static Config()
    {
        Path = Environment.GetEnvironmentVariable("APP_PATH") ?? "default";
    }
}
```

В обычном .NET гарантируется, что этот код выполнится ровно один раз и до того, как к `Path` впервые обратятся, причем потокобезопасно. Это значит, что рантайм должен отслеживать состояние каждого типа: "конструктор еще не запускался", "выполняется прямо сейчас другим потоком", "уже выполнен".

Eager (нетерпеливые) конструкторы выполняются сразу при старте, для всех типов, которые компилятор пометил как нужные. Это те самые `RunInitializers(..., ReadyToRunSectionType.EagerCctor)`. Плюс: к моменту `Main` все готово, никаких проверок в горячем пути. Минус: старт дольше, потому что делается работа, которая могла бы не понадобиться.

Lazy (ленивые) конструкторы выполняются при первом обращении. Плюс: старт быстрее. Минус: на каждом обращении к статике нужна проверка состояния и, при необходимости, ожидание. Компилятор решает, какие конструкторы можно сделать eager, а какие нельзя (например, если они зависят от данных, доступных только в рантайме).

Метаданные и рефлексия

Тримминг выбросил все, что не используется. Значит, рефлексия по умолчанию почти ничего не найдет. Есть несколько уровней:

- `--reflectiondata:none` - рефлексии нет вообще, самый маленький бинарник.
- `--reflectiondata:all` - сгенерировать метаданные для всего, что осталось.
- `--completetypemetadata` - полные метаданные типов.

Управляется это в `Program.Run` (`Program.cs:116`):

```
bool supportsReflection = reflectionData != "none" && systemModuleName ==
Helpers.DefaultSystemModule;
```

Покажем проблему на примере. Пусть есть такой код:

```
string name = "MyApp.Models.Order";
Type t = Type.GetType(name);
```

```
object instance = Activator.CreateInstance(t);
```

Компилятор смотрит на этот код и не может доказать, какой именно тип имеется в виду: строка `name` собирается в рантайме. Если тип `MyApp.Models.Order` больше нигде не упоминается явно, граф зависимостей до него не дотянется, и тримминг его выбросит. В рантайме `Type.GetType` вернет `null`, и код упадет.

Решения два. Либо перечислить тип в rd.xml, чтобы он попал в корни. Либо пометить в коде, что он нужен:

```
[DynamicallyAccessedMembers(DynamicallyAccessedMemberTypes.All)]
static void Create(string name) { ... }
```

Атрибут `DynamicallyAccessedMembers` говорит компилятору: "этот метод может использовать рефлексию, не вырезай то, к чему он может обратиться". Компилятор проверит, что вы не передаете сюда произвольную строку из воздуха, и выдаст предупреждение, если не может гарантировать безопасность.

Если рефлексия не поддерживается, ставится `FullyBlockedMetadataBlockingPolicy` (`Program.cs:449`), и любая попытка рефлексии ведет к ошибке, а не к тихому null. Это лучше, чем узнать в продакшене, что тип был вырезан.

Типы, которые нужны только через рефлексию, приходится перечислять в rd.xml. Синтаксис описан в `nativeaot/docs/rd-xml-format.md`.

Interop

Экспорт управляемого метода в нативный мир делается атрибутом `UnmanagedCallersOnly` с именем точки входа. Внутри самой библиотеки тот же механизм через `RuntimeExport`. Импорт нативных функций - `LibraryImport` или `DllImport`, которые ILC превращает в прямые вызовы.

Пример экспорта управляемого кода наружу:

```
using System.Runtime.InteropServices;

public static class NativeApi
{
    [UnmanagedCallersOnly(EntryPoint = "sum_array")]
    public static unsafe double SumArray(double* data, int length)
    {
        double sum = 0;
        for (int i = 0; i < length; i++)
            sum += data[i];
        return sum;
    }
}
```

После `dotnet publish` этот метод можно вызвать из C++:

```
extern "C" double sum_array(double* data, int length);
```

Обратите внимание: метод статический, параметры только blittable-типов, никаких исключений наружу. Все потому, что нативная сторона не умеет разбирать управляемые объекты и исключения. Это не ограничение NativeAOT, а фундаментальная граница между двумя мирами.

Пример импорта нативной функции в управляемый код:

```
using System.Runtime.InteropServices;

public static partial class Native
{
    [LibraryImport("mylib", EntryPoint = "native_compute")]
    internal static partial double Compute(double x, int mode);
}
```

`LibraryImport` - это современный генераторный вариант `DllImport`. На этапе компиляции он создает код маршалинга, который ILC затем оптимизирует. Для простых blittable-типов маршалинга не будет вообще: вызов превратится в прямой переход в нативную функцию. Для `string` или массивов появится код конвертации, и это место, где легко случайно начать аллоцировать.

Из интересного на уровне сборки: статическая линковка ICU и OpenSSL (`nativeaot/docs/compiling.md:68`), чтобы бинарник не зависел от системных библиотек, и кросс-архитектурная компиляция. Важная деталь (`compiling.md:38`):

```
Native AOT toolchain allows targeting ARM64 on an x64 host and vice versa for both Windows
and Linux.
Cross-OS compilation, such as targeting Linux on a Windows host, is not supported.
```

То есть собрать ARM64 на x64 можно, а Linux на Windows нельзя. Это прямое следствие того, что для линковки нужен нативный тулчейн целевой платформы.

Что покрутить в проекте

Минимум для включения:

```
<PropertyGroup>
  <PublishAot>true</PublishAot>
</PropertyGroup>
```

Что происходит при `dotnet publish` с этой настройкой. SDK видит `PublishAot` и вместо обычной публикации с `runtimeconfig.json` и DLL-ками запускает ILC. Тот компилирует всю программу в объектные файлы, а нативный тулчейн линкует их с рантаймом. На выходе в `publish` лежит один исполняемый файл, например `MyApp.exe` на Windows или `MyApp` на Linux, плюс, возможно, файл отладочных символов (`MyApp.pdb` или `MyApp.dbg`). Ни `dotnet` в PATH, ни установленного рантайма для запуска не нужно.

Полезные свойства (`nativeaot/docs/optimizing.md`):

- `<IlcInstructionSet>avx2,bmi2,fma,pclmul,popcnt,aes</IlcInstructionSet>` - разрешить инструкции новее базовых. `native` под текущий CPU.

- `<IlcMaxVectorTBitWidth>256</IlcMaxVectorTBitWidth>` - ограничить размер `Vector<T>`, чтобы бинарник не требовал AVX512 там, где не нужно.
- `<IlcGenerateMapFile>true</IlcGenerateMapFile>` - карта того, что попало в бинарник. Без нее непонятно, почему он такой большой.
- `<IlcGenerateMstatFile>true</IlcGenerateMstatFile>` - статистика по типам и методам.
- `<IlcGenerateDgmlFile>true</IlcGenerateDgmlFile>` - граф зависимостей, открывается в VS.

Отдельно про размер: `IlcGenerateCompleteTypeMetadata` и высокий уровень рефлексии заметно раздувают бинарник. Если рефлексия нужна только для пары типов, дешевле перечислить их точечно.

Как заглянуть внутрь бинарника

Когда бинарник весит 40 МБ, хочется понять, что там. Включаем карту и статистику:

```
<PropertyGroup>
  <IlcGenerateMapFile>true</IlcGenerateMapFile>
  <IlcGenerateMstatFile>true</IlcGenerateMstatFile>
</PropertyGroup>
```

После публикации рядом появятся `MyApp.map` и `MyApp.mstat`. В map-файле видно, какие методы и типы попали в бинарник и сколько каждый весит. В mstat - сводка по категориям. Типичный сценарий: видите в mstat, что половину занимают метаданные рефлексии, и отключаете `IlcGenerateCompleteTypeMetadata`, если рефлексия не нужна.

Граф зависимостей можно открыть в Visual Studio:

```
<IlcGenerateDgmlFile>true</IlcGenerateDgmlFile>
```

Получится `MyApp.dgml`, в котором узлы - типы и методы, а ребра - зависимости. Очень помогает, когда непонятно, почему в бинарник затянулся целый пласт кода: ищешь путь от корня до подозрительного узла и находишь виновника.

Еще можно управлять объемом метаданных. По умолчанию ILC тримит их, оставляя только то, что достижимо. Если рефлексия не нужна совсем, не включайте полные метаданные:

```
<!-- по умолчанию true: метаданные тримятся -->
<IlcTrimMetadata>true</IlcTrimMetadata>
```

Если же выставить `<IlcTrimMetadata>false</IlcTrimMetadata>`, ILC передаст компилятору `--reflectiondata:all` (`Microsoft.NETCore.Native.targets:315`), и в бинарник попадут метаданные для всего, что осталось после тримминга кода. Это резко увеличивает размер, зато рефлексия начинает находить типы. Включайте осознанно и только там, где она реально нужна.

Ловушки

Нет загрузки новых сборок в рантайме. Все, что не попало в бинарник при сборке, недоступно. `Assembly.LoadFrom` в привычном виде не работает.

Это прямое следствие AOT: код уже скомпилирован и лежит в бинарнике, загрузить что-то новое неоткуда. Плагинная архитектура с внешними DLL в привычном виде не собирается. Вместо этого плагины компилируются в тот же бинарник и выбираются по имени типа, либо используется нативный interop.

Reflection.Emit не работает. Кодогенерации в рантайме нет. Если библиотека строит типы на лету, она несовместима с NativeAOT.

`Reflection.Emit` - это API, который позволяет создать новый тип прямо во время выполнения и сгенерировать для него код. Это возможно только когда есть JIT. Типичные жертвы: сериализаторы, генерирующие код на ходу, некоторые ORM и прокси-библиотеки. Ищите source generators или AOT-совместимые альтернативы.

Тримминг ломает рефлексию. Компилятор предупреждает о местах, где рефлексия может не найти тип, но не всегда может доказать проблему. `[DynamicallyAccessedMembers]` и rd.xml - ваши инструменты.

Практическое правило: включайте предупреждения об AOT как ошибки в CI (`<TrimmerSingleWarn>false</TrimmerSingleWarn>`, `TreatWarningsAsErrors`), чтобы проблема всплыла на сборке, а не у пользователя. И покрывайте рефлексивные пути тестами именно в AOT-сборке, потому что обычный `dotnet test` их не поймает.

`Vector<T>` фиксируется на этапе сборки. JIT мог бы подстроиться под конкретный CPU, а ILC решает заранее. Отсюда `IlcMaxVectorTBitWidth` и требование инструкций в `IlcInstructionSet`.

`Vector<T>` - это тип, который использует SIMD-инструкции процессора. Его размер (128, 256, 512 бит) зависит от того, какие инструкции доступны. JIT определяет это при запуске, а ILC обязан выбрать размер при сборке, потому что иначе нельзя сгенерировать код. Компромисс: собирайте под базовый набор для переносимости или под AVX2/AVX512 для скорости, но тогда бинарник не запустится на старом железе.

CPU без нужных инструкций не запустится. Это проверка в `DetectCPUFeatures`, и это осознанный размен: скорость против переносимости.

Если нужна максимальная совместимость, не указывайте `IlcInstructionSet` вообще: тогда ILC целится в минимальный набор целевой платформы. Если нужна скорость и вы контролируете парк машин, указывайте конкретный набор и документируйте требования.

Стабильная скорость, но не всегда лучшая. Нет tiered compilation и динамического PGO, поэтому пиковая производительность горячего цикла может быть ниже, чем у CoreCLR с динамической профилировкой. Зато нет пауз на JIT и предсказуемый старт.

`Tiered compilation` в CoreCLR сначала компилирует метод быстро и неоптимально, а потом, если метод оказался горячим, перекомпилирует его с оптимизациями. `PGO` использует профиль реального выполнения, чтобы лучше расположить ветки и инлайны. ILC ничего этого в рантайме сделать не может. Частично это компенсируется статическим анализом и MBIC-профилями, но не полностью.

EH и stack walking устроены иначе. Рантайм разбирает стек через `StackFrameIterator` и диапазон управляемого кода. Если вы делаете глубокий interop с нативным кодом и полагаетесь на точный stack trace через границы, проверяйте поведение отдельно.

`EH` (exception handling) - обработка исключений. В NativeAOT раскрутка стека завязана на GC info и диапазоны кода, которые компилятор сгенерировал для каждого метода. Через нативные границы точность может теряться: кадры нативного кода не всегда попадают в управляемый stack trace. Для диагностики управляйте данными трассировки: `<StackTraceSupport>true</StackTraceSupport>` (по умолчанию включено) добавляет имена методов, а `<StackTraceLineNumberSupport>true</StackTraceLineNumberSupport>` еще и номера строк, но заметно раздувает бинарник. Проверяйте, что нужные кадры на месте, именно в AOT-сборке.

Что в итоге

NativeAOT - это компилятор ILC, который видит всю программу целиком и генерирует нативный код через тот же RyuJIT, плюс нативный рантайм, который переиспользует GC из CoreCLR. Управляемый код и рантайм сцеплены через таблицу classlib-функций и bookend-секции, которые линкер склеивает в единые диапазоны. Объект - это `ObjHeader` плюс указатель на `MethodTable`, а аллокация - все тот же сдвиг указателя в per-thread контексте.

Цена всего этого - отказ от динамической загрузки и рантайм-кодогенерации, фиксация набора инструкций и обязательный тримминг. Взамен - один бинарник, быстрый старт и отсутствие JIT. Для CLI-утилит, сервисов и нативных библиотек это почти всегда выигрыш. Для приложений, которые живут рефлексией и плагинами, - почти всегда боль.

Безаллокационный TCP-сервер на C#: ноль байт из кучи в горячем пути

Давайте я покажу вам два сервера. Первый пишут в туториалах, он занимает десять строк и на локальном ноутбуке показывает миллион запросов в секунду. Второй выглядит страшнее, но под реальной нагрузкой держит p99 latency ровным, пока первый уже тонет в паузах сборщика мусора. Разница между ними не в сети и не в железе, а в том, сколько объектов они рождают в куче.

На пальцах: почему мусор в сервере это боль

Сначала разберемся, почему вообще аллокации в серверном коде так важны. В .NET выделить объект дешево: это сдвиг указателя, о чем мы говорили в уроке про GC. Дорого стоит не выделение, а уборка.

Объекты живут в **куче**, разбитой на **поколения**. Молодые объекты попадают в gen0. Когда gen0 заполняется, сборщик запускает **сборку**: останавливает все потоки приложения, проходит по живым объектам и переносит их дальше. Эта остановка называется **паузой GC**. Для сервера пауза означает, что запросы в этот момент не обрабатываются.

Теперь представьте сервер, который на каждый запрос создает десяток объектов. Пятьдесят тысяч запросов в секунду - это полмиллиона объектов. Gen0 переполняется постоянно, сборки идут одна за другой, и в хвостах latency (тот самый p99) появляются пики. Средняя latency может остаться неплохой, но один процент самых медленных запросов будет плохим именно из-за пауз.

Различают две метрики. **Пропускная способность** (throughput) - сколько запросов в секунду сервер переваривает. **Задержка** (latency) - сколько времени занимает один запрос. Можно иметь высокую пропускную способность и плохую задержку одновременно. Для интерактивных сервисов задержка важнее, и именно ее портят паузы GC.

Хорошая новость: большая часть этого мусора не нужна. Буфер можно переиспользовать, вместо `Task` использовать callback, а состояние операции хранить в одном объекте на все соединение. Этим и занимается `SocketAsyncEventArgs`.

Словарь терминов

Аллокация - выделение нового объекта в куче.

Gen0 / gen1 / gen2 - поколения объектов в куче. Молодые живут в gen0, пережившие сборку переезжают дальше.

Пауза GC - момент, когда сборщик останавливает потоки приложения, чтобы прибрать мусор.

Throughput (пропускная способность) - сколько работы сервер делает в единицу времени.

Latency (задержка) - сколько времени занимает одна операция. p99 - задержка, которую не превысили 99 процентов запросов.

SAEA (SocketAsyncEventArgs) - переиспользуемая структура-коробка для одной асинхронной операции над сокетом.

IOCP (Input/Output Completion Port) - механизм Windows для асинхронного ввода-вывода. Ядро выполняет операцию и кладет результат в очередь завершений.

Overlapped - нативная структура Windows, описывающая асинхронную операцию. `PreAllocatedOverlapped` - заранее выделенная обертка над ней.

epoll - механизм Unix для отслеживания готовности множества сокетов. Аналог IOCP, но работает по-другому: сообщает, когда сокет готов, а не когда операция завершена.

Edge-triggered (по фронту) - режим epoll, при котором уведомление приходит один раз при изменении состояния. Надо вычитать все доступные данные, иначе уведомление потеряется.

Level-triggered (по уровню) - режим, при котором уведомление повторяется, пока есть данные. Проще, но менее эффективен.

DMA (Direct Memory Access) - способ, которым устройство пишет в память напрямую, без участия процессора. Ядро Windows использует его для сетевого ввода-вывода.

Pinning (пиннинг) - запрет сборщику перемещать объект. Нужен, пока ядро пишет в буфер.

ThreadPool - пул потоков для выполнения задач.

State machine (машина состояний) - объект, в который компилятор превращает асинхронный метод с `await`.

ExecutionContext - контекст выполнения: `AsyncLocal`, `Activity`, данные логирования. Захват контекста может аллоцировать.

AsyncLocal - переменная, значение которой свое для каждого асинхронного потока управления.

Callback - функция, которую вызывают по завершении операции.

Work item - единица работы, которую ThreadPool ставит в очередь.

Backlog - очередь входящих соединений, которые ядро держит до вызова `Accept`.

NoDelay - опция сокета, отключающая алгоритм Нейгла. Полезно для интерактивных протоколов.

ArrayPool - пул переиспользуемых массивов.

POH (Pinned Object Heap) - куча для объектов, закрепленных навсегда. Позволяет пинить без создания GC-хэндла.

CancellationToken - токен для отмены операции.

Сервер, который мусорит

Вот так пишут почти все:

```
var listener = new TcpListener(IPAddress.Any, 5000);
listener.Start();

while (true)
{
    using TcpClient client = await listener.AcceptTcpClientAsync();
    _ = HandleAsync(client);
}

static async Task HandleAsync(TcpClient client)
{
    byte[] buffer = new byte[4096];
    NetworkStream stream = client.GetStream();

    while (true)
    {
        int read = await stream.ReadAsync(buffer);
        if (read == 0) break;
        await stream.WriteAsync(buffer.AsMemory(0, read));
    }
}
```

Что тут происходит с памятью, когда приходит тысяча соединений. На каждое соединение создается `TcpClient`, `NetworkStream` и массив `byte[4096]`. На каждую операцию чтения-записи - `Task`, продолжения, регистрации в пуле, плюс state machine асинхронного метода, который компилятор размещает в куче при первом реальном ожидании. Пятьдесят тысяч запросов в секунду превращаются в миллионы маленьких объектов, которые переживают одну сборку gen0 и промоутятся в gen1.

Посчитаем по объектам, чтобы стало наглядно. Возьмем одно соединение и один цикл чтения-записи:

Что	Когда создается	Размер примерно
`TcpClient`	на соединение	40 байт
`NetworkStream`	на соединение	40 байт
`byte[4096]`	на соединение	4 КБ плюс заголовок
state machine `HandleAsync`	при первом `await`	100+ байт
`Task` от `ReadAsync`	на каждое чтение	70+ байт
`Task` от `WriteAsync`	на каждую запись	70+ байт
продолжения и регистрации	на каждое ожидание	десятки байт

На соединение с 100 обменами это тысячи объектов. Умножьте на тысячи одновременных соединений - и получите постоянный поток мусора, который GC не успевает разгребать.

.NET аллоцирует быстро, бампом указателя. Проблема не в скорости аллокации, а в том, что заполненный gen0 рано или поздно останавливает потоки. Для сервера это прямой удар по хвостам latency. И самое обидное - большая часть этого мусора не нужна вообще. Массив можно переиспользовать, `Task` не создавать, а состояние операции хранить в одном объекте на все соединение.

SocketAsyncEventArgs: коробка, которую создают один раз

`SocketAsyncEventArgs` (дальше SAEA) - это переиспользуемая коробка под одну асинхронную операцию над сокетом. Не под соединение, а именно под операцию: положили буфер, запустили чтение, получили результат, очистили, запустили следующее. Вся суть в том, что коробка живет долго, а операций через нее проходит бесконечно много.

API устроен необычно для .NET. Метод не возвращает `Task`, он возвращает `bool`:

```
SocketAsyncEventArgs args = new SocketAsyncEventArgs(unsafeSuppressExecutionContextFlow:
true);

args.SetBuffer(buffer, 0, buffer.Length);
args.Completed += OnCompleted;

bool pending = socket.ReceiveAsync(args);
if (!pending)
{
    // операция завершилась прямо здесь, до возврата
    OnCompleted(socket, args);
}
```

`true` значит операция повисла и callback придет позже. `false` значит она завершилась синхронно, и callback надо вызвать руками. Забыть про второй случай - классическая ошибка, из-за которой сервер молча перестает обслуживать соединение. Компилятор вам не подскажет, потому что `bool` он не проверяет.

Обратите внимание на `unsafeSuppressExecutionContextFlow: true`. Это не случайный флаг, за ним стоит конкретная строчка в рантайме.

Что внутри коробки

Открываем `src/libraries/System.Net.Sockets/src/System/Net/Sockets/SocketAsyncEventArgs.cs`. Ключевые поля:

```
// SocketAsyncEventArgs.cs:21
private Memory<byte> _buffer;
// SocketAsyncEventArgs.cs:72
private readonly bool _flowExecutionContext;
// SocketAsyncEventArgs.cs:87
```

```
private OperationState _operating; // Configuring = -1, Free = 0, InProgress = 1, Disposed =
2
```

А теперь `StartOperationCommon`, который вызывается перед каждой нативной операцией:

```
// SocketAsyncEventArgs.cs:565
if (_flowExecutionContext ||
    (SocketsTelemetry.Log.IsEnabled() && (operation == SocketAsyncOperation.Connect ||
operation == SocketAsyncOperation.Accept)))
{
    _context = ExecutionContext.Capture(); // SocketAsyncEventArgs.cs:568
}
```

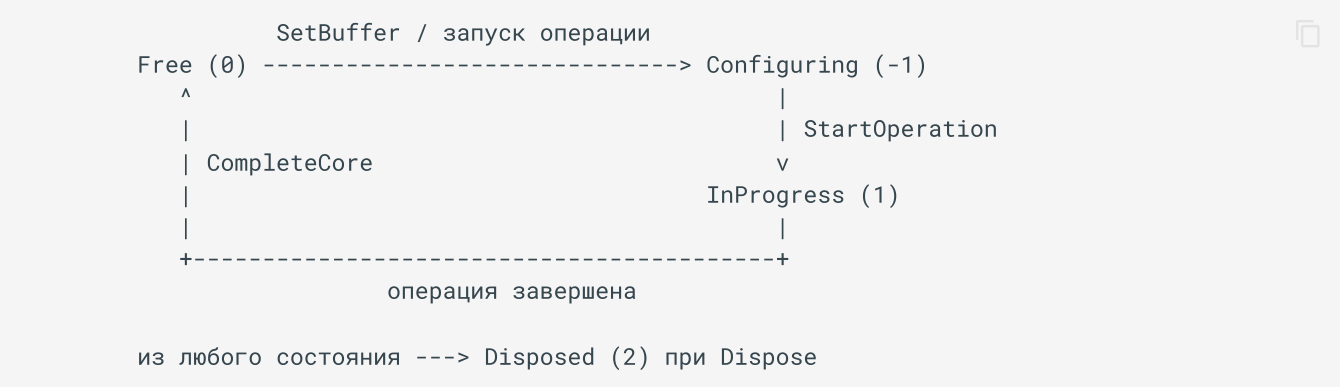
`ExecutionContext.Capture()` - это захват всего контекста выполнения на каждой операции. Если у вас есть `AsyncLocal`, `Activity` из OpenTelemetry или логирование через контекст, объект не пустой, и захват стоит аллокации. Дефолтный конструктор `SocketAsyncEventArgs()` выставляет `_flowExecutionContext = true` (`SocketAsyncEventArgs.cs:91`), а конструктор с `unsafeSuppressExecutionContextFlow: true` (`SocketAsyncEventArgs.cs:100`) его выключает. Рантайм сам этим пользуется: в `Socket.Tasks.cs` кэшированный `AwaitableSocketAsyncEventArgs` создается через `base(unsafeSuppressExecutionContextFlow: true)`, потому что `Task` / `ValueTask` выше уже разобрались с контекстом.

Теперь `_operating`. Это маленький state machine, который защищает коробку от повторного использования. Вот `SetBuffer` дергает `StartConfiguring` (`SocketAsyncEventArgs.cs:525`):

```
private void StartConfiguring()
{
    OperationState status = Interlocked.CompareExchange(ref _operating,
OperationState.Configuring, OperationState.Free);
    if (status != OperationState.Free)
    {
        ThrowForNonFreeStatus(status);
    }
}
```

Если операция еще в полете, вызов кинет `InvalidOperationException` с текстом `net_socketopinprogress`. Один SAEA - одна операция за раз. Это не баг, это защита от гонки, и об этом легко забыть.

Вот этот state machine на схеме:



Правила простые: пока не пришел `Completed`, состояние `InProgress`, и трогать SAEA нельзя. `SetBuffer` до старта операции переводит в `Configuring`. После завершения рантайм возвращает `Free`, и коробку можно готовить к следующей операции. Если попытаться настроить SAEA, пока он занят, получите исключение, а не тихую порчу данных.

Windows: IOCP, overlapped и долгий пин

Чтобы понять этот раздел, разберемся с двумя словами. **Overlapped** - это нативная структура Windows, описывающая асинхронную операцию ввода-вывода. Вы заполняете ее буфером и параметрами, отдаете ядру, и ядро само выполняет операцию, пока ваш поток занимается другими делами. **IOCP** (Input/Output Completion Port) - это очередь, в которую ядро кладет уведомление о завершении. Поток берет из очереди готовые операции и обрабатывает их. Смысл в том, что поток не ждет, а разбирает уже завершенное.

Модель Windows принципиально асинхронная на уровне ядра: вы запускаете `WSARecv`, и он возвращается сразу, а данные придут позже. Именно поэтому буфер нельзя двигать все время операции, и его приходится пинить.

На Windows файл `SocketAsyncEventArgs.Windows.cs`. Каждая коробка в конструкторе создает `PreAllocatedOverlapped`:

```
// SocketAsyncEventArgs.Windows.cs, InitializeInternals
_preAllocatedOverlapped = PreAllocatedOverlapped.UnsafeCreate(s_completionPortCallback,
_strongThisRef, null);
```

Это нативная структура `OVERLAPPED` плюс callback, выделенная один раз. В горячем пути `WSARecv` / `WSASend` берут ее через `AllocateNativeOverlapped` и отдают ядру. На операцию - ноль нативных аллокаций.

Но с буфером все интереснее. Смотрите `ProcessIOCPResult`:

```
if (!bufferToPin.Equals(default))
{
    _singleBufferHandle = bufferToPin.Pin();
}
```

На Windows ядро пишет в буфер асинхронно, через DMA, пока операция висит в IOCP. Значит буфер нельзя двигать сборщиком. Рантайм его пинит на все время операции, `MemoryHandle` лежит в поле SAEA, а освобождается в `CompleteCore`. Если вы берете буфер из `ArrayPool<byte>.Shared`, на каждую операцию создается GC-хэндл и пин. Это не куча, но это работа и давление на GC: закрепленные объекты усложняют компактировку.

Хитрость: `GC.AllocateUninitializedArray<byte>(size, pinned: true)` создает массив, закрепленный навсегда, в отдельной куче. `Memory<T>.Pin()` над таким массивом почти бесплатен. Если число соединений фиксировано, это лучший вариант.

Еще одна деталь из того же файла - `_asyncCompletionOwnership`, `ulong`, через который поток-запускатель и IOCP-callback договариваются, кто владеет завершением. Когда операция завершается синхронно, но хэндл не помечен `SkipCompletionPortOnSuccess`, рантайм не вызывает callback, а ждет

пакет из IOCP. Если callback успел раньше, чем поток закончил настройку, владение переходит запускающему потоку, и операция обрабатывается как синхронная. Это экономит переключение контекста на быстрых accept'ах.

Unix: epoll, один поток и короткий пин

epoll - это механизм Unix, который решает похожую задачу, но иначе. Вы регистрируете в нем набор сокетов и говорите "разбуди меня, когда на каком-нибудь из них появятся данные". Ядро держит этот набор и сообщает о готовности. Ключевое отличие от IOCP: epoll сообщает о **готовности** ("на этом сокете можно читать"), а не о **завершении операции**. Само чтение вы делаете потом, синхронно, но только когда ядро гарантировало, что данные есть.

Отсюда и короткий пин на Unix: `recv` выполняется быстро, данные копируются сразу, и буфер закреплен только на время этого вызова. Никакого длительного DMA, как на Windows.

На Unix все иначе. Здесь нет IOCP. Есть `SocketAsyncEngine` (`SocketAsyncEngine.Unix.cs`), который крутит `epoll_wait` в цикле. Загляните в нативный шим `src/native/libs/System.Native/pal_networking.c`:

```
// pal_networking.c:3205
evt.events = GetEPollEvents(newEvents) | (unsigned int)EPOLLET;
evt.data.ptr = (void*)data;
int err = epoll_ctl(port, op, socket, &evt);
```

`EPOLLET` - edge-triggered режим. После одного уведомления надо вычерпать сокет до `EWOULDBLOCK`, иначе уведомление потеряется. Поэтому в `SocketAsyncContext.Unix.cs` есть `OperationQueue` с `_sequenceNumber` и состояниями `Ready/Waiting/Processing/Stopped`. Логика описана прямо в комментарии начиная со строки 749.

Движок создает поток `.NET Sockets` (`SocketAsyncEngine.Unix.cs:200`). Сколько их? Смотрите `GetEngineCount` (`SocketAsyncEngine.Unix.cs:49`): на x64 рантайм считает один движок на 30 ядер, на ARM - на 12. На обычной 16-ядерной машине это ровно один поток, который читает epoll и раскидывает события. Переопределяется переменной окружения `DOTNET_SYSTEM_NET_SOCKETS_THREAD_COUNT`.

События не обрабатываются в потоке epoll. Рантайм пакует их и кидает в ThreadPool:

```
// SocketAsyncEngine.Unix.cs:301-308
for (int i = 0; i < count; i += MaxTreeSize)
{
    int treeSize = Math.Min(MaxTreeSize, count - i);
    SocketIOEvent root = asyncEvents[i];
    LinkChildren(root, new ReadOnlySpan<SocketIOEvent>(asyncEvents, i + 1, treeSize - 1));
    ThreadPool.UnsafeQueueUserWorkItem(root, preferLocal: false);
}
```

Зачем тут бинарное дерево? Чтобы на пачку из 1024 событий поставить в ThreadPool не 1024 work item, а около 32. Корень дерева при выполнении распаковывает детей в локальные очереди. `SocketIOEvent` берутся из пула `_eventPool` (`ConcurrentQueue<SocketIOEvent>`), так что на каждое событие аллокации нет.

А буфер на Unix пинится ровно на время системного вызова. Смотрите `SocketPal.Unix.cs`:

```
// SocketPal.Unix.cs, TryCompleteReceive
fixed (byte* b = &MemoryMarshal.GetReference(buffer))
{
    errno = Interop.Sys.Recv(...);
}
```

Ядро копирует данные синхронно во время `recv`, потому что операция выполняется только когда epoll сказал, что сокет готов. Никакого длительного пина через асинхронный DMA тут нет. Для аллокаций и для GC это огромное преимущество Unix-пути.

Сводка различий:

Что	Windows (IOCP)	Unix (epoll)
Кто ждет готовность	ядро, IOCP	поток `.NET Sockets`, `epoll_wait`
Буфер закреплен	на всю операцию	только внутри `recv` / `send`
Нативная структура на операцию	`PreAllocatedOverlapped`, одна на SAEA	нет
Диспетчеризация завершения	IOCP-порт	ThreadPool, батчами по дереву
Захват `ExecutionContext`	на каждой операции	на каждой операции

Собираем сервер

Идея простая: на каждое соединение один объект `Connection`, внутри два SAEA (чтение и запись) и один буфер из пула. Ни одного `await` в горячем пути.

```
using System;
using System.Buffers;
using System.Net;
using System.Net.Sockets;

sealed class EchoServer
{
    private const int BufferSize = 16 * 1024;

    private readonly Socket _listener;
    private readonly SocketAsyncEventArgs _acceptArgs =
        new SocketAsyncEventArgs(unsafeSuppressExecutionContextFlow: true);

    public EchoServer(IPEndPoint endPoint)
    {
```

```
        _listener = new Socket(endPoint.AddressFamily, SocketType.Stream, ProtocolType.Tcp);
        _listener.SetSocketOption(SocketOptionLevel.Socket, SocketOptionName.ReuseAddress,
true);
        _listener.Bind(endPoint);
        _listener.Listen(512);
    }

    public void Start()
    {
        _acceptArgs.Completed += OnAccept;
        StartAccept();
    }

    private void StartAccept()
    {
        _acceptArgs.AcceptSocket = null;
        if (!_listener.AcceptAsync(_acceptArgs))
        {
            OnAccept(_listener, _acceptArgs);
        }
    }

    private void OnAccept(object? sender, SocketAsyncEventArgs e)
    {
        if (e.SocketError == SocketError.Success)
        {
            Socket socket = e.AcceptSocket!;
            socket.NoDelay = true;
            _ = new Connection(socket, BufferSize);
        }

        StartAccept();
    }
}
```

Теперь само соединение. `SetBuffer` вызывается либо до старта, либо после завершения, когда SAEA снова `Free`.

```
sealed class Connection
{
    private readonly Socket _socket;
    private readonly byte[] _buffer;
    private readonly SocketAsyncEventArgs _recv =
        new SocketAsyncEventArgs(unsafeSuppressExecutionContextFlow: true);
    private readonly SocketAsyncEventArgs _send =
        new SocketAsyncEventArgs(unsafeSuppressExecutionContextFlow: true);

    private int _sendOffset;
    private int _sendCount;

    public Connection(Socket socket, int bufferSize)
    {
        _socket = socket;
        _buffer = ArrayPool<byte>.Shared.Rent(bufferSize);

        _recv.UserToken = this;
        _recv.SetBuffer(_buffer, 0, _buffer.Length);
        _recv.Completed += OnReceive;

        _send.UserToken = this;
```

```
        _send.Completed += OnSend;

        StartReceive();
    }

    private void StartReceive()
    {
        if (!_socket.ReceiveAsync(_recv))
        {
            OnReceive(_socket, _recv);
        }
    }

    private void OnReceive(object? sender, SocketAsyncEventArgs e)
    {
        if (e.SocketError != SocketError.Success || e.BytesTransferred == 0)
        {
            Close();
            return;
        }

        _sendOffset = 0;
        _sendCount = e.BytesTransferred;
        StartSend();
    }

    private void StartSend()
    {
        _send.SetBuffer(_buffer, _sendOffset, _sendCount);
        if (!_socket.SendAsync(_send))
        {
            OnSend(_socket, _send);
        }
    }

    private void OnSend(object? sender, SocketAsyncEventArgs e)
    {
        if (e.SocketError != SocketError.Success)
        {
            Close();
            return;
        }

        _sendOffset += e.BytesTransferred;
        _sendCount -= e.BytesTransferred;

        if (_sendCount > 0)
        {
            StartSend();
            return;
        }

        StartReceive();
    }

    private void Close()
    {
        ArrayPool<byte>.Shared.Return(_buffer);
        _socket.Dispose();
    }
}
```

Считаем аллокации на соединение:

- `Socket` для принятого соединения создает сам рантайм в `GetOrCreateAcceptSocket` (`Socket.cs:2862`). Это единственная аллокация, которую обойти нельзя: у соединения должно быть управляемое состояние.
- `byte[]` берется из `ArrayPool<byte>.Shared`. Первый рент аллоцирует массив, дальше он переиспользуется между соединениями.
- Два `SocketAsyncEventArgs` и объект `Connection` - по одному на соединение.
- На каждую операцию чтения или записи: ноль управляемых аллокаций.

Про `AcceptSocket`. В `Socket.cs:2862` рантайм делает `GetOrCreateAcceptSocket(e.AcceptSocket, ...)`. Если не обнулить `e.AcceptSocket` перед следующим `AcceptAsync`, он попробует переиспользовать уже отданный вам сокет. Поэтому в `StartAccept` стоит `_acceptArgs.AcceptSocket = null`. На Unix, кстати, рантайм все равно создает временный `Socket` под капотом и копирует состояние в ваш (`FinishOperationAccept` в `SocketAsyncEventArgs.Unix.cs`), так что сэкономить там не выйдет.

Ловушки

CancellationToken в горячем пути аллоцирует. В `SocketAsyncContext.Unix.cs:892` и в Windows-варианте регистрация делается только если `cancellationToken.CanBeCanceled`. `CancellationToken.None` не может быть отменен, регистрации нет. Как только вы передаете отменяемый токен на каждую операцию, рантайм делает `UnsafeRegister`, а это структура регистрации и потенциальная аллокация. Не тащите токен туда, где он не нужен.

`await socket.ReceiveAsync(memory)` не равен нулевым аллокациям. Рантайм держит кэшированный `AwaitableSocketAsyncEventArgs` на сокет (`Socket.Tasks.cs:19-21`), и `ValueTask` - структура. Но если операция реально повиснет, компилятор разместит state machine вашего асинхронного метода в куче. Если метод завершается синхронно, аллокации нет. Для гарантированного нуля в горячем пути - callback'и на SAEA, а не `await`.

Один кэшированный SAEA на сокет - это один опережающий receive. Перегрузки в `Socket.Tasks.cs` берут SAEA через `Interlocked.Exchange(ref _singleBufferReceiveEventArgs, null)`. Если запустить второй `ReceiveAsync(Memory)` на том же сокете, пока первый висит, кэш пуст, и рантайм создаст второй SAEA. Для пайплайна с несколькими одновременными операциями на сокет используйте явный пул SAEA, а не `ValueTask`-перегрузки.

Буфер нельзя возвращать в пул раньше завершения. На Windows ядро пишет в него асинхронно. Вернете в `ArrayPool` в момент старта операции - получите порчу данных, которые уже отданы другому соединению. Возвращайте только в колбэке завершения.

`SetBuffer` во время операции кинет исключение. Один SAEA - одна операция за раз. Не переиспользуйте SAEA, пока не пришел `Completed`.

`SocketError.IOPending` не приходит наружу как исключение. `AcceptAsync` / `ReceiveAsync` / `SendAsync` возвращают `bool`. Забыть про синхронный случай и не вызвать callback - классическая ошибка, сервер просто перестанет обслуживать соединение.

Частые ошибки на примерах

Соберу главные грабли в виде "было / стало".

Забыли синхронный случай.

```
// было: если ReceiveAsync вернул false, callback не придет, сервер встал
_socket.ReceiveAsync(_recv);

// стало: проверяем bool и вызываем callback руками
if (!_socket.ReceiveAsync(_recv))
{
    OnReceive(_socket, _recv);
}
```

Это ошибка номер один, потому что компилятор не подсказывает: `bool` можно проигнорировать без предупреждения.

Вернули буфер в пул до завершения операции.

```
// было: ядро еще пишет в буфер, а он уже отдан другому соединению
_socket.SendAsync(_send);
ArrayPool<byte>.Shared.Return(_buffer); // порча данных

// стало: возвращаем только в колбэке завершения
private void OnSend(object? sender, SocketAsyncEventArgs e)
{
    // ...
    if (_sendCount == 0)
    {
        ArrayPool<byte>.Shared.Return(_buffer);
        _socket.Dispose();
    }
}
```

Настройка SAEA во время операции.

```
// было: InvalidOperationException, коробка занята
_socket.ReceiveAsync(_recv);
_recv.SetBuffer(other, 0, other.Length); // падение

// стало: настраиваем только когда коробка Free
if (!_socket.ReceiveAsync(_recv))
    OnReceive(_socket, _recv);
// менять буфер можно только после Completed
```

Переиспользование AcceptSocket без сброса.

```
// было: рантайм попробует переиспользовать уже отданный сокет
private void StartAccept()
{
    _listener.AcceptAsync(_acceptArgs); // AcceptSocket не сброшен
}

// стало: обнуляем перед следующим accept
private void StartAccept()
```

```
{
    _acceptArgs.AcceptSocket = null;
    if (!_listener.AcceptAsync(_acceptArgs))
        OnAccept(_listener, _acceptArgs);
}
```

Отменяемый токен в горячем пути.

```
// было: на каждую операцию регистрация и аллокация
await socket.ReceiveAsync(memory, cancellationToken); // token CanBeCanceled

// стало: либо CancellationToken.None, либо вообще callback API
if (!socket.ReceiveAsync(_recv)) // без токена
    OnReceive(socket, _recv);
```

Если отмена нужна, держите один токен на соединение и не создавайте новый на каждую операцию.

`await` там, где нужен гарантированный ноль.

```
// было: при реальном ожидании создается state machine в куче
while (true)
{
    int n = await socket.ReceiveAsync(memory);
    if (n == 0) break;
    await socket.SendAsync(memory.Slice(0, n));
}

// стало: callback-цикл на SAEA, ноль управляемых аллокаций на операцию
private void OnReceive(object? sender, SocketAsyncEventArgs e)
{
    // ... обработали, запустили следующую операцию ...
    if (!_socket.ReceiveAsync(_recv))
        OnReceive(_socket, _recv);
}
```

`await`-версия вполне нормальна для обычного кода: `ValueTask` и кэшированный SAEA сильно снижают аллокации. Но если вы целитесь ровно в ноль на горячем пути, callback надежнее.

Как проверить, что аллокаций нет

Не верьте на слово, замеряйте.

Простейший способ - счетчик аллокаций текущего потока:

```
long before = GC.GetAllocatedBytesForCurrentThread();
// ... прогон нагрузки ...
long after = GC.GetAllocatedBytesForCurrentThread();
Console.WriteLine($"allocated: {after - before} bytes");
```

Живые метрики:

```
dotnet-counters monitor --counters System.Runtime --process-id <PID>
```

Смотрите `alloc-rate` и `gen-0-gc-count`. На безаллокационном сервере `alloc-rate` почти замирает после прогрева, а `gen-0-gc-count` перестает расти.

Для точного профиля с точками аллокаций - `dotnet-trace` или PerfView. Для микробенчмарков - BenchmarkDotNet с `[MemoryDiagnoser]`.

Полезные переменные окружения, которые меняют поведение сетевого движка:

- `DOTNET_SYSTEM_NET_SOCKETS_INLINE_COMPLETIONS=1` - выполнять продолжения прямо в потоке epoll, без ThreadPool. Меньше переключений контекста и latency, но если колбэк блокирует, встанет весь движок.
- `DOTNET_SYSTEM_NET_SOCKETS_THREAD_COUNT=<N>` - число потоков-движков. По умолчанию на x64 один движок на 30 ядер.

Что в сухом остатке

`SocketAsyncEventArgs` - это коробка состояния, которую создают один раз и гоняют по кругу. `unsafeSuppressExecutionContextFlow: true` убирает `ExecutionContext.Capture()` из каждой операции. Буферы берутся из `ArrayPool<byte>`, а для фиксированного набора - через `GC.AllocateUninitializedArray<byte>(size, pinned: true)`, чтобы не пинить на Windows каждую операцию. Callback на `Completed` вместо `await` в горячем пути дает гарантированный ноль управляемых аллокаций на операцию. И помните про разницу: на Unix рантайм пинит буфер только внутри `recv` / `send`, на Windows - на всю операцию.

System.IO.Pipelines: разбор протокола поверх TCP без копирований

В прошлом уроке мы довели TCP-сервер до нуля аллокаций на операцию ввода-вывода. Но если вы попробуете прикрутить к нему реальный протокол, вы почти сразу уткнетесь в проблему, которую сокет не решает и решить не может. TCP - это поток байтов, а не поток сообщений. То, что вы отправили одним `Send`, придет не обязательно одним `Receive`. Два сообщения могут слипнуться в один буфер, а одно сообщение может быть разорвано между двумя чтениями. И вот тут начинается копирование, склейки через `MemoryStream` и аллокации, от которых мы только что убежали.

`System.IO.Pipelines` - это слой, который берет на себя всю возню с накоплением и нарезкой байтов, не копируя их. Kestrel работает именно так: `Socket` снизу, `Pipe` сверху.

На пальцах: поток против сообщений

Чтобы понять, зачем все это, нужно усвоить одну вещь: TCP не сохраняет границы сообщений.

Когда вы вызываете `Send` с массивом из 100 байт, это не значит, что получатель получит ровно 100 байт одним `Receive`. TCP - это поток байтов, как вода из крана. Вы налили стакан, но тот, кто на другом конце, может зачерпнуть его двумя ковшами, или наоборот зачерпнуть два ваших стакана одним ковшом. Гарантируется только порядок байтов, а не границы.

А приложению нужны сообщения. Ему нужен не поток байтов, а "вот это одно сообщение целиком". Значит, кто-то должен: - копить байты, пока не наберется целое сообщение; - уметь ждать, если пришла только половина; - отдать следующее сообщение, если пришло полтора.

Писать это руками на массивах - боль. Вот как это выглядит без `Pipe`:

```
byte[] buffer = new byte[4096];
int offset = 0;

while (true)
{
    int read = socket.Receive(buffer, offset, buffer.Length - offset, SocketFlags.None);
    offset += read;

    // а если сообщение больше буфера? доращивать?
    // а если в буфере два сообщения? сдвигать остаток?
    // а если пришла половина заголовка? ждать?
    // ...
}
```

Каждый вопрос - это ветвление, копирование и потенциальная аллокация. `Pipe` берет эту возню на себя. Вы говорите ему "вот байты", он их копит и отдает вам тогда, когда вы попросите, в виде `ReadOnlySequence<byte>`. И делает это без копирования.

Словарь терминов

TCP - протокол передачи данных, который дает надежный упорядоченный поток байтов, но не сохраняет границы сообщений.

Поток против сообщений - фундаментальное различие: TCP работает с байтами, приложение хочет сообщения. Мост между ними и есть задача `Pipe`.

Сегмент - кусок памяти фиксированного размера, из которого собирается буфер `Pipe`. Сегменты связаны в односвязный список.

Pipe - буфер-очередь между писателем и читателем. Сам по себе не делает ввод-вывод, только хранит байты.

PipeWriter / PipeReader - две "руки" пайпа: одна складывает байты, другая забирает.

ReadOnlySequence - структура, представляющая логическую последовательность, которая физически может лежать в нескольких несвязанных буферах. Именно так `Pipe` отдает данные.

SequenceReader - удобный курсор поверх `ReadOnlySequence<T>`. Прячет разрывы между сегментами.

Backpressure (обратное давление) - механизм, который не дает быстрому производителю завалить медленного потребителя и съесть всю память.

PauseWriterThreshold / ResumeWriterThreshold - пороги в байтах, при которых писатель тормозит и снова разблокируется.

GetMemory / GetSpan - просьба к пайпу выдать участок для записи.

Advance - сообщить пайпу, сколько байтов из выданного участка реально занято.

FlushAsync - закоммитить записанное и разбудить читателя.

AdvanceTo(consumed, examined) - сообщить пайпу, до какого места данные потреблены и до какого осмотрены. От этого зависит возврат сегментов в пул и backpressure.

IValueTaskSource - интерфейс для ручной реализации `ValueTask` без создания `Task` на каждый вызов.

ValueTask - облегченная замена `Task` для случая, когда операция часто завершается синхронно.

MemoryPool / ArrayPool - пулы памяти, откуда берутся и куда возвращаются буферы.

NoDelay - опция сокета, отключающая алгоритм Нейгла. Полезно для протоколов, где важна задержка, а не экономия пакетов.

async-метод (корутина) - метод, который компилятор превращает в state machine и который может приостанавливаться на `await`.

Проблема, которую сокет не решает

Представьте, что клиент шлет сообщения в формате "4 байта длины, потом сама длина". Вы читаете 4096 байт и получаете что-то такое:

- первый `Receive` вернул половину заголовка, потому что пакет разорвался;
- второй `Receive` вернул хвост первого сообщения и начало второго;
- третье сообщение вообще уместилось целиком плюс начало четвертого.

Если писать это руками на массивах, вы быстро скатываетесь в буфер, который надо сдвигать, доращивать, а на границах копировать. Каждый такой сдвиг - это `Array.Copy` и, скорее всего, новая аллокация.

Хочется другого: пусть байты копятся сами, а я просто спрошу "дай мне следующее полное сообщение". Ровно это и делает `Pipe` вместе с `ReadOnlySequence<byte>`.

Один буфер, две руки

`Pipe` - это очередь сегментов между двумя сторонами. С одной стороны писатель складывает байты, с другой читатель их забирает. Сегменты - это маленькие куски памяти, связанные в список. Писатель двигает хвост, читатель двигает голову, а сегменты, которые читатель целиком проглотил, возвращаются в пул и идут под следующую запись.

Ключевое поле в `Pipe.cs` - `_unconsumedBytes` (`Pipe.cs:60`), количество закоммиченных, но еще не прочитанных байтов. Вокруг него крутится вся механика. Голова и хвост описаны так:

```
// Pipe.cs:76-89
private BufferSegment? _readHead;      // начало прочитанного
private int _readHeadIndex;
private BufferSegment? _readTail;      // конец доступного читателю
private int _readTailIndex;
private BufferSegment? _writingHead;   // конец записанного писателем
private Memory<byte> _writingHeadMemory;
```

Сами сегменты - это `BufferSegment` (`BufferSegment.cs:10`):

```
internal sealed class BufferSegment : ReadOnlySequenceSegment<byte>
{
    private IMemoryOwner<byte>? _memoryOwner;
    private byte[]? _array;
    private BufferSegment? _next;
    private int _end;
}
```

Один сегмент держит либо `IMemoryOwner<byte>` (из `MemoryPool`), либо `byte[]` (из `ArrayPool`). Смотрите `ResetMemory` (`BufferSegment.cs:73`), который возвращает буфер владельцу:

```
IArrayPoolBuffer? ...
if (memoryOwner != null)
{
    _memoryOwner = null;
    memoryOwner.Dispose();
}
```

```
else
{
    Debug.Assert(_array != null);
    ArrayPool<byte>.Shared.Return(_array);
    _array = null;
}
```

То есть память под данные берется из тех же пулов, что мы разбирали, и возвращается туда же.

Вот как выглядит пайп с данными схематично. Читатель уже проглотил начало, писатель дописал в конец:

```
_readHead                  _readTail              _writingHead
   |                          |                      |
   v                          v                      v
 [seg A][seg B][seg C][seg D] . . . . . . . . . . [seg E]
   ^     ^      ^      ^                           ^       ^
 прочит.  читаемые данные (от _readHead до _readTail)   записанное
                                                (_readTail до _writingHead)
```

- `_readHead` - самый старый сегмент, который читатель еще не освободил.
- `_readTail` - докуда читателю разрешено читать (все закоммичено через `FlushAsync`).
- `_writingHead` - докуда писатель уже записал, но, возможно, еще не закоммитил.

Когда читатель вызывает `AdvanceTo` и говорит, что потребил данные из сегмента `A`, пайп может вернуть `A` в пул и переиспользовать под новую запись. Это и есть источник нулевых аллокаций: список сегментов циклически переиспользуется.

Важная деталь: сегменты никогда не сдвигаются в памяти. Они связаны указателями `_next`, и "продвижение" читателя - это просто переход к следующему узлу списка. Никаких `Array.Copy`.

Где здесь аллокации, а где их нет

Первое, что бросается в глаза: сам `Pipe` - это объект. Плюс он создает `DefaultPipeReader` и `DefaultPipeWriter` (`Pipe.cs:33-34`). Это разовые аллокации на соединение. Дальше начинается интересное.

Сегменты переиспользуются. Смотрите `GetOrCreateSegment` и `ReturnSegment` (`Pipe.cs:350-372`):

```
private BufferSegment GetOrCreateSegment()
{
    if (_bufferSegmentPool.TryDequeue(out BufferSegment? segment))
    {
        return segment;
    }

    return new BufferSegment();
}

private void ReturnSegment(BufferSegment segment)
{
    // ...
    if (_bufferSegmentPool.Count < _options.MaxSegmentPoolSize)
```

```
        {
            _bufferSegmentPool.Enqueue(segment);
        }
    }
```

Пул сегментов - это `SingleProducerSingleConsumerQueue<BufferSegment>` (`Pipe.cs:31`), то есть один читатель и один писатель почти никогда не блокируют друг друга. Ограничение `MaxSegmentPoolSize` по умолчанию 256 (`PipeOptions.cs:50`), а комментарий рядом объясняет арифметику: один сегмент около 96 байт, значит потолок пула примерно 24 КБ на соединение. Буферы данных живут отдельно в `MemoryPool` / `ArrayPool` и возвращаются туда еще до того, как сегмент попадет в пул.

Второй момент: все структуры, которые видит пользователь, не аллоцируют.

- `ReadResult` (`ReadResult.cs:9`) - `readonly struct` из `ReadOnlySequence<byte>` и флагов.
- `ReadOnlySequence<T>` (`ReadOnlySequence.cs:15`) - структура из четырех полей:

```
// ReadOnlySequence.cs:18-21
private readonly object? _startObject;
private readonly object? _endObject;
private readonly int _startInteger;
private readonly int _endInteger;
```

Это разобранные на части две позиции. Копирование такой структуры не трогает кучу.

- `PipeAwaitable` (`PipeAwaitable.cs:12`) - тоже структура, и она реализует асинхронное ожидание через `IValueTaskSource`. Посмотрите на `DefaultPipeReader` (`Pipe.DefaultPipeReader.cs:13`): он `IValueTaskSource<ReadResult>`, а `ReadAsync` возвращает `ValueTask`, а не `Task`:

```
// Pipe.cs:878
result = new ValueTask<ReadResult>(_reader, token: 0);
```

`_reader` тут - это один заранее созданный объект на весь pipe. То есть в нормальном случае `await pipe.Reader.ReadAsync()` не создает ни `Task`, ни замыканий. Тот же трюк для записи в `FlushAsync` (`Pipe.cs:504`): `new ValueTask<FlushResult>(_writer, token: 0)`.

Итого в установившемся режиме: ноль аллокаций на чтение и запись, пока данных хватает в уже выделенных сегментах. Аллокация происходит только когда нужен новый сегмент, и это амортизируется пулом.

Разберем трюк с `ValueTask` подробнее, потому что это ключ к нулевым аллокациям на асинхронности. Обычный `Task` - это класс, объект в куче. Каждый `await SomeTask()` уже имеет готовый `Task`, но методы, которые его создают, аллоцируют. Если метод вызывается миллион раз в секунду, миллион объектов `Task` - это много мусора.

`ValueTask` - это структура. Если операция завершилась синхронно, `ValueTask` вообще не создает объект, он просто хранит результат. Если асинхронно, он хранит ссылку на `IValueTaskSource` - объект, который умеет возобновлять ожидание.

`IValueTaskSource` позволяет реализовать асинхронную операцию вручную и переиспользовать один объект. Посмотрите на объявление (`Pipe.DefaultPipeReader.cs:13`):

```
internal sealed class DefaultPipeReader : PipeReader, IValueTaskSource<ReadResult>
{
    // ...
}
```

А `ReadAsync` возвращает:

```
result = new ValueTask<ReadResult>(_reader, token: 0);
```

Здесь `_reader` - это и есть источник, а `token` - номер "попытки". Дело в том, что `ValueTask` нельзя ожидать дважды. Один и тот же `_reader` переиспользуется для всех вызовов `ReadAsync`, и `token` увеличивается, чтобы рантайм мог понять, к какой именно попытке относится возобновление. Если вы попытаетесь ожидать один и тот же `ValueTask` два раза, получите ошибку.

Вот как это работает по шагам:

1. Вы зовете `ReadAsync()`. Пайп возвращает `ValueTask`, который ссылается на `_reader`.
2. Если данные уже есть, `ValueTask` завершается синхронно, объект не создается.
3. Если данных нет, `await` подписывается на `_reader` и паркует продолжение.
4. Когда приходят данные, пайп будит `_reader`, и тот вызывает ваше продолжение.

Ни на одном шаге не создается `Task`. Один объект `_reader` живет столько же, сколько пайп.

Практическое следствие: не сохраняйте `ValueTask` в поле и не ждите его повторно. Если нужно сохранить результат, сохраните его, а не сам `ValueTask`.

Backpressure: не даем быстрому продюсеру съесть память

Что будет, если клиент шлет данные быстрее, чем вы их обрабатываете? Без ограничений очередь сегментов будет расти, пока не кончится память. Поэтому у `Pipe` есть два порога (`PipeOptions.cs:53-56`):

- `PauseWriterThreshold`, по умолчанию 64 КБ. Когда в пайпе накопилось столько, `FlushAsync` начинает блокироваться.
- `ResumeWriterThreshold`, по умолчанию 32 КБ. Как только читатель опустил объем ниже, писатель снова разблокируется.

Логика в `CommitUnsynchronized` (`Pipe.cs:406-411`):

```
else if (PauseWriterThreshold > 0 &&
    oldLength < PauseWriterThreshold &&
    _unconsumedBytes >= PauseWriterThreshold &&
    !_readerCompletion.IsCompleted)
{
    _writerAwaitable.SetUncompleted();
}
```

А читатель отпускает писателя в `AdvanceReader`, когда `_unconsumedBytes` падает ниже порога возобновления (`Pipe.cs:607-613`). Здесь важна деталь: писатель тормозит не `Thread.Sleep`, а парковкой awaitable. Поток не простаивает, он уходит на другую работу.

Вот как выглядят пороги на шкале заполнения пайпа:

```
0 байт          32 КБ                 64 КБ
|---------------|---------------------|---------------->
   писатель        resume                pause
   свободно        порог                 порог

  < 32 КБ:        писатель не тормозит
  32 - 64 КБ:     писатель уже разбужен, но не запаркован
  >= 64 КБ:       FlushAsync паркует писателя, пока читатель не опустит ниже 32 КБ
```

Зачем два порога, а не один. Если бы порог был один, писатель дергался бы на каждой границе: чуть превысил - запарковался, чуть отпустил - разбудился. Это называется thrashing. Гистерезис между 64 и 32 КБ дает буфер, в котором писатель работает спокойно, а читатель успевает разгрести накопленное.

Есть еще защита от дедлока: если читатель ждет минимум N байт (`ReadAtLeastAsync`), а писатель как раз запаркован на пороге, читатель принудительно будит писателя (`Pipe.cs:835-838`).

ReadOnlySequence и SequenceReader: парсим по кускам

`ReadOnlySequence<byte>` - это ответ на вопрос "как передать данные, которые лежат в нескольких несвязанных буферах, как одну логическую ленту". Он умеет `Length`, `IsSingleSegment` (`ReadOnlySequence.cs:41`), `FirstSpan` (`ReadOnlySequence.cs:55`) и `Slice`.

Но вручную по нему ходить неудобно. Для этого есть `SequenceReader<T>` (`SequenceReader.cs:42`), структура поверх последовательности. У нее есть `Remaining`, `Position`, `UnreadSpan`, `TryRead`, `Advance`, `Rewind`. Вся прелесть в том, что пока нужные байты лежат в одном сегменте, `SequenceReader` отдает их как обычный `ReadOnlySpan` без копирования. Копирование включается только когда данные реально разорваны между сегментами, и то не всегда.

Вот основные операции `SequenceReader` на примерах.

Создание и проверка, сколько осталось:

```
var reader = new SequenceReader<byte>(buffer);

if (reader.Remaining < 4)
    return false; // даже заголовок не влез
```

Чтение числа в big-endian:

```
if (!reader.TryReadBigEndian(out int length))
    return false; // не хватило байтов, позиция не сдвинулась
```

Важно: если `TryReadBigEndian` вернул `false`, курсор `reader.Position` не двигается. Можно спокойно подождать еще данных и попробовать снова.

Просмотр без потребления:

```
// посмотреть следующие 2 байта, не сдвигая курсор
ReadOnlySpan<byte> peek = reader.UnreadSpan;
if (peek.Length >= 2 && peek[0] == 0xAB && peek[1] == 0xCD)
{
    reader.Advance(2); // теперь двигаем
}
```

Чтение до разделителя (например, строки до `\n`):

```
if (reader.TryReadTo(out ReadOnlySpan<byte> line, (byte)'\n'))
{
    // line - кусок до перевода строки, без копии, если он в одном сегменте
    Process(line);
}
```

Если строка разорвана между сегментами, `TryReadTo` с `ReadOnlySpan` вернет `false`, потому что span не может пересекать границу сегмента. Для этого случая есть перегрузка с `ReadOnlySequence<byte>`, которая умеет отдавать составной кусок.

Ключевое различие, которое стоит усвоить: `reader.Position` - это курсор чтения внутри `SequenceReader`, а `reader.Consumed` - сколько байтов уже пройдено. Когда вы передаете данные наружу, вы обычно используете `reader.Position` как новую границу потребленного.

Собираем протокол поверх нашего TCP-сервера

Теперь соединим это с прошлым уроком. Берем принятый `Socket`, заводим `Pipe`, одна корутина качает байты из сокета в пайп, вторая их разбирает. Формат сообщения: 4 байта длины в big-endian, потом тело.

```
using System;
using System.Buffers;
using System.IO.Pipelines;
using System.Net.Sockets;
using System.Threading.Tasks;

sealed class PipeConnection
{
    private readonly Socket _socket;
    private readonly Pipe _pipe;

    public PipeConnection(Socket socket)
    {
        _socket = socket;
        _socket.NoDelay = true;
        _pipe = new Pipe(new PipeOptions(
            pauseWriterThreshold: 64 * 1024,
            resumeWriterThreshold: 32 * 1024,
            minimumSegmentSize: 4096,
            useSynchronizationContext: false));
    }

    public Task RunAsync() => Task.WhenAll(FillPipeAsync(), ParsePipeAsync());
```

```
    private async Task FillPipeAsync()
    {
        PipeWriter writer = _pipe.Writer;
        try
        {
            while (true)
            {
                Memory<byte> memory = writer.GetMemory(4096);
                int bytesRead = await _socket.ReceiveAsync(memory, SocketFlags.None)
                    .ConfigureAwait(false);
                if (bytesRead == 0)
                    break;

                writer.Advance(bytesRead);

                FlushResult result = await writer.FlushAsync().ConfigureAwait(false);
                if (result.IsCompleted)
                    break;
            }
        }
        finally
        {
            await writer.CompleteAsync().ConfigureAwait(false);
        }
    }

    private async Task ParsePipeAsync()
    {
        PipeReader reader = _pipe.Reader;
        try
        {
            while (true)
            {
                ReadResult result = await reader.ReadAsync().ConfigureAwait(false);
                ReadOnlySequence<byte> buffer = result.Buffer;

                while (TryReadMessage(ref buffer, out ReadOnlySequence<byte> message))
                {
                    // важно: досылаем до AdvanceTo, иначе сегменты вернутся в пул
                    await EchoAsync(message).ConfigureAwait(false);
                }

                reader.AdvanceTo(buffer.Start, buffer.End);

                if (result.IsCompleted)
                    break;
            }
        }
        finally
        {
            await reader.CompleteAsync().ConfigureAwait(false);
        }
    }

    private static bool TryReadMessage(ref ReadOnlySequence<byte> buffer, out
ReadOnlySequence<byte> message)
    {
        var reader = new SequenceReader<byte>(buffer);

        if (reader.Remaining < 4)
```

```
        {
            message = default;
            return false;
        }

        if (!reader.TryReadBigEndian(out int length))
        {
            message = default;
            return false;
        }

        if (reader.Remaining < length)
        {
            // тело еще не пришло целиком, оставляем буфер как есть
            message = default;
            return false;
        }

        message = reader.Sequence.Slice(reader.Position, length);
        buffer = reader.Sequence.Slice(reader.Position + length);
        return true;
    }

    private async ValueTask EchoAsync(ReadOnlySequence<byte> message)
    {
        foreach (ReadOnlyMemory<byte> segment in message)
        {
            await _socket.SendAsync(segment, SocketFlags.None).ConfigureAwait(false);
        }
    }
}
```

Разберем, что тут к чему.

`GetMemory(4096)` выдает участок текущего сегмента, куда можно читать прямо из сокета. `Advance(bytesRead)` говорит пайпу, сколько из этого реально занято. `FlushAsync` коммитит данные и будит читателя. Это классический цикл заполнения.

На стороне парсинга `ReadAsync` возвращает `ReadResult` с `ReadOnlySequence<byte>`. Мы крутим `TryReadMessage`, пока он не скажет "мало данных". Затем `AdvanceTo(buffer.Start, buffer.End)` сообщает две вещи: что мы реально потребили (`buffer.Start`, который сдвинулся после каждого полного сообщения) и до какого места мы все осмотрели (`buffer.End`). Именно второй аргумент управляет backpressure.

`TryReadMessage` не копирует. Он смотрит заголовок через `TryReadBigEndian`, проверяет, хватает ли тела, и если да, отдает `ReadOnlySequence` - срез исходной последовательности. Тело остается в тех же сегментах пайпа.

Отдельно про `EchoAsync`. Отправка идет по сегментам. В типичном случае сообщение лежит в одном сегменте, и это один `SendAsync` без копии. Если сообщение разорвано, будет несколько отправок, но байты все равно не копируются.

И самое важное: `EchoAsync` вызывается до `AdvanceTo`. Если сначала вернуть сегменты в пул, а потом отправить по ним данные, пайп успеет переиспользовать память под следующую запись, и вы отправите мусор. Порядок здесь не стилистический, а обязательный.

Ловушки

Забыли `AdvanceTo` - читатель встал. Пока вы не сказали пайпу, сколько потребили, он считает данные необработанными, и backpressure в итоге запаркует писателя. В дебаге это выглядит как зависший сервер без ошибок.

Нельзя держать `ReadResult.Buffer` после `AdvanceTo`. После возврата сегментов в пул эта память уже чужая. Все, что нужно сохранить, копируйте до `AdvanceTo`.

`GetMemory` / `GetSpan` без `Advance` - ошибка. `GetMemory` не резервирует данные, он лишь выдает доступный участок. Пока не вызван `Advance`, пайп не знает, что вы туда что-то положили.

Не предполагайте один сегмент. `ReadOnlySequence` может состоять из нескольких кусков. Код, который делает `buffer.FirstSpan` и считает, что это все сообщение, рано или поздно сломается. Для этого и нужен `SequenceReader`.

`Pipe` не делает ввод-вывод. Это буфер, а не транспорт. Байты из сокета в него кладете вы, и обратно в сокет тоже пишете вы. Отсюда и две корутины: одна качает в пайп, вторая разбирает.

`useSynchronizationContext: false` для сервера. Иначе продолжения `await` попытаются вернуться в захваченный контекст, которого на сервере нет, и вы просто получите лишнюю работу. Kestrel выставляет это в `false`.

Частые ошибки на примерах

Соберу грабли в виде "было / стало".

Забыли `AdvanceTo`.

```
// было: читатель встал, писатель запаркован навсегда
ReadResult result = await reader.ReadAsync();
ReadOnlySequence<byte> buffer = result.Buffer;
Process(buffer);
// AdvanceTo забыли

// стало: всегда сообщаем, что прочитали
ReadResult result2 = await reader.ReadAsync();
ReadOnlySequence<byte> buffer2 = result2.Buffer;
Process(buffer2);
reader.AdvanceTo(buffer2.End);
```

Использование буфера после `AdvanceTo`.

```
// было: данные уже вернулись в пул и перезаписаны
ReadResult result = await reader.ReadAsync();
byte[] copy = result.Buffer.ToArray(); // это ок, копия сделана
reader.AdvanceTo(result.Buffer.End);
// а вот так нельзя: result.Buffer уже чужая память
```

Правило: все, что нужно сохранить за пределами обработки, копируйте до `AdvanceTo`.

`GetMemory` без `Advance`.

```
// было: пайп не знает, что данные записаны
Memory<byte> memory = writer.GetMemory(1024);
memory.Span[0] = 42;
// Advance забыли, данные потеряны

// стало
Memory<byte> memory2 = writer.GetMemory(1024);
memory2.Span[0] = 42;
writer.Advance(1);
await writer.FlushAsync();
```

Предположение об одном сегменте.

```
// было: сломается, если сообщение разорвано
ReadOnlySpan<byte> first = buffer.FirstSpan;
Process(first); // обработает только первый кусок

// стало: идем через SequenceReader или по сегментам
var reader = new SequenceReader<byte>(buffer);
while (reader.TryReadBigEndian(out int value))
{
    Process(value);
}
```

Повторное ожидание `ValueTask`.

```
// было: ошибка, ValueTask нельзя await дважды
ValueTask<ReadResult> task = reader.ReadAsync();
ReadResult r1 = await task;
ReadResult r2 = await task; // InvalidOperationException

// стало: ждем один раз и сохраняем результат
ValueTask<ReadResult> task2 = reader.ReadAsync();
ReadResult result = await task2;
Use(result);
```

Отправка после возврата сегментов.

```
// было: сегменты вернулись в пул, память перезаписана
reader.AdvanceTo(buffer.End);
await SendAsync(buffer); // отправляем мусор

// стало: сначала используем данные, потом сообщаем о потреблении
await SendAsync(buffer);
reader.AdvanceTo(buffer.End);
```

Что в итоге

`System.IO.Pipelines` решает ровно одну задачу: превратить поток байтов в удобные логические куски, не копируя их и не аллоцируя на каждом шаге. Сегменты берутся из пула, буферы из `MemoryPool` / `ArrayPool`, асинхронность идет через `IValueTaskSource` без `Task`, а `ReadOnlySequence` и `SequenceReader` дают доступ к данным, даже если они разорваны между сегментами.

Вместе с прошлым уроком получается полный низкоуровневый стек: `SocketAsyncEventArgs` или `Pipe` для транспорта, `Pipe` и `SequenceReader` для протокола. И на всем пути, от `recv` до разобранного сообщения, ни одной лишней аллокации.

Zig 0.17.0-dev с нуля

Восемь уроков по актуальному мастеру Zig. Начинаем с hello world, заканчиваем структурами — и всё это на новом API, а не на примерах из туториалов двухлетней давности.

Каждый урок строится одинаково: **зачем это нужно** → **базовый синтаксис** → **смотрим исходники std** → **где можно споткнуться** → **практика**.

Как проходить

1. Читаешь урок.
2. Решаешь задачу руками, не подглядывая в решение.
3. Сверяешься с разбором и советами.

Программа

#	Урок	Задача	Решение
1	Первый hello world	Задача	Ответ
2	Переменные и их типы	Задача	Ответ
3	Строки	Задача	Ответ
4	Ошибки и их обработка	Задача	Ответ
5	Условия if/else и optionals	Задача	Ответ
6	Циклы while и for	Задача	Ответ
7	Функции	Задача	Ответ
8	Структуры	Задача	Ответ

Версия

Материалы сверены со свежим master Zig 0.17.0-dev. Если пример из старого туториала не собирается — так и задумано: здесь только актуальный API.

1. Первый hello world

Зачем это нужно и в чем прикол

В C ты пишешь `int main(int argc, char** argv)` и молча ждешь, что ОС подкинет тебе аргументы и переменные окружения. В Python точка входа - это вообще не функция, а просто "все, что сверху скрипта". Никто не спрашивает, откуда берутся `sys.argv` и `os.environ`.

Zig делает иначе: функция `main` получает на вход готовый, собранный для тебя объект со всем окружением процесса. Ты явно видишь, что у твоей программы есть аргументы командной строки, переменные окружения, аллокатор и подсистема ввода-вывода. Ничего не появляется "из воздуха".

И да, сразу важная штука: эта версия Zig (0.17.0-dev) недавно переехала на новую подсистему I/O. Если ты гуглил `std.debug.print` и `std.io.getStdOut()` - забудь, это прошлое. Дальше разберем, что стало вместо этого.

Базовый синтаксис

Вот минимальная программа, которая реально собирается на текущем мастере:

```
const std = @import("std");

pub fn main(init: std.process.Init) !void {
    var buffer: [4096]u8 = undefined;
    var stdout = std.Io.File.stdout().writerStreaming(init.io, &buffer);

    try stdout.interface.print("Hello, {s}!\n", .{"world"});
    try stdout.flush();
}
```

Разбираем по строчкам:

- `const std = @import("std");` - `@import` это встроенная функция, она подключает стандартную библиотеку на этапе компиляции. Никакого линкера-рантайма, как в C с `#include` и заголовочниками.
- `pub fn main(init: std.process.Init) !void` - точка входа. Заметь тип параметра: `std.process.Init`, готовый пакет окружения. А `!void` означает "либо ничего не возвращаю, либо ошибку". Подробнее про `!` разберем в отдельном уроке, пока считай это "может вернуть ошибку".
- `var buffer: [4096]u8 = undefined;` - стековый буфер для вывода. Никакого неявного выделения памяти из кучи, мы сами даем писателю куда складывать байты.
- `std.Io.File.stdout()` - получаем файл стандартного вывода.
- `.writerStreaming(init.io, &buffer)` - оборачиваем его в буферизованный writer, который пишет через нашу подсистему I/O (`init.io`) и использует наш буфер.

- `try stdout.interface.print("Hello, {s}!\n", .{"world"});` - собственно печать. `{s}` - плейсхолдер строки, `.{...}` - кортеж аргументов.
- `try stdout.flush();` - выталкиваем буфер в реальный файл. Без этого вывод может не долететь до терминала.

Есть и совсем короткий вариант для ленивых:

```
const std = @import("std");

pub fn main(init: std.process.Init) !void {
    try std.Io.File.stdout().writeStreamingAll(init.io, "Hello, world!\n");
}
```

`writeStreamingAll` внутри делает "создай writer, запиши байты, сбрось буфер" одной строкой. Для hello world - самое то.

Смотрим в сурсы: что там под капотом

Начнем с того, кто вообще вызывает `main`. Это `lib/std/start.zig`. Когда компилятор собирает исполняемый файл, он прилинковывает стартовый код, который экспортирует символ `_start`. Дальше цепочка такая: `_start` -> `callMain` (start.zig:786).

Вот что делает `callMain`, по сути:

```
inline fn callMain(args: std.process.Args.Vector, environ: std.process.Environ.Block) u8 {
    // ... определение gpa: в debug/safe режиме это SafeAllocator с проверкой утечек
    const gpa = if (use_safe_allocator)
        safe_allocator.allocator()
    else if (builtin.link_libc)
        std.heap.c_allocator
    // ...

    var arena_allocator = std.heap.ArenaAllocator.init(arena_backing_allocator);
    var threaded: std.Io.Threaded = .init(gpa, .{ ... });
    // ... парсинг окружения, преопенсы

    return wrapMain(root.main(.{
        .minimal = .{ .args = ..., .environ = ... },
        .arena = &arena_allocator,
        .gpa = gpa,
        .io = threaded.io(),
        .environ_map = &environ_map,
        .preopens = preopens,
    }));
}
```

То есть еще до твоего `main` рантайм поднял: аллокатор общего назначения (в debug-сборке - с проверкой утечек), арену, потоковый I/O, карту переменных окружения. Все это пакуется в `std.process.Init` и передается тебе. Определение структуры лежит в `lib/std/process.zig:31`:

```
pub const Init = struct {
    minimal: Minimal,       // args + environ
    arena: *std.heap.ArenaAllocator,
```

```
    gpa: Allocator,
    io: Io,
    environ_map: *Environ.Map,
    preopens: Preopens,
};
```

Дальше `wrapMain` (start.zig:840) смотрит на то, что вернула твоя функция. Разрешенные типы возврата строго такие: `void`, `!void`, `noreturn`, `u8`, `!u8`. Если `main` вернул ошибку, `wrapMain` ловит ее, пишет в лог через `std.log.err` и возвращает код выхода 1:

```
const unwrapped_result = result catch |err| {
    std.log.err("{t}", .{err});
    // ...
    return 1;
};
```

Заметь `{t}` - это формат-спецификатор для имени ошибки (он же подходит для энумов).

Теперь про сам вывод. `std.Io.File.stdout()` в `lib/std/Io/File.zig:91` просто оборачивает нативный дескриптор в структуру `File`:

```
pub fn stdout() File {
    return switch (native_os) {
        .windows => .{
            .handle = std.os.windows.peb().ProcessParameters.hStdOutput,
            .flags = .{ .nonblocking = false },
        },
        else => .{
            .handle = std.posix.STDOUT_FILENO,
            .flags = .{ .nonblocking = false },
        },
    };
}
```

На Windows handle берется прямо из PEB процесса, на POSIX - это файловый дескриптор `1`. Никакой магии, просто число.

А `print` живет не там, где ты мог бы ожидать. `writerStreaming` возвращает `File.Writer` (это `lib/std/Io/File/Writer.zig`), у которого внутри есть поле `interface: Io.Writer`. Сам `print`, `writeAll`, `flush` - методы этого `Io.Writer` из `lib/std/Io/Writer.zig:697`. Поэтому и пишем `stdout.interface.print(...)`.

Сам `print` - это comptime-магия: строка формата разбирается на этапе компиляции, аргументы проверяются по типам. Есть жесткое ограничение - максимум 32 аргумента на один вызов, иначе `@compileError("32 arguments max are supported per format call")`.

Где можно споткнуться

1. Забыть `flush`. Writer буферизует в твой стековый `buffer`. Пока буфер не заполнился, байты никуда не ушли. Завершил программу без `flush` - "Hello, world!" просто не появился на экране. Классика.

2. Написать `stdout.print(...)` вместо `stdout.interface.print(...)`. `File.Writer` - это отдельная структура, у которой `print` нет. Компилятор честно скажет `error: no field or member function named 'print' in 'Io.File.Writer'`. Спотыкание на этом - норма, просто помни про `.interface`.

3. Устаревший API из гайдов. `std.debug.print("hi", .{})` еще существует, но теперь пишет в stderr (и с 64-байтным буфером), а `std.io.getStdOut()` в этой версии уже нет вообще. Если дергаешь код из старого туториала - получишь ошибку "no member". Ориентируйся на `std.Io.File.stdout()`.

4. Неправильный тип возврата у `main`. Вернул `i32` или `error` без `!` - компилятор воткнет `expected return type of main to be 'void', '!void', 'noreturn', 'u8', or '!u8'`. Хочешь свой код выхода - возвращай `u8`.

Пробуем руками

📝 **Задача:** открыть

✅ **Ответ:** открыть

2. Переменные и их типы

Зачем это нужно и в чем прикол

В C ты можешь написать `int x = 300; unsigned char y = x;` и компилятор молча обрежет значение до 44. Никакой ошибки, никакого предупреждения. В Python переменная вообще не имеет типа, а `x = "5" + 5` взрывается только в рантайме, когда ты до этой строки доехал.

Zig ставит вопрос ребром: каждая переменная имеет конкретный тип, и этот тип обязан быть совместимым с присваиваемым значением. Хочешь сузить `u32` до `u8` - будь добр написать это явно. Компилятор не даст потерять данные молча, а переполнение целого в debug-сборке ловит прямо в момент выполнения с паникой и стек-трейсом.

Вторая фишка - вывод типов. Ты не обязан писать тип везде, Zig сам выведет его из значения. Но в отличие от Python, тип выводится один раз, на этапе компиляции, и дальше уже не меняется.

Базовый синтаксис

Два способа объявить переменную:

```
const std = @import("std");

pub fn main(init: std.process.Init) !void {
    var buffer: [4096]u8 = undefined;
    var out = std.Io.File.stdout().writerStreaming(init.io, &buffer);

    // var - можно менять, const - нельзя
    var age: u8 = 30;
    const name: []const u8 = "Heller";

    age += 1; // ок
    // name = "Other"; // ошибка: cannot assign to constant

    // тип можно не писать, Zig выведет сам
    const answer = 42; // это comptime_int
    var score = 0;     // тоже comptime_int, но с var

    try out.interface.print("{s} is {d}, answer {d}, score {d}\n", .{ name, age, answer,
score });
    try out.flush();
}
```

Что важно понимать:

- `var` - изменяемая переменная. `const` - неизменяемая. Правило простое: по умолчанию пиши `const`, а `var` - только когда реально будешь менять значение. Zig настолько это любит, что объявление `var`, которую ты ни разу не поменял, это вообще ошибка компиляции (`local variable is never mutated`, подсказка: "consider using 'const'").

- `age: u8` - явная аннотация типа через двоеточие. Слева имя, справа тип.
- `const answer = 42;` - тип не написан, но он есть: `comptime_int`. Об этом ниже.

Теперь про сами типы чисел. Zig не прячет их за `int` и `float` как в C, а называет прямо и честно:

```
const a: u8 = 255;    // беззнаковое, 8 бит, 0..255
const b: i8 = -128;   // знаковое, 8 бит, -128..127
const c: u32 = 4_000_000_000;
const d: i64 = -9_223_372_036_854_775_808;
const e: usize = 100; // размер указателя: 8 байт на x86_64
const f: f32 = 3.14;  // float 32 бита
const g: f64 = 2.71828;
const h: bool = true;
```

Названия говорят сами за себя: `u` - unsigned, `i` - signed, число - количество бит. Стандартный набор: `u8` / `i8` через `u128` / `i128`, плюс `usize` / `isize` (размером с указатель) и `f16` / `f32` / `f64` / `f80` / `f128` для дробей. `bool` хранит `true` / `false` и занимает 1 байт.

А теперь прикол, которого нет ни в C, ни в Python: Zig позволяет объявить целое любой битности:

```
const small: u3 = 5;   // 3 бита, 0..7
const tiny: i2 = -1;   // 2 бита, -2..1
```

Зачем? Когда ты хранишь, скажем, день недели или уровень приоритета, `u8` - это расточительно, а `u3` или `u4` - ровно столько, сколько нужно. Компилятор сам подберет хранение в битовых полях.

Смотрим в сурсы: что там под капотом

Как Zig хранит информацию о типах? Через `@typeInfo(T)`, который возвращает структуру-объединение `std.lang.Type`. Определение лежит в `lib/std/lang.zig:641`:

```
pub const Type = union(enum) {
    type,
    void,
    bool,
    noreturn,
    int: Int,
    float: Float,
    pointer: Pointer,
    array: Array,
    @"struct": Struct,
    comptime_float,
    comptime_int,
    undefined,
    null,
    // ... optional, error_union, enum, union, fn и так далее
};
```

То есть тип - это не "строка с именем", а конкретное значение-тег. Для целых чисел тег `int` несет внутри пару: знаковость и число бит. Это `lang.zig:670`:

```
pub const Int = struct {
    signedness: Signedness,
    bits: u16,
};

pub const Float = struct {
    bits: u16,
};
```

Смотрите, `u8` - это буквально `.{ .int = .{ .signedness = .unsigned, .bits = 8 } }`. Никакой магии, просто данные.

Отсюда же растет `std.math.maxInt` и `std.math.minInt` (`lib/std/math.zig:1437`). Это не таблица с зашитыми константами, а честный расчет из `bits` и `signedness`:

```
pub fn maxInt(comptime T: type) comptime_int {
    const info = @typeInfo(T).int;
    return (1 << (info.bits - @intFromBool(info.signedness == .signed))) - 1;
}
```

Для `u8` это `(1 << 8) - 1 = 255`, для `i8` - `(1 << 7) - 1 = 127`. Все вычисляется на этапе компиляции, потому что параметр помечен `comptime`.

Теперь самое интересное - типы-литералы. Когда ты пишешь `const x = 42`, компилятор дает этому значению тип `comptime_int`. Это произвольная точность: там поместится любое целое число, хоть со ста знаками. Аналогично `3.14` - это `comptime_float`. Вот как `std.math.IntFittingRange` (`lib/std/math.zig:800`) пользуется этим, чтобы подобрать минимальный подходящий тип:

```
/// Returns the smallest integer type that can hold both from and to.
pub fn IntFittingRange(comptime from: comptime_int, comptime to: comptime_int) type {
    // ...
    return @Int(signedness, bits);
}
```

`IntFittingRange(0, 4095)` вернет `u12`, а `IntFittingRange(-1, 1)` - `i2`. Запомни главное: `comptime_int` и `comptime_float` существуют только на этапе компиляции. Как только значение попадает в рантайм, оно обязано иметь конкретный машинный тип - `u8`, `f32` или что-то такое.

Где можно споткнуться

1. Переполнение ловится в рантайме. Возьми `u8 = 255` и прибавь единицу. В debug-сборке получишь панику с трейсом:

```
thread 46476 panic: integer overflow
    x += 1;
```

Это не UB, это осознанная проверка. В release-сборке (режимы `ReleaseFast` / `ReleaseSmall`) проверка отключается и переполнение молча заворачивается по модулю, как в C. Имей в виду.

2. Сужение типов не происходит само. Попытка `const y: u8 = x;` где `x: u32`, дает ошибку компиляции:

```
error: expected type 'u8', found 'u32'
note: unsigned 8-bit int cannot represent all possible unsigned 32-bit values
```

И это не про "подзаебавшее предупреждение" - это ошибка, код не соберется. Сужение нужно писать явно:

```
const y: u8 = @intCast(x);  // проверяет, что влезает; паника, если нет
const z: u8 = @truncate(x); // тупо обрезает старшие биты, без проверок
```

Расширение (u8 -> u16) наоборот безопасно и происходит само. А `@as(T, value)` - это просто "приведи значение к типу T", базовая операция явной типизации.

3. `comptime_int` - это не `u8`. `const x = 42;` имеет тип `comptime_int`, а не `u8`. Пока ты не отдал его в рантайм, компилятор ведет себя по-особому: он знает значение и может отказаться собирать код на этапе компиляции. Например, `@intCast` из константы `1000` в `u8` упадет не в рантайме, а прямо при компиляции: `error: type 'u8' cannot represent integer value '1000'`. Потому что значение известно заранее.

4. Читать `undefined` - это UB. `var u: u8 = undefined;` оставляет память неинициализированной. Прочитать ее - неопределенное поведение. Чтобы такое легче было ловить, debug-сборка заливает `undefined`-память байтом `0xaa`, поэтому в отладке ты увидишь значение вроде `aa` или `aaaaaaaa`. На это не закладывайся: в release там будет мусор.

5. Shadowing запрещен. Объявить переменную с тем же именем во вложенном блоке не выйдет:

```
error: local constant 'n' shadows local constant from outer scope
```

Zig сознательно не разрешает затенять имена - меньше путаницы, когда читаешь чужой код. Разные scope - можно, но не вложенный поверх существующего.

Пробуем руками

📝 **Задача:** открыть

✅ **Ответ:** открыть

3. Строки

Зачем это нужно и в чем прикол

В C строка - это `char*`, который кончается нулевым байтом `\0`. Чтобы узнать длину, `strlen` бежит по памяти до этого нуля. Отсюда вся классика: забыл терминатор - читаешь чужую память, сравниваешь строки через `==` - сравниваешь указатели, `"hello"` в одном месте и `"hello"` в другом - это два разных адреса.

В Zig никакой магии нет вообще. Строка - это просто срез байтов `[]const u8`, а его длина лежит прямо рядом с указателем внутри структуры среза. Никакого скрытого нулевого байта, никакой неявной аллокации, никакого копирования при передаче. Две "одинаковые" строки - это два набора байтов одинаковой длины.

Что это дает на практике: ты всегда знаешь длину за O(1), никто не сканирует память до нуля, и любая операция над строкой честно скажет, сколько байтов она трогает. А если тебе нужна C-совместимость, ты можешь попросить нулевой терминатор явно - об этом ниже.

Базовый синтаксис

Строковый литерал в Zig - это не `char*` и не объект, а указатель на массив с терминатором:

```
const std = @import("std");

pub fn main(init: std.process.Init) !void {
    var buffer: [4096]u8 = undefined;
    var out = std.Io.File.stdout().writerStreaming(init.io, &buffer);

    const lit = "hello";
    try out.interface.print("literal type: {s}\n", .{@typeName(@TypeOf(lit))});
    // напечатает: literal type: *const [5:0]u8

    try out.flush();
}
```

Разбираем `*const [5:0]u8`:

- `[5:0]u8` - массив из 5 элементов типа `u8`, а `:0` - сентинел. Это значит, что сразу после пяти байтов лежит еще один байт со значением `0`.
- `*const` - указатель на этот массив.

То есть `"hello"` в памяти - это 5 байт `h e l l o` и нулевой байт в конце, итого 6 байт, но длина строки - 5. Сентинел нужен именно для совместимости с C-функциями, которым нужен `\0`.

Для работы литерал обычно приводят к обычному срезу:

```
const name: []const u8 = "Heller";  // неявное приведение *const [6:0]u8 -> []const u8
try out.interface.print("name len: {d}\n", .{name.len});  // 6
```

Приведение здесь бесплатное: компилятор просто "забывает" про сентинел и отдает тебе указатель плюс длину. А вот чтобы получить строку с гарантированным нулевым байтом на конце, есть тип `[:0]u8` (срез с сентинелом) - он пригодится, когда будешь звать сишные функции.

Дальше - операции:

```
const a: []const u8 = "hello";
const b: []const u8 = "world";

// сравнение - ТОЛЬКО через std.mem.eql, == для срезов запрещен
const same = std.mem.eql(u8, a, a);

// поиск и проверки
try out.interface.print("{}\n", .{std.mem.startsWith(u8, "hello world", "hello")}); // true
try out.interface.print("{}\n", .{std.mem.endsWith(u8, "hello world", "world")});   // true
try out.interface.print("{?d}\n", .{std.mem.indexOf(u8, "hello world", "world")});  // 6

// срезы по байтам
const first_two: []const u8 = a[0..2];  // "he"

// итерация по байтам
for (a) |byte| {
    try out.interface.print("{c} ", .{byte});
}
```

Обрати внимание на последний кусок: `for` по строке бежит по байтам, а не по символам. Про это - в разделе про подводные камни.

Форматирование строк в этой версии Zig делается через `std.mem.print` (печать в готовый буфер) или через аллокатор `alloc.print`:

```
var buf: [64]u8 = undefined;
const s = try std.mem.print(&buf, "name={s} age={d}", .{ "Heller", 42 });
try out.interface.print("built: {s}\n", .{s});
```

Спецификаторы: `{s}` - строка, `{d}` - число в десятичной, `{x}` - в hex, `{c}` - символ по коду, `{any}` - что угодно (для отладки).

Смотрим в сурсы: что там под капотом

Начнем со сравнения. Вот `std.mem.eql` из `lib/std/mem.zig:753`:

```
/// Returns true if and only if the slices have the same length and all elements
/// compare true using equality operator.
pub fn eql(comptime T: type, a: []const T, b: []const T) bool {
    // ...
    if (a.len != b.len) return false;
    if (a.len == 0) return true;
    if (@typeInfo(T) != .float and a.ptr == b.ptr) return true;
```

```
    for (a, b) |a_elem, b_elem| {
        if (a_elem != b_elem) return false;
    }
    return true;
}
```

Никакой магии: сначала быстрая проверка длин, потом короткое замыкание "указатели совпали - значит это одна и та же строка", потом честный цикл по байтам. Для длинных строк выше по файлу есть еще векторная версия `eqlBytes`, которая сравнивает по 16-32 байта за раз через SIMD.

Поиск подстроки. В этой версии `indexOf` переименовали в `find`, а `indexOf` оставили как совместимый алиас. Смотри `lib/std/mem.zig:1436`:

```
/// Deprecated in favor of `find`.
pub const indexOf = find;

/// Search for needle in haystack and return the index of the first occurrence.
/// Uses Boyer-Moore-Horspool algorithm on large inputs; linear search on small inputs.
/// Returns null if needle is not found.
pub fn find(comptime T: type, haystack: []const T, needle: []const T) ?usize {
    return findPos(T, haystack, 0, needle);
}
```

Строка - это данные, и алгоритм поиска тут честный: на больших строках Бойер-Мур-Хорспул, на маленьких - линейный перебор. Возвращает `?usize` - опциональный индекс, `null` если не нашли.

Форматирование. Раньше все писали `std.fmt.bufPrint`, теперь его вынесли в `std.mem.print` (`lib/std/mem.zig:5209`), а старый оставили с пометкой Deprecated. Вот сам `print`:

```
pub const PrintError = error{
    NoSpaceLeft,
};

pub fn print(buffer: []u8, comptime format: []const u8, args: anytype) PrintError![]u8 {
    var w: std.Io.Writer = .fixed(buffer);
    w.print(format, args) catch |err| switch (err) {
        error.WriteFailed => return error.NoSpaceLeft,
    };
    return w.buffered();
}
```

Смотри как красиво: это просто обертка над `std.Io.Writer` с фиксированным буфером. Не влезло - получаешь ошибку `NoSpaceLeft`, а не молчаливое переполнение как `sprintf` в C.

А для динамического выделения - `Allocator.print` (`lib/std/mem/Allocator.zig:486`), который внутри сам резервирует память через `std.Io.Writer.Allocating`:

```
pub fn print(a: Allocator, comptime format: []const u8, args: anytype) Error![]u8 {
    var aw = try std.Io.Writer.Allocating.initCapacity(a, format.len);
    defer aw.deinit();
    aw.writer.print(format, args) catch |err| switch (err) {
        error.WriteFailed => return error.OutOfMemory,
    };
    return aw.toOwnedSlice();
}
```

То есть `fmt.bufPrint` и `fmt.allocPrint` в `lib/std/fmt.zig:600` и `:625` - это теперь тонкие Deprecated-обертки над `mem.print` и `Allocator.print`.

Где можно споткнуться

1. `==` для строк не работает. Попытка `a == b` на двух срезах дает ошибку компиляции:

```
error: operator == not allowed for type '[]const u8'
```

Это сделано намеренно: сравнение срезов неоднозначно (сравнивать указатели? содержимое?). Zig заставляет тебя ответить явно - `std.mem.eql(u8, a, b)` для содержимого.

2. `.len` - это байты, а не символы. Строка `"привет"` содержит 6 символов, но каждый кириллический символ в UTF-8 занимает 2 байта, поэтому `len` вернет 12:

```
const ru = "привет";
try out.interface.print("bytes: {d}\n", .{ru.len});              // 12
try out.interface.print("codepoints: {d}\n", .{try std.unicode.utf8CountCodepoints(ru)}); //
6
```

И срез `ru[0..2]` отрежет ровно один символ "п", а `ru[0..1]` отрежет половину символа и даст битый байт. Zig честно работает с байтами; если нужна работа с символами - `std.unicode`.

3. Литерал - это указатель, а не срез. `@TypeOf("hello")` это `*const [5:0]u8`. Пока ты не привел его к `[]const u8`, у него нет поля `.len` в привычном смысле - у указателя на массив длина известна из типа. Обычно это не мешает, потому что приведение неявное, но в сигнатурах функций нужно указывать срез, а не указатель.

4. Конкатенация `++` работает только на этапе компиляции. `"foo" ++ "bar"` сработает и даст литерал `"foobar"`, но это потому что оба аргумента известны компилятору. Рантайм-строки так не сложить - для этого `std.mem.concat` (в аллокатор) или `std.mem.print` в буфер:

```
var arena = std.heap.ArenaAllocator.init(std.heap.page_allocator);
defer arena.deinit();
const joined = try std.mem.concat(arena.allocator(), u8, &.{ a, b });
```

5. Старые туториалы ведут по устаревшим именам. `std.fmt.bufPrint` и `std.fmt.allocPrint` помечены Deprecated и теперь живут как `std.mem.print` и `Allocator.print`. Если копируешь пример из старой статьи и видишь `std.fmt.bufPrint` - знай, что в свежем Zig это `std.mem.print`.

6. Нет нулевого байта, если не попросил. Срез `[]const u8` ничего не знает про `\0`. Если отдаешь строку в C-функцию, нужен срез с сентинелом `[:0]u8`, иначе рискуешь утечкой чтения. Компилятор поможет: передать `[]const u8` туда, где ждут `[*:0]const u8`, не выйдет без явного приведения.

Пробуем руками

📝 **Задача:** открыть

✅ **Ответ:** открыть

4. Ошибки и их обработка

Зачем это нужно и в чем прикол

В C ошибка - это договоренность на словах: функция возвращает `-1`, а что это значит - смотри в комментарии или в глобальной `errno`. В Go - везде `if err != nil { return err }`, и половина кода это перекладывание ошибки наверх. В исключениях (Python, Java, C++) - неявный поток управления: ошибка может выпрыгнуть из любого места, и ты не видишь по сигнатуре, что функция вообще способна упасть.

Zig пошел третьим путем. Ошибка - это значение, не событие. Если функция может не справиться, это написано прямо в ее типе: `error{NotFound}!u8` - "вернет либо `u8`, либо ошибку из множества `{NotFound}`". Никакого скрытого прыжка через стек, никакого нулевого overhead на раскрутку.

Главный бонус - компилятор не дает ошибку замолчать. Вызов функции, которая возвращает ошибку, но результат которой ты просто отбросил, это ошибка сборки:

```
error: error union is discarded
note: consider using 'try', 'catch', or 'if'
```

Ты обязан явно решить, что делать с каждой ошибкой. Про "забыл проверить errno" можно забыть навсегда.

Базовый синтаксис

Объявляем свой набор ошибок - просто список имен:

```
const std = @import("std");

const MyError = error{
    NotFound,
    TooBig,
    Invalid,
};

fn mayFail(x: u8) MyError!u8 {
    if (x == 0) return error.NotFound;
    if (x > 100) return error.TooBig;
    return x * 2;
}
```

Тип `MyError!u8` называется error union (объединение с ошибкой). Это "или `u8`, или одна из ошибок". Возврат ошибки пишется `return error.NotFound` - через точку, как поле.

Теперь обработка. Три основных инструмента:

```zig
pub fn main(init: std.process.Init) !void {
    var buffer: [4096]u8 = undefined;
    var out = std.Io.File.stdout().writerStreaming(init.io, &buffer);

    // try: если ошибка - пробросить наверх из текущей функции, если успех - отдать значение
    const ok = try mayFail(10);
    try out.interface.print("ok={d}\n", .{ok});

    // catch со значением по умолчанию (тип обязан совпасть с payload)
    const def = mayFail(0) catch 0;
    try out.interface.print("default={d}\n", .{def});

    // catch |err|: поймать и посмотреть, что за ошибка
    const named = mayFail(0) catch |err| blk: {
        try out.interface.print("caught: {s}\n", .{@errorName(err)});
        break :blk 0;
    };
    _ = named;

    // if с двумя ветками: |value| для успеха, |err| для ошибки
    if (mayFail(200)) |value| {
        try out.interface.print("got {d}\n", .{value});
    } else |err| {
        try out.interface.print("else branch err={s}\n", .{@errorName(err)});
    }

    try out.flush();
}
```

`try` - это главный инструмент: "все норм - отдай значение, ошибка - пробрось наверх". Он сам распаковывает error union. Работает только внутри функции, которая тоже возвращает error union.

`@errorName(err)` дает имя ошибки строкой - удобно для логов. Ошибка - это полноценное значение, ее можно сравнивать и смотреть в `switch`:

```zig
const e: MyError = error.Invalid;
switch (e) {
    error.NotFound => try out.interface.print("is NotFound\n", .{}),
    error.TooBig => try out.interface.print("is TooBig\n", .{}),
    error.Invalid => try out.interface.print("is Invalid\n", .{}),
}
```

Ошибка занимает 0 бит в рантайме - она кодируется как целое число, которое передается в регистре, отдельно от payload. Про это - в разделе про сурсы.

Смотрим в сурсы: что там под капотом

Что такое error union с точки зрения компилятора? Заглянем в `lib/std/lang.zig:795` - там лежит структура, которую возвращает `@typeInfo`:

```zig
pub const ErrorUnion = struct {
    error_set: type,
    payload: type,
};
```

```
pub const ErrorSet = struct {
    error_names: ?[]const [:0]const u8,
};
```

Никакой магии. Error union - это пара "тип ошибки + тип полезной нагрузки". A error set - это просто список имен ошибок строкой. Обрати внимание на `error_names: ?...` - это опциональный указатель. Для `anyerror` (множество всех ошибок) он равен `null`, потому что перечислить все имена невозможно.

Почему ошибка ничего не весит? Потому что она кодируется целым числом, а не строкой и не объектом. `@errorName(err)` берет это число и смотрит его в таблице имен, которую компилятор кладет в бинарник. Вместе с самим значением ошибки по стеку передается один индекс.

Смотри, как выглядит типовой набор ошибок аллокатора в `lib/std/mem/Allocator.zig:14`:

```
pub const Error = error{OutOfMemory};
```

Одна ошибка. А чтобы собрать несколько источников ошибок в один тип, Zig использует оператор `||` - объединение множеств. Пример из `lib/std/crypto.zig:187`:

```
pub const KdfError = errors.Error || std.mem.Allocator.Error || std.Thread.SpawnError ||
std.Io.Cancelable;
```

Это буквально "множество всех ошибок из этих четырех наборов". То же самое сработает и у тебя: `MyError || error{OtherThing}` даст набор `error{NotFound, TooBig, Invalid, OtherThing}`.

Еще одна фишка под капотом - трейс ошибки. Функция `@errorReturnTrace()` возвращает стек вызовов в точке, где ошибка была создана. В `lib/std/lang.zig:11`:

```
pub const StackTrace = struct {
    index: usize,
    instruction_addresses: []usize,
};
```

В debug-сборке Zig бесплатно собирает цепочку адресов от места `return error.X` до точки обработки. Это не исключения, это просто запись адресов - она стоит почти ничего и полностью отключается в release-режиме.

Где можно споткнуться

1. `!T` - это не `anyerror!T`. Это самое частое недопонимание. Когда в сигнатуре функции написано просто `!u8`, это inferred error set - компилятор сам выводит множество ошибок из тела функции. А `anyerror!u8` - это "любая ошибка", явный широкий тип. Разница видна в имени типа:

```
@typeName(error{A, B}!u8)  // "error{A,B}!u8"
@typeName(anyerror!u8)     // "anyerror!u8"
```

Для публичных API принято писать конкретный набор, а не `anyerror` - так вызывающий код видит, чего ждать.

2. Ошибку нельзя отбросить молча. `_ = mayFail(5);` не скомпилируется - будет `error: error union is discarded`. И это правильно: в C именно такие отбросы и создают баги. Если совсем уверен, что ошибки не будет, пиши `catch unreachable` - но тогда при ошибке программа упадет с паникой.

3. Значение в `catch` обязано совпадать по типу с payload. `mayFail(0) catch 999` не соберется, если payload это `u8`, а 999 в `u8` не лезет. `catch` подменяет значение на тот же тип, что вернула бы функция в успешном случае.

4. `errdefer` - это не `defer`. Оба откладывают выполнение на выход из скоупа, но:

```
fn withErrdefer() !void {
    defer std.debug.print("normal defer\n", .{});
    errdefer std.debug.print("errdefer fired\n", .{});
    return error.Blah;
}
```

`defer` выполняется всегда, `errdefer` - только если функция выходит с ошибкой. Тут сработают оба, потому что `defer` не различает причину выхода. `errdefer` удобен для отката: выделил память - на успехе оставил, на ошибке освободил.

5. `@errorReturnTrace()` живет только в debug/safe-режиме. В `ReleaseFast` и `ReleaseSmall` он вернет `null`. Не закладывайся на него в логике, только в отладке.

6. Inferred error set делает функцию "общей" (generic). Если у двух функций в сигнатуре `!void` без явного набора, компилятор выводит набор для каждого вызова отдельно. Поэтому такие функции нельзя просто взять и положить в указатель на функцию с фиксированным типом - тип у них не один, а по одному на каждое место вызова.

Пробуем руками

📝 **Задача:** открыть

✅ **Ответ:** открыть

5. Условия if/else и optionals

Зачем это нужно и в чем прикол

В C и Python есть неявная правдивость: `if (x)` работает с чем угодно - с числами, указателями, строками. `0` ложен, `""` ложен, `NULL` ложен, и границы этой логики постоянно приводят к багам вроде "условие сработало на пустой строке".

В Zig `if` принимает ровно один тип - `bool`. Написал `if (5)` - получишь ошибку компиляции `expected type 'bool', found 'comptime_int'`. Никакой магии, никаких догадок: условие либо `true`, либо `false`.

Вторая тема - optionals. В C отсутствие значения выражается нулевым указателем `NULL`, и любая функция может внезапно вернуть `NULL`, а ты про это узнаешь только по сегфолту. В Zig отсутствие значения - это отдельный тип. `?u8` значит "либо `u8`, либо `null`", и `null` не свалится тебе на голову оттуда, где ты его не ждешь: обычный `u8` не может быть `null` в принципе.

Красота в связке этих двух идей - паттерн захвата. Вместо "проверь на null, потом достань значение":

```
if (opt != null) {
    const v = opt.?;  // так не делают
}
```

Zig пишет это одним движением:

```
if (opt) |v| {
    // v доступна здесь, и она гарантированно не null
}
```

Базовый синтаксис

Обычный if/else. Обрати внимание: `else` обязан стоять на той же строке, что и закрывающая скобка `}` - в Zig это синтаксическое требование:

```
const std = @import("std");

fn classify(n: i32) []const u8 {
    if (n > 0) {
        return "positive";
    } else if (n < 0) {
        return "negative";
    } else {
        return "zero";
    }
}
```

Для коротких веток без скобок все пишется в одну строку:

```
if (n > 0) return "positive" else if (n < 0) return "negative" else return "zero";
```

`if` в Zig - это выражение, оно возвращает значение:

```
const a = 10;
const label = if (a > 5) "big" else "small";
```

Теперь optionals. Объявляются знаком вопроса:

```
var maybe: ?u8 = null;   // сейчас значения нет
maybe = 42;              // а теперь есть
```

Три способа достать значение:

```
// 1. захват в if - значение видно только внутри ветки
if (maybe) |v| {
    std.debug.print("value: {d}\n", .{v});
} else {
    std.debug.print("null\n", .{});
}

// 2. orelse - подставить значение по умолчанию
const val = maybe orelse 0;

// 3. .? - принудительно развернуть; на null упадет с паникой
const unwrapped = maybe.?;
```

Паттерн захвата работает и в `while`, пока значение не станет `null`:

```
var counter: ?u8 = 3;
while (counter) |c| {
    std.debug.print("countdown {d}\n", .{c});
    counter = if (c > 0) c - 1 else null;
}
// напечатает 3, 2, 1, 0 и остановится
```

Смотрим в сурсы: что там под капотом

Optional с точки зрения компилятора - это предельно простая структура. В `lib/std/lang.zig:789`, куда `@typeInfo` кладет описание типа:

```
pub const Optional = struct {
    child: type,
};
```

Всего одно поле - тип вложенного значения. Больше ничего не нужно: `null` это отдельный специальный тег, а не значение вложенного типа.

Главное, что под капотом происходит с памятью. Возьми `?u8`:

```
@sizeOf(u8)   // 1
@sizeOf(?u8)  // 2
```

Почему 2 байта? Потому что `u8` использует все 256 значений от 0 до 255, и свободного "пустого" значения для `null` не остается. Компилятор добавляет скрытый тег-байт: 1 байт на признак "есть значение или null", 1 байт на сам `u8`.

А теперь `?*u8`:

```
@sizeOf(*u8)   // 8
@sizeOf(?*u8)  // 8
```

Тут тега нет вообще, оба по 8 байт. Почему? Потому что указатели и так имеют зарезервированное значение "ноль", которое не указывает ни на какую память. Компилятор просто использует нулевой указатель как представление `null` - это бесплатно, ноль байт накладных расходов. Та же оптимизация срабатывает и для `?enum`, когда у enum есть свободное значение.

Это и есть идея Zig: `null` - это явное значение, которое живет в типе, а не указатель на ноль, который можно спутать с чем угодно.

Где можно споткнуться

1. Нет правдивости. Условие обязано быть `bool`. Ни `if (5)`, ни `if (maybe)` не скомпилируются:

```
error: expected type 'bool', found '?u8'
```

Для optional нужно либо сравнение `if (maybe != null)`, либо паттерн захвата `if (maybe) |v|`. Между этими двумя вариантами предпочитай второй - он безопаснее, потому что не дает забыть про разворачивание.

2. `else` должен быть на одной строке с `}`. Разнесешь:

```
if (cond) {
    foo();
}
else {        // ошибка: expected statement, found 'else'
    bar();
}
```

Почему так - Zig не требует скобок вокруг условия и не ставит точку с запятой после `if`, поэтому перенос `else` на новую строку делает его отдельным statement. Держи `} else {` слитно.

3. `.?` на null - это паника. `.?` это не "проверь и продолжь", это "я на 100% уверен, что значение есть, разверни без проверок". На `null` получишь `panic: attempt to use null value` со стек-трейсом. Пользуйся им только там, где значение гарантированно есть, иначе - `orelse` или захват.

4. `orelse` обязан дать значение того же типа, что payload. `maybe orelse 0` работает, а `maybe orelse "нет"` - нет, если `maybe: ?u8`. Значение справа от `orelse` подставляется ровно в тип вложенного значения.

5. Optional - это не флаг. `?bool` это НЕ три состояния "да/нет/неизвестно". Это два уровня: внешний "есть/нет" и внутренний `true` / `false`. `?bool` реально занимает три логических состояния, но будь аккуратен - легко запутаться. А `??u8` (optional от optional) - это вообще "может быть null, может быть Some(null), может быть Some(Some(42))", что почти никогда не нужно.

Пробуем руками

📝 **Задача:** открыть

✅ **Ответ:** открыть

6. Циклы while и for

Зачем это нужно и в чем прикол

В Zig ровно два цикла - `while` и `for`. Никакого `do..while`, никакого сишного `for (i = 0; i < n; i++)` с тремя выражениями через точку с запятой. Два инструмента, каждый с четкой ролью.

`while` - это цикл с условием. Его фишка в отдельном выражении продолжения:

```
var i: u8 = 0;
while (i < 3) : (i += 1) {
    // тело
}
```

Условие `i < 3` и шаг `i += 1` разделены. Шаг выполняется после каждой итерации, включая итерацию, из которой ты сделал `continue`. Такой синтаксис убирает целый класс ошибок, когда в сишном `for` забываешь про `i++` в теле с `continue`.

`for` - это цикл по коллекции: массиву, срезу, диапазону. Индекс не надо считать руками, он всегда под рукой:

```
for (arr, 0..) |item, idx| {
    // item - элемент, idx - его индекс
}
```

А еще `for` умеет идти по нескольким последовательностям сразу, и циклы можно помечать метками, чтобы `break` / `continue` управляли нужным уровнем вложенности.

Базовый синтаксис

Простой `while`:

```
const std = @import("std");

pub fn main(init: std.process.Init) !void {
    var buffer: [4096]u8 = undefined;
    var out = std.Io.File.stdout().writerStreaming(init.io, &buffer);

    var i: u8 = 0;
    while (i < 3) : (i += 1) {
        try out.interface.print("i={d}\n", .{i});
    }
    // печатает 0, 1, 2

    try out.flush();
}
```

`while` умеет захватывать optional и error union - это мы уже видели в прошлых уроках:

```
var counter: ?u8 = 3;
while (counter) |c| {
    counter = if (c > 0) c - 1 else null;
}
// цикл крутится, пока значение не станет null
```

Теперь `for`. По массиву с индексом:

```
const arr = [_]u8{ 10, 20, 30 };
for (arr, 0..) |item, idx| {
    try out.interface.print("arr[{d}] = {d}\n", .{ idx, item });
}
```

По срезу (строке - по байтам):

```
for ("hi!") |c| {
    try out.interface.print("{c} ", .{c});
}
```

По нескольким последовательностям одновременно - итерации идут параллельно, длина по самой короткой:

```
const a = [_]u8{ 1, 2, 3 };
const b = [_]u8{ 4, 5, 6 };
for (a, b) |x, y| {
    try out.interface.print("{d}+{d}={d}\n", .{ x, y, x + y });
}
```

По диапазону `0..3` - это 0, 1, 2 (полуинтервал, правая граница не включается):

```
for (0..3) |n| {
    try out.interface.print("{d}\n", .{n});
}
```

Чтобы `for` изменял элементы, а не копировал их, итерируйся по указателю - с амперсандом:

```
var nums = [_]u8{ 1, 2, 3 };
for (&nums) |*item| {
    item.* *= 10;
}
// nums теперь { 10, 20, 30 }
```

`break` и `continue` работают как обычно, а для вложенных циклов есть метки:

```
const arr2 = [_]u8{ 10, 20, 30 };
outer: for (arr2) |x| {
    for (arr2) |y| {
        if (y == 20) continue :outer; // пропустить остаток внешней итерации
        if (x == 30) break :outer;    // выйти из обоих циклов
    }
}
```

И последнее - цикл может быть выражением, которое возвращает значение через `break`:

```
const result = blk: while (true) {
    break :blk 99;
};
// result == 99
```

Смотрим в сурсы: что там под капотом

Циклы - это фича языка, но в стандартной библиотеке полно живых примеров, как их пишут на практике. Возьми сравнение строк `std.mem.eql` из `lib/std/mem.zig:764` - там `for` идет по двум срезам сразу:

```
for (a, b) |a_elem, b_elem| {
    if (a_elem != b_elem) return false;
}
```

Два среза, одна пара элементов на итерацию, никаких ручных индексов. Это самый идиоматичный способ сравнивать две коллекции поэлементно.

А вот классический `while` с выражением продолжения в поиске подстроки `findPosLinear` из `lib/std/mem.zig:1469`:

```
var i: usize = start_index;
const end = haystack.len - needle.len;
while (i <= end) : (i += 1) {
    if (eql(T, haystack[i..][0..needle.len], needle)) return i;
}
return null;
```

Обрати внимание на `: (i += 1)` - шаг инкремента вынесен в заголовок, а тело занимается только поиском. Еще одна вариация - обратный проход в `findLastLinear` (`lib/std/mem.zig:1455`):

```
var i: usize = haystack.len - needle.len;
while (true) : (i -= 1) {
    if (mem.eql(T, haystack[i..][0..needle.len], needle)) return i;
    if (i == 0) return null;
}
```

Тут условие - просто `true` (бесконечный цикл), а выход сделан через `return`. Это стандартный прием, когда "условие выхода" живет внутри тела и его неудобно выносить в заголовок.

Где можно споткнуться

1. Нет сишного `for (init; cond; step)`. Забудь про `for (i = 0; i < n; i++)`. В Zig это либо `while (i < n) : (i += 1)`, либо `for (0..n) |i|`. Оба читаются лучше, потому что роли разнесены.

2. Переменная `for` неизменяема. `for (arr) |item|` дает копию элемента, и это `const` - `item *= 10` не скомпилируется (`cannot assign to constant`). Хочешь менять - итерируй по `&arr` и получай

`|*item|`, а внутри разыменовывай `item.*`.

3. `continue` не пропускает выражение продолжения. В `while (i < 5) : (i += 1)` вызов `continue` все равно выполнит `i += 1`. Это удобно (нет бесконечных циклов из-за забытого инкремента), но если шаг зависит от условия, учитывай это.

4. Диапазон `0..n` не включает `n`. `for (0..3) |i|` даст 0, 1, 2. Это полуинтервал `[0, 3)`, как в Python. Левая граница включается, правая - нет.

5. Нет `do..while`. Нужен цикл с проверкой в конце - пиши `while (true) { ... break; }` и ставь `break` там, где нужно выйти. Так же делают и в стандартной библиотеке, смотри пример выше.

6. Метки идут строго перед циклом. Синтаксис `outer: for (...)`, и в `break :outer` / `continue :outer` двоеточие обязательно. Без метки `break` выходит только из ближайшего цикла.

7. `for` по нескольким срезам идет до самого короткого. Если длины разные, лишние элементы длинного среза молча игнорируются. Нужно точное соответствие - проверяй длины заранее или используй вариант с проверкой.

8. Оператор `%` работает только с беззнаковыми. В теле цикла часто считаешь остаток от деления: `if (i % 3 == 0)`. Но если `i` знаковое (`i32`, `i64`), то `%` не скомпилируется - Zig требует явно указать, какой остаток нужен: `@rem` (знак от делимого) или `@mod` (знак от делителя). Для знаковых целых пиши `@rem(i, 3)`, для беззнаковых `%` годится.

Пробуем руками

📝 **Задача:** открыть

✅ **Ответ:** открыть

7. Функции

Зачем это нужно и в чем прикол

Функция в Zig - это явный контракт: тип каждого аргумента и тип возврата написаны в сигнатуре. Никакой неявной правдивости, никакого неявного приведения. Если функция вернула `i32`, она вернула `i32`, а не "что-то, что можно интерпретировать".

Три вещи, которые отличают Zig от C/C++ и Python:

Во-первых, параметры неизменяемы. Аргумент приходит как `const`, и попытка `x += 1` прямо в теле даст `cannot assign to constant`. Хочешь менять - заведи локальную переменную. Это убирает целый класс багов "ой, я поменял аргумент и вызывающий код про это не знает".

Во-вторых, нет неявного возврата последнего выражения. В Rust последнее выражение без точки с запятой становится результатом функции, в Zig - нет. Тут `return` обязателен, и это честно: ты видишь все точки выхода явно.

В-третьих, дженерики сделаны без отдельного синтаксиса. Никаких `<T>` и `template<>`. Просто параметр с `comptime` или `anytype`, и компилятор сам генерирует специализацию под конкретный тип в точке вызова.

Базовый синтаксис

Простая функция:

```
const std = @import("std");

fn add(a: i32, b: i32) i32 {
    return a + b;
}

fn noReturn(x: i32) void {
    _ = x; // void - функция ничего не возвращает
}
```

`fn` - ключевое слово, дальше имя, параметры в скобках, тип возврата после скобок, тело в фигурных. Чтобы функцию было видно из других файлов, помечай `pub`:

```
pub fn addPub(a: i32, b: i32) i32 {
    return a + b;
}
```

Параметр по умолчанию `const`. Чтобы его поменять, скопируй в локальную:

```
fn mutate(x: i32) i32 {
    var y = x; // локальная копия, можно менять
    y += 1;
    return y;
}
```

Теперь дженерики. Два способа. Первый - явный тип-параметр через `comptime`:

```
fn doubleIt(comptime T: type, value: T) T {
    return value * 2;
}

// вызов с конкретным типом
const a = doubleIt(u8, 7);   // 14, тип u8
const b = doubleIt(i32, 7);  // 14, тип i32
```

`comptime T: type` - это параметр, который известен на этапе компиляции и содержит тип. Под каждый вызов компилятор генерирует отдельную версию функции.

Второй - `anytype`, тип выводится сам:

```
fn identity(value: anytype) @TypeOf(value) {
    return value;
}
```

`anytype` значит "тип определяется из аргумента", а `@TypeOf(value)` возвращает этот тип как тип возврата.

`inline` - просит компилятор встроить тело функции в место вызова:

```
inline fn square(x: i32) i32 {
    return x * x;
}
```

И наконец - функция это значение. Ее можно положить в указатель и передавать:

```
fn double(x: i32) i32 {
    return x * 2;
}

fn applyTwice(f: *const fn (i32) i32, x: i32) i32 {
    return f(f(x));
}

pub fn main(init: std.process.Init) !void {
    var buffer: [4096]u8 = undefined;
    var out = std.Io.File.stdout().writerStreaming(init.io, &buffer);

    try out.interface.print("applyTwice: {d}\n", .{applyTwice(&double, 5)}); //
double(double(5)) = 20

    // указатель на функцию
    const fp: *const fn (i32, i32) i32 = &add;
    try out.interface.print("fp: {d}\n", .{fp(10, 20)});
```

```
    try out.flush();
}
```

Тип указателя на функцию: `*const fn (параметры) возврат`. Амперсанд `&double` берет адрес функции.

Смотрим в сурсы: что там под капотом

Как компилятор описывает функцию? Через `@typeInfo`, который возвращает структуру `Fn` из `lib/std/lang.zig:848`:

```
pub const Fn = struct {
    attrs: Attributes,
    is_generic: bool,
    /// `null` means the return type is generic, i.e. it depends on a function argument.
    return_type: ?type,

    /// A `null` element represents either an `anytype` parameter, or a parameter with a
generic
    /// type, i.e. where the type depends on a previous function argument.
    param_types: []const ?type,
    param_attrs: []const ParamAttributes,
    // ...
};
```

Обрати внимание на `return_type: ?type` и `param_types: []const ?type` - тип возврата и параметры опциональны. Для `anytype`-функции или когда возврат зависит от аргумента, эти поля `null`. Именно поэтому `identity(value: anytype) @TypeOf(value)` не имеет "одного" типа - у нее по типу на каждое место вызова. Компилятор помечает такую функцию `is_generic = true`.

Атрибуты функции - это в первую очередь calling convention, `lang.zig:164`:

```
pub const CallingConvention = union(enum(u8)) {
    auto,
    @"inline",
    // ... x86_64_sysv, x86_64_win, c, winapi и так далее
};
```

`auto` - стандартный для Zig (никаких гарантий по регистрам, только внутри компиляции), `@"inline"` - то, что делает ключевое слово `inline`, а `c` и `winapi` нужны для взаимодействия с сишными библиотеками.

Теперь посмотри, как дженерики выглядят в реальной стандартной библиотеке. `std.math.maxInt` из `lib/std/math.zig:1437`:

```
pub fn maxInt(comptime T: type) comptime_int {
    const info = @typeInfo(T).int;
    return (1 << (info.bits - @intFromBool(info.signedness == .signed))) - 1;
}
```

`comptime T: type` - тип известен при компиляции, значит и вся функция выполняется при компиляции, возвращая `comptime_int`. А `std.mem.print` из `lib/std/mem.zig:5209` использует `anytype` для аргументов форматирования:

```zig
pub fn print(buffer: []u8, comptime format: []const u8, args: anytype) PrintError![]u8 {
    // ...
}
```

Тут `format` - строка формата, известная на этапе компиляции, а `args` - кортеж любых значений. Комбинация `comptime format` + `anytype args` и есть стандартный паттерн для "переменное число аргументов любого типа".

Где можно споткнуться

1. Параметры неизменяемы. `fn f(x: i32) { x += 1; }` не соберется - `cannot assign to constant`. Это не ограничение, а защита: вызывающий код не ждет, что функция поменяет его переменную. Нужен результат - верни новое значение, нужен побочный эффект - передай указатель.

2. Нет неявного возврата. Написал в конце тела `x * 2` без `return` - получишь `expected ';' after statement`, потому что компилятор считает это выражением-оператором, а не возвратом. Точки выхода всегда явные.

3. Тип возврата обязателен. В отличие от многих языков, Zig не выводит тип возврата функции - его надо писать всегда. Это осознанное решение: сигнатура функции это публичный контракт, и он не должен зависеть от реализации в теле.

4. `anytype` порождает по функции на каждый тип. `fn id(x: anytype)` это не одна функция, а шаблон. Вызвал с `u8` и с `[]const u8` - получил две разные функции в бинарнике. Для маленьких утилит это ок, но для больших функций раздувает код. Если набор типов известен заранее - лучше `comptime T: type`.

5. Указатель на `inline`-функцию - только comptime. `inline`-функцию нельзя просто взять по адресу и положить в обычный указатель, потому что она семантически встроена в место вызова. Если функция должна быть указателем - не делай ее `inline`.

6. Типы указателей на функции строгие. `*const fn (i32) i32` и `*const fn (i64) i64` - разные типы. При передаче функции как аргумента сигнатуры должны совпадать в точности, никакого неявного приведения.

Пробуем руками

📝 **Задача:** открыть

✅ **Ответ:** открыть

8. Структуры

Зачем это нужно и в чем прикол

Структура - это способ собрать данные под одним именем: несколько полей с типами, плюс функции, которые с этими полями работают. В Zig нет классов и нет наследования - есть просто struct, внутри которого лежат и данные, и методы.

Главный прикол Zig - в том, как структура располагается в памяти. В C поля лежат строго в том порядке, в котором ты их объявил, и между ними зияют дыры выравнивания. В Zig по умолчанию компилятор сам переставляет поля, чтобы убрать лишний padding. Возьми структуру из трех полей `{ a: u8, b: u32, c: u8 }`:

- в C она заняла бы 12 байт (u8 + 3 байта дыры + u32 + u8 + 3 байта дыры);
- в Zig компилятор переставит поля в `b, a, c` и уложится в 8 байт.

Ты не платишь за то, что не используешь. А если нужен именно C-порядок (для FFI или файлового формата), ты говоришь об этом явно ключевым словом `extern`. Есть и третий режим - `packed`, когда поля упаковываются впритык, бит к биту.

Базовый синтаксис

Объявление и создание:

```
const std = @import("std");

const Point = struct {
    x: i32,
    y: i32,
};

pub fn main(init: std.process.Init) !void {
    var buffer: [4096]u8 = undefined;
    var out = std.Io.File.stdout().writerStreaming(init.io, &buffer);

    const p = Point{ .x = 3, .y = 4 };
    try out.interface.print("x={d} y={d}\n", .{ p.x, p.y });

    try out.flush();
}
```

Структура - это значение, ее тип описывается как `struct { ... }`. Экземпляр создается литералом `Point{ .x = 3, .y = 4 }` - поля через точку. Каждое поле без значения по умолчанию обязано быть заполнено, иначе ошибка компиляции `missing struct field`.

Поля со значениями по умолчанию:

```
const Player = struct {
    name: []const u8,
    hp: u32 = 100,
    score: u32 = 0,
};

const hero = Player{ .name = "hero" }; // hp и score берутся из значений по умолчанию
```

Методы - это обычные функции, объявленные внутри struct, с явным параметром `self`. Никакого неявного `this`:

```
const Point = struct {
    x: i32,
    y: i32,

    fn distance2(self: Point) i32 {
        return self.x * self.x + self.y * self.y;
    }
};
```

Вызываются через точку: `p.distance2()`. Хочешь менять поля внутри метода - принимай указатель:

```
const Counter = struct {
    count: u32 = 0,

    const Self = @This(); // @This() - тип текущей структуры

    pub fn incr(self: *Self) void {
        self.count += 1;
    }
};

var c = Counter{};
c.incr(); // c.count == 1
```

Тут два идиоматических момента: `const Self = @This()` избавляет от повторения имени структуры, а `*Self` дает методу мутировать экземпляр. Точка `c.incr()` сама передает указатель - компилятор видит, что параметр это `*Counter`, и подставляет `&c`.

Изменение полей напрямую работает только на `var`-экземпляре:

```
var p = Point{ .x = 3, .y = 4 };
p.x = 10; // ок

const q = Point{ .x = 1, .y = 2 };
// q.x = 5; // ошибка: cannot assign to constant
```

Смотрим в сурсы: что там под капотом

Как компилятор описывает структуру? Через `@typeInfo`, который возвращает `Struct` из `lib/std/lang.zig:751`:

```
pub const Struct = struct {
    is_tuple: bool,
    layout: ContainerLayout,
    /// Always `null` if `layout != .@"packed"`.
    backing_integer: ?type,

    field_names: []const [:0]const u8,
    field_types: []const type,
    field_attrs: []const FieldAttributes,

    decl_names: []const [:0]const u8,
    // ...
};
```

Поля раскладываются по полочкам: имена отдельно, типы отдельно, атрибуты отдельно. `is_tuple` различает именованные структуры и анонимные кортежи, а `layout` задает способ укладки. Три режима из `lang.zig:743`:

```
pub const ContainerLayout = enum(u2) {
    auto,
    @"extern",
    @"packed",
};
```

`auto` - компилятор сам оптимизирует порядок полей. `@"extern"` - C-порядок, поля как объявил, с обычным выравниванием. `@"packed"` - битовая упаковка впритык.

Значения по умолчанию хранятся в атрибутах поля - смотри `FieldAttributes` в `lang.zig:765`:

```
pub const FieldAttributes = struct {
    @"comptime": bool = false,
    @"align": ?usize = null,
    default_value_ptr: ?*const anyopaque = null,
    // ...
};
```

`default_value_ptr` - указатель на значение по умолчанию, стертый до `anyopaque`. Когда ты пишешь `hp: u32 = 100`, компилятор кладет `100` в бинарник и ставит сюда указатель на него. Поэтому при создании `Player{ .name = "hero" }` пропущенные поля заполняются из этого указателя.

Посмотрим укладку вживую. Одна и та же тройка полей `{ a: u8, b: u32, c: u8 }`:

```
const Auto = struct { a: u8, b: u32, c: u8 };
const External = extern struct { a: u8, b: u32, c: u8 };

@sizeOf(Auto)      // 8
@sizeOf(External)  // 12
```

Разница в 4 байта - это тот самый padding, который `auto`-режим убирает. А вот как `Auto` переставил поля - через `@offsetOf`:

```
@offsetOf(Auto, "a")  // 4
@offsetOf(Auto, "b")  // 0
```

```
@offsetOf(Auto, "c")  // 5
```

Поле `b` (4 байта, требует выравнивания 4) встало на смещение 0, а два байта `a` и `c` прижались к нему на смещения 4 и 5. Итог 8 байт. В `extern`-режиме порядок как в C: `a` на 0, три байта дыры, `b` на 4, `c` на 8, еще три байта дыры - 12 байт. Именно поэтому для FFI и бинарных форматов нужен явный `extern`, иначе полагаться на порядок полей нельзя.

А `packed` - это про битовую упаковку. Поля могут занимать доли байта:

```
const Flags = packed struct {
    a: u3,
    b: u1,
    c: u4,
};

@sizeOf(Flags)  // 1
```

Три поля в один байт: `u3` + `u1` + `u4` = ровно 8 бит. Это то, ради чего `packed` существует.

Где можно споткнуться

1. Пропустил поле - не соберется. Литерал `S{ .a = 1 }` для `struct { a: u8, b: u8 }` дает `error: missing struct field: b`. Исключение - только поля со значениями по умолчанию. Это ловит опечатки на этапе компиляции.

2. `const`-экземпляр неизменяем целиком. `const p = Point{...}; p.x = 5;` не выйдет - `cannot assign to constant`. Изменяемость структуры определяется тем, как ты объявил переменную: `var` - поля можно менять, `const` - нет. Это распространяется и на вложенные структуры.

3. В `auto`-режиме не полагайся на порядок полей. Смещения полей - дело компилятора. Если записываешь структуру в файл или отдаешь в C - используй `extern`. А если нужна максимально плотная упаковка (протоколы, регистры, флаги) - `packed`.

4. У метода нет неявного `this`. В отличие от C++/Python, получатель пишется явно первым параметром: `fn area(self: Rectangle) u32`. Без этого параметра функция - просто свободная функция внутри namespace структуры, а не метод.

5. Нет классов и наследования. Не ищи `class`, `extends`, виртуальные методы. Композиция через вложенные структуры и указатели. Полиморфизм в Zig делается иначе - через `union(enum)` и `@fieldParentPtr`, но это отдельная тема.

6. `packed`-структуры медленнее в работе с отдельными полями. Поле в `packed`-структуре может не попадать в границы байта, и чтение требует битовых операций. `packed` - это про компактность хранения, а не про скорость доступа.

Пробуем руками

📝 **Задача:** открыть

✅ **Ответ:** открыть